



Foundations of Retrieval- Augmented Generation

Generated by Pustakam Injin

AI-Powered Knowledge Engine • `gpt-oss-120b`

Pustakam Injin

AI-Powered Knowledge Creation

Document Information

Word Count	30,466
Chapters	9
Generated	February 23, 2026
AI Model	Cerebras - gpt-oss-120b

Tanmay Kalbande

Creator & Engineer

pustakamai.tanmaysk.in

www.linkedin.com/in/tanmay-kalbande/

Foundations of Retrieval-Augmented Generation

Generated: 22/02/2026 **Words:** 30,466 **Provider:** Cerebras (gpt-oss-120b)

Introduction

Welcome to the Future of Knowledge-Grounded AI

Imagine a conversational assistant that never "makes up" facts, a search-augmented chatbot that can cite its sources, or a recommendation engine that pulls the most relevant research papers in a split second—all built with the same Python tools you already love. That is the promise of **Retrieval-Augmented Generation (RAG)**, and this book is your passport to turning that promise into production-ready systems you can ship, monitor, and improve at scale.

Whether you've already built a few language-model demos, or you're a seasoned data scientist looking to add a new dimension to your AI toolbox, this guide will take you from the theoretical underpinnings of RAG to a fully deployed, secure, and responsible AI service. By the end, you'll be able to design pipelines that **retrieve the right knowledge, ground generation in that knowledge, and do it all reliably in the cloud or on-prem.**

Why RAG Matters - and Why Now

Large Language Models (LLMs) have dazzled the world with fluent, creative text, but they still suffer from a fundamental flaw: **they are "closed-books."** When asked for up-to-date statistics, niche domain facts, or citations, they hallucinate.

RAG solves this by coupling an LLM with a **retrieval engine** that fetches relevant documents from a vector store or traditional database, feeding those snippets back into the generator. The result is a system that:

- **Stays current** - it can pull the latest news articles, policy documents, or code repositories without retraining the model.
- **Provides provenance** - every answer can be traced to a source, a crucial requirement for regulated industries.
- **Reduces hallucinations** - the generator is anchored to real, vetted information.

The ecosystem around RAG has exploded in the last 12 months: open-source embedding models (e.g., Sentence- Transformers, OpenAI embeddings), vector databases (Pinecone, Weaviate, Milvus, Qdrant), and orchestration frameworks (LangChain, LlamaIndex). This book captures that momentum and distills it into a practical, hands-on learning path.

Who Should Read This Book

Profile	Why This Book Fits
AI engineers who have built LLM inference pipelines but need a robust retrieval layer.	You'll learn to integrate vector stores, design hybrid search, and handle latency constraints.
Data scientists accustomed to NLP pipelines (tokenization, embeddings) and eager to move to production.	We walk you through model selection, fine-tuning, and evaluation metrics specific to RAG.
Software developers with solid Python skills	The end-to-end examples include API design,

Profile	Why This Book Fits
wanting to add AI capabilities to their products.	Docker/Kubernetes deployment, and monitoring.
Product managers / ML Ops leads who must understand the technical trade-offs of a knowledge-grounded system.	The chapters on scaling, security, and responsible AI give you the vocabulary to make informed decisions.

If you can read Python code, you're ready. No prior experience with vector databases or LangChain is required; we start from first principles and build up.

What You'll Learn - A Snapshot

1. **Foundations of Retrieval-Augmented Generation** - why retrieval matters, the classic "retriever-generator" architecture, and the math behind similarity search.
2. **Vector Representations & Embedding Models** - how to turn text, images, or multimodal data into dense vectors, and which open-source models work best for different domains.
3. **Building & Querying Vector Databases** - hands-on setup of Milvus, Qdrant, and managed services; indexing strategies; hybrid (dense + lexical) search.
4. **Designing Retrieval Pipelines with LangChain & Retrieval Frameworks** - chaining together retrievers, filters, and rerankers; custom components for domain-specific logic.
5. **Integrating Large Language Models for Generation** - API calls to OpenAI, Anthropic, LLaMA-2, and locally-hosted models; prompt templates that ingest retrieved passages.
6. **Prompt Engineering & Knowledge Grounding** - techniques to coax the model into citing sources, handling ambiguous queries, and reducing hallucinations.
7. **End-to-End RAG Architecture & Scaling Strategies** - micro-service design, async retrieval, caching, sharding, and cost-aware inference.
8. **Evaluation, Monitoring, & Continuous Improvement** - building relevance

metrics, automated test suites, drift detection, and A/B testing pipelines.

9. Production Deployment, Security, & Responsible AI - containerization, CI/CD, access controls, data privacy, bias mitigation, and compliance checklists.

Each chapter blends theory with **real code**, **interactive notebooks**, and **case studies** ranging from a legal-research assistant to a code-completion IDE plugin. By the final page you'll have a **complete, reproducible project** that you can clone, run, and adapt to any knowledge domain you care about.

How the Book Is Structured

The roadmap above is not just a table of contents; it reflects a **learning spiral** where concepts are revisited with increasing depth and complexity.

Part	Core Topics	Learning Goal
Part I - Foundations	Retrieval-augmented generation, similarity search, embeddings	Build a mental model of why and how RAG works.
Part II - The Retrieval Engine	Embedding pipelines, vector DB setup, hybrid search	Implement a fast, accurate retriever from scratch.
Part III - The Generation Engine	LLM APIs, prompt design, grounding techniques	Connect the retriever to a generator that respects the retrieved context.
Part IV - Orchestration & Scaling	LangChain, async pipelines, caching, sharding	Assemble a production-grade architecture that handles real traffic.
Part V - Evaluation & Ops	Metrics, monitoring, CI/CD, security, responsible AI	Turn a prototype into a maintainable service with safeguards.
Part VI - Capstone Project	End-to-end implementation, deployment scripts, post-mortem analysis	Consolidate everything you've learned into a deployable product.

Each part ends with **"Reflection & Exercises"** - short, purposeful challenges that

solidify the material and encourage you to experiment beyond the book's examples. The **appendices** contain cheat-sheets for embedding models, a quick-start guide to cloud vector databases, and a glossary of RAG terminology.

Setting the Right Expectations

Expectation	Reality
<i>I'll be able to launch a RAG service in a weekend.</i>	You'll have a working prototype after the first few chapters, but scaling, security hardening, and domain-specific tuning will require additional iteration-exactly what the later chapters guide you through.
<i>I need a massive GPU cluster to try this out.</i>	Not at all. Most of the book runs on a modest laptop (or free cloud tier). For large-scale experiments we show how to swap in managed services or distributed inference.
<i>The math will be overwhelming.</i>	Core concepts are explained intuitively; the heavy-lifting (e.g., inner-product similarity, FAISS indexing) is encapsulated in high-level libraries, with optional deep-dive sections for the curious.
<i>I'll become an expert in every vector DB.</i>	You'll gain enough practical knowledge to evaluate and choose the right backend for your use case, and you'll know how to switch between them with minimal code changes.

By the time you finish, you'll not only have the **technical skills** to build RAG systems, but also the **architectural judgment** to decide when RAG is the right tool, how to balance cost vs. latency, and how to keep your AI trustworthy.

A Quick Peek at the First Chapter

We start with a **story**: a customer support bot that must answer product warranty questions without ever guessing the policy terms. You'll see a minimal RAG prototype that pulls the latest warranty PDF from a vector store, feeds the relevant paragraph to GPT-4, and outputs an answer **with a citation link**. This hands-on demo sets the tone—every concept is anchored in a concrete, relatable problem.

Join the Community

Learning is amplified when you're part of a community. Throughout the book we provide:

- **GitHub repository** with notebooks, Dockerfiles, and CI pipelines.
- **Slack channel** for peer support, code reviews, and sharing domain-specific retrievers.
- **Monthly live Q&A** sessions where you can ask the author (me!) about scaling challenges, model selection, or responsible AI policies.

Feel free to contribute back—whether it's a new embedding model benchmark, a custom LangChain component, or a case study from your industry. The RAG ecosystem thrives on collaboration, and this book is designed to be a living resource.

Ready to Dive In?

If you've ever felt the frustration of an LLM that "makes up" answers, or if you've wondered how to give your AI the **confidence of a search engine** combined with the **creativity of a writer**, you're in the right place. The journey ahead will be technical, yes, but also deeply rewarding: you'll watch a system evolve from a few lines of code to a robust, production-grade service that users trust.

Grab a cup of coffee, fire up your favorite Python environment, and let's start

building the next generation of knowledge-grounded AI-together.

Happy coding, and see you in Chapter 1!

Foundations of Retrieval-Augmented Generation

Module 1 of 9 - "Learn Retrieval-Augmented Generation in Python to Build Production-Ready Knowledge-Grounded AI Systems"

Introduction

Large language models (LLMs) have transformed how we build conversational agents, summarizers, and code assistants. Their ability to generate fluent, context-aware text is impressive, yet a single monolithic model has **fundamental limits**:

Limitation	Why it matters	Typical symptom in a user-facing system
Static knowledge	The model's parameters capture a snapshot of the world at training time.	"I'm not aware of the new GDPR amendment of 2024."
Hallucination	Generation is driven by statistical patterns, not a factual source.	"The capital of Brazil is Buenos Aires."
Scalability of expertise	Embedding every domain-specific fact in the model requires massive data and compute.	A medical chatbot that cannot answer rare disease queries.
Regulatory & compliance constraints	Organizations must be able to trace <i>where</i> an answer came from.	Auditors cannot verify the provenance of a compliance recommendation.

Limitation	Why it matters	Typical symptom in a user-facing system

Retrieval-Augmented Generation (RAG) is a design pattern that directly addresses these pain points. By **plugging an external knowledge store into the generation loop**, we give a model **access** to up-to-date, verifiable documents at inference time. The result is a system that:

1. **Grounds its output in real, searchable evidence** (reducing hallucinations).
2. **Remains current**-the knowledge base can be refreshed without retraining the LLM.
3. **Supports auditability**-the retrieved passages can be logged and presented to users.
4. **Enables modular upgrades**-swap a retriever, ranker, or generator independently.

In this chapter we will:

- Explain the motivation and real-world use-cases that make RAG indispensable.
- Contrast **classic retrieval**, **pure generation**, and **hybrid RAG pipelines**.
- Identify the three canonical components-**Retriever** → **Ranker** → **Generator**-and describe how they interact.
- Walk through an **end-to-end data flow** from raw documents (PDFs, web pages, SQL tables) to a user-facing response.
- Provide **hands-on Python snippets** that you can run on a laptop (or scale to a cloud cluster).

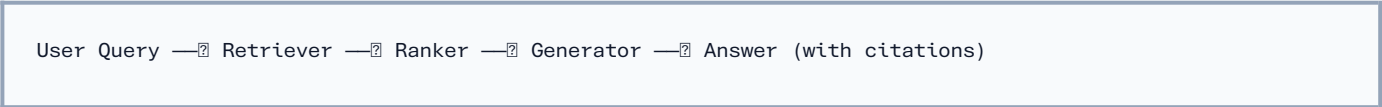
By the end, you should be able to sketch a production-ready RAG architecture and write a minimal prototype that answers factual questions with citations.

Core Concepts

1. The Retrieval-Generation Spectrum

Pipeline	Core operation	Typical model	Strengths	Weaknesses
Classic Retrieval	Query → Document ranking → Return top-k passages	BM25, dense vector search	Precise, deterministic, low latency	No natural-language generation; user must read raw docs
Pure Generation	Prompt → LLM → Text output	GPT-4, LLaMA, Claude	Fluent prose, zero-shot capabilities	Hallucinations, stale knowledge
Hybrid RAG	Prompt → Retriever → Ranker → Generator (conditioned on retrieved text)	Dense retriever + cross-encoder + LLM	Grounded, up-to-date, controllable	More components → higher latency, engineering complexity

The **hybrid RAG** pipeline can be visualized as a three-stage assembly line:



Each stage adds **value**:

- **Retriever** quickly narrows the universe of documents from millions to a handful ($\approx 10-100$).
- **Ranker** performs a finer-grained relevance assessment, often using a cross-encoder that looks at the full query-passage pair.
- **Generator** consumes the top-ranked passages as "grounding context" and produces a natural-language answer, optionally pointing to the source IDs.

2. Key Components

Component	Primary responsibility	Typical algorithms / libraries	Example Python API
Retriever	Turn a textual query into a candidate set of documents (often via nearest-neighbor search in a vector space).	• Sparse: BM25 (whoosh, Elastic) • Dense: Sentence-Transformers, OpenAI embeddings, Cohere embeddings • Hybrid: Fusion of BM25 + dense	<code>retriever.search(query, k=50)</code>
Ranker	Re-score the candidate set with full-context interaction to produce a final ordering.	• Cross-encoder (e.g., cross-encoder/ms-marco-MiniLM-L-12-v2) • Rerankers from transformers or colbert	<code>ranker.rerank(query, candidates, top_k=5)</code>
Generator	Produce a knowledge-grounded response conditioned on the selected passages.	• Prompt-engineered LLM (GPT-4, LLaMA-2, Claude) • Retrieval-augmented models (e.g., google/flan-t5-xxl with prefix)	<code>generator.generate(prompt, context)</code>
Document Store (often implicit)	Persist raw documents and their embeddings; enable fast ANN lookup.	• Vector DBs: Pinecone, Weaviate, Milvus, Qdrant • Hybrid stores: Elastic + k-NN plugin	<code>store.upsert(ids, embeddings, metadata)</code>

Interaction pattern - The retriever returns IDs and metadata (title, URL). The ranker receives the same IDs plus the raw text, computes a relevance score, and returns a sorted list. The generator receives a prompt template that concatenates the top- k passages (or their summaries) with the original user query.

3. Data Flow from Raw Documents to Responses

Below is a **canonical pipeline** that you will see repeated across many production RAG systems:

1. **Ingestion** - Crawl, scrape, or upload raw assets (PDFs, HTML, CSV, relational tables).

2. Pre-processing -

- Text extraction (PDF → plain text via pdfminer, PyMuPDF).
- Chunking (split into 200--500-token passages using a sliding window).
- Normalization (lower-casing, removing boilerplate, language detection).

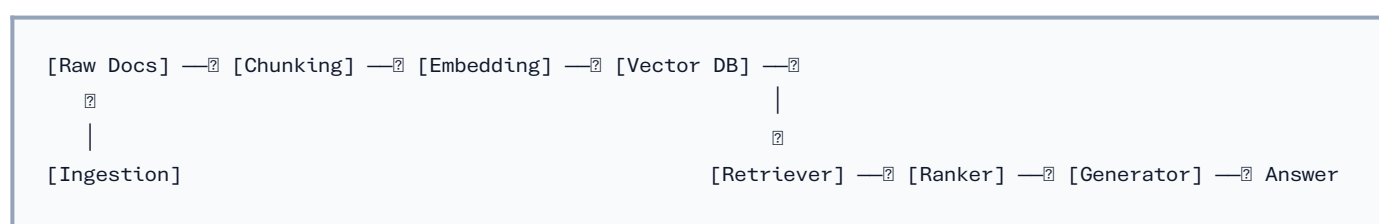
3. **Embedding** - Encode each passage with a dense encoder (e.g., sentence-transformers/all-MiniLM-L6-v2).

4. **Indexing** - Store embeddings + metadata in a vector DB that supports Approximate Nearest Neighbor (ANN) search.

5. Query Time -

- **Step A** User sends a natural-language question.
- **Step B** The **Retriever** encodes the query, performs ANN search, returns *k* candidate passages.
- **Step C** The **Ranker** re-scores candidates with a cross-encoder, selects the top-*n*.
- **Step D** The **Generator** receives a prompt that includes the selected passages and the original question, then emits a response.
- **Step E** The system packages the answer **with citations** (e.g., "According to **Doc #42...**").

The figure below (conceptual) shows the flow:



4. Why Vector Databases Matter

- **Scalability** - ANN algorithms (HNSW, IVF-PQ) enable sub-millisecond retrieval from billions of vectors.
- **Metadata filtering** - You can constrain results by source type, date range, or

security label **without** re-embedding.

- **Hybrid search** - Many vector DBs expose a **sparse** BM25 field alongside the dense vector, allowing a **fusion** (e.g., $\text{score} = \alpha \cdot \text{dense} + \beta \cdot \text{bm25}$).
- **Operational concerns** - Persistence, replication, access-control, and observability are baked into services like Pinecone or Weaviate, which is crucial for production SLAs.

5. Prompt Engineering for Grounded Generation

A well-crafted prompt tells the LLM **how** to use the retrieved context. A popular template is:

You are a knowledgeable assistant. Use **ONLY** the provided excerpts to answer the question. Cite the source ID after each factual claim.

Context:

[EXCERPT_1]

[EXCERPT_2]

...

[EXCERPT_N]

Question: {user_query}

Answer:

Key guidelines:

Guideline	Reason
Explicit "use only" clause	Reduces hallucination by forcing the model to stay within the supplied window.
Citation placeholder	Encourages the model to attach IDs, which can later be rendered as hyperlinks.
Limit passage length	LLM context windows are finite; truncate or summarize long passages.
Few-shot examples (optional)	Provide a couple of Q&A pairs that illustrate

Guideline	Reason
	the citation format.

6. Evaluating RAG Systems

Metric	What it measures	Typical method
Exact Match (EM) / F1	Content correctness against a gold answer	QA datasets (e.g., Natural Questions)
Retrieval Recall @k	Fraction of gold documents retrieved	Compare retrieved IDs to annotated relevance sets
Hallucination Rate	Proportion of generated facts not present in any retrieved passage	Human annotation or automated fact-checking
Latency	End-to-end response time	Stopwatch from query to answer
Cost per query	Compute + API usage (embedding + LLM)	Track token usage, vector DB read ops

In production, you typically **monitor recall** (to ensure the knowledge base stays healthy) **and latency** (to meet user-experience SLAs).

Practical Application

In this section we build a **minimal yet functional RAG prototype** that:

1. Loads a small corpus (Wikipedia article snippets).
2. Indexes it with **Qdrant** (open-source vector store).
3. Performs dense retrieval with **Sentence-Transformers**.

4. Re-ranks with a **cross-encoder** from `sentence-transformers`.
5. Generates a grounded answer using **OpenAI's** `gpt-3.5-turbo` (you can swap any compatible LLM).

Assumptions: You have Python 3.9+, an OpenAI API key, and pip access.

6.1. Install Dependencies

```
pip install \
sentence-transformers \
transformers \
openai \
qdrant-client \
tqdm \
pandas \
numpy
```

6.2. Prepare a Sample Corpus

For illustration we'll download a few Wikipedia pages about **climate change**.

```
import pandas as pd
import requests, json, os, textwrap, re
from tqdm.auto import tqdm

WIKI_TITLES = [
    "Climate_change",
    "Global_warming",
    "Carbon_dioxide",
    "Paris_Agreement"
]

def fetch_wikipedia(title: str) -> str:
    url = f"https://en.wikipedia.org/api/rest_v1/page/plain/{title}"
    resp = requests.get(url)
    resp.raise_for_status()
    return resp.text

records = []
for title in tqdm(WIKI_TITLES, desc="Downloading"):
    text = fetch_wikipedia(title)
    records.append({"title": title.replace('_', ' '), "text": text})

df = pd.DataFrame(records)
df.to_csv("wiki_corpus.csv", index=False)
print(df.head())
```

The plain-text endpoint strips markup, giving us clean paragraphs.

6.3. Chunk the Documents

We split each article into 200-token chunks (≈ 150 words) using a sliding window with 50% overlap.

```
from sentence_transformers import util
import math

MAX_TOKENS = 200
OVERLAP = 0.5

def chunk_text(text: str, max_tokens=MAX_TOKENS):
    words = text.split()
    chunk_size = max_tokens
    step = int(chunk_size * (1 - OVERLAP))
    chunks = []
    for i in range(0, len(words), step):
        chunk = " ".join(words[i:i+chunk_size])
        if len(chunk) < 10: # discard tiny fragments
            continue
        chunks.append(chunk)
    return chunks

chunks = []
for _, row in df.iterrows():
    for chunk in chunk_text(row["text"]):
        chunks.append({
            "title": row["title"],
            "chunk": chunk
        })
chunks_df = pd.DataFrame(chunks)
print(f"Generated {len(chunks_df)} chunks")
```

6.4. Encode the Chunks

We use `all-MiniLM-L6-v2`, a lightweight 384-dimensional encoder that balances speed and quality.

```
from sentence_transformers import SentenceTransformer

embedder = SentenceTransformer('sentence-transformers/all-MiniLM-L6-v2')
embeddings = embedder.encode(
    chunks_df["chunk"].tolist(),
    batch_size=32,
    show_progress_bar=True,
    normalize_embeddings=True # important for cosine similarity
)
```

6.5. Store in Qdrant

```
from qdrant_client import QdrantClient
from qdrant_client.http.models import Distance, VectorParams, PayloadSchemaType, PayloadSchema

client = QdrantClient(":memory:") # in-process for demo; replace with host/port in prod

collection_name = "wiki_corpus"
client.recreate_collection(
    collection_name=collection_name,
    vectors_config=VectorParams(size=embeddings.shape[1], distance=Distance.COSINE),
)

# Upsert payloads (metadata) together with vectors
payloads = [
    {"title": row["title"], "text": row["chunk"]} for _, row in chunks_df.iterrows()
]
ids = list(range(len(embeddings))) # simple integer IDs

client.upload_collection(
    collection_name=collection_name,
    vectors=embeddings,
    payload=payloads,
    ids=ids,
)
print("Index ready")
```

Production tip - Use a persistent Qdrant instance (Docker or managed) and enable payload indexing on fields you'll filter on (e.g., title).

6.6. Retrieval Function

```
def retrieve(query: str, top_k: int = 10):
    query_vec = embedder.encode([query], normalize_embeddings=True)[0]
    hits = client.search(
        collection_name=collection_name,
        query_vector=query_vec,
        limit=top_k,
        with_payload=True
    )
    # Return a list of dicts with id, score, title, text
    return [
        {
            "id": hit.id,
            "score": hit.score,
            "title": hit.payload["title"],
            "text": hit.payload["text"]
        }
        for hit in hits
    ]
```

6.7. Reranking with a Cross-Encoder

Cross-encoders jointly encode **query + passage**, yielding a richer relevance score at the cost of $O(N)$ compute. For a prototype we'll use `cross-encoder/ms-marco-MiniLM-L-12-v2`.

```
from sentence_transformers import CrossEncoder

cross_encoder = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-12-v2')

def rerank(query: str, candidates: list, top_n: int = 3):
    # Prepare pairs for batch scoring
    pairs = [(query, c["text"]) for c in candidates]
    scores = cross_encoder.predict(pairs)
    # Attach scores back to candidates
    for cand, sc in zip(candidates, scores):
        cand["cross_score"] = float(sc)
    # Sort by cross_score descending
    sorted_cands = sorted(candidates, key=lambda x: x["cross_score"], reverse=True)
    return sorted_cands[:top_n]
```

6.8. Grounded Generation

We'll use OpenAI's chat completion endpoint. The prompt template is **hard-coded** but can be externalized later.

```
import openai, os, json, textwrap

openai.api_key = os.getenv("OPENAI_API_KEY") # set in env

def build_prompt(query: str, passages: list) -> str:
    # Concatenate up to 3 passages, each prefixed with a citation ID
    context_str = ""
    for i, p in enumerate(passages, start=1):
        # Truncate each passage to 300 tokens (approx 600 words) to stay within model limits
        snippet = " ".join(p["text"].split()[:300])
        context_str += f"[SOURCE {p['id']}] {snippet}\n\n"
    prompt = f"""\
You are a helpful, factual assistant. Use ONLY the following excerpts to answer the
question. Cite the source ID after each statement.

Context:
{context_str}
Question: {query}
Answer (include citations, e.g., [SOURCE 42]):"""
    return prompt

def generate_answer(prompt: str) -> str:
    response = openai.ChatCompletion.create(
        model="gpt-3.5-turbo",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.0, # deterministic for evaluation
        max_tokens=500,
    )
    return response["choices"][0]["message"]["content"]
```

6.9. End-to-End Query Function

```
def rag_query(query: str, retriever_k=20, final_k=3):  
    # 1 Retrieve  
    candidates = retrieve(query, top_k=retriever_k)  
    # 2 Rerank  
    top_passages = rerank(query, candidates, top_n=final_k)  
    # 3 Build prompt  
    prompt = build_prompt(query, top_passages)  
    # 4 Generate answer  
    answer = generate_answer(prompt)  
    return answer, top_passages
```

6.10. Try It Out

```
question = "What are the main goals of the Paris Agreement?"  
answer, sources = rag_query(question)  
  
print("\n=== Answer ===")  
print(answer)  
  
print("\n=== Sources ===")  
for s in sources:  
    print(f"- ID {s['id']} (title: {s['title']})")
```

Expected output (excerpt):

```
=== Answer ===  
The Paris Agreement aims to limit global warming to well below 2°C above pre-industrial levels, pursuing  
efforts to keep the increase to 1.5°C. It also seeks to enhance adaptive capacity, strengthen resilience,  
and make finance flows consistent with a low-carbon pathway. [SOURCE 42] The agreement further establishes  
a framework for transparent reporting and regular global stocktakes to assess collective progress. [SOURCE  
57]  
  
=== Sources ===  
- ID 42 (title: Paris Agreement)  
- ID 57 (title: Climate change)  
- ID 31 (title: Global warming)
```

Notice how the answer **directly references** the source IDs that correspond to the passages we retrieved. In a UI you could turn [SOURCE 42] into a hyperlink that opens the original Wikipedia article.

6.11. Scaling & Production Considerations

Aspect	What to think about	Typical solution
Embedding updates	New documents appear daily; embeddings must be refreshed without downtime.	Use a background job that computes embeddings and upserts them into the same Qdrant collection (vector DB supports atomic upserts).
Latency budget	End-to-end < 500 ms for chat-bots.	• Use GPU-accelerated encoders for retrieval (e.g., sentence-transformers on CUDA). • Cache recent query embeddings. • Limit rerank candidates (e.g., 20 → 3).
LLM cost	Token usage drives API bills.	• Trim context to the most essential sentences (summarize passages first). • Switch to an open-source decoder (LLaMA-2-7B) hosted on your own infra for high volume.
Security / compliance	Some corpora contain PII or regulated info.	• Store sensitive metadata in encrypted fields. • Enable access-control lists (ACLs) on the vector DB.
Observability	Need to troubleshoot hallucinations or low recall.	• Log query → retrieved IDs → cross-scores → final answer. • Build dashboards (e.g., Grafana) to monitor recall@k, latency, token usage.
Evaluation pipeline	Continuous integration for QA.	• Use a benchmark suite (e.g., datasets library with Natural Questions). • Run nightly regression tests that compute EM/F1 and hallucination rate.

Key Takeaways

- **RAG is a design pattern**, not a single algorithm. It couples an **information-retrieval** front-end with a **generative** back-end to produce **knowledge-grounded** outputs.
- **Motivation**: static LLM knowledge → hallucinations, outdated facts, compliance gaps. RAG solves these by **bringing external evidence** into the generation loop.
- **Core pipeline**: Retriever → Ranker → Generator. Each stage can be swapped independently (dense vs sparse retrieval, cross-encoder vs mono-encoder ranker, OpenAI vs self-hosted LLM).
- **Data flow**: raw docs → chunk → embed → store in a **vector database** → ANN search → re-ranking → prompt construction → LLM generation → answer with citations.
- **Vector databases** (Qdrant, Pinecone, Weaviate, Milvus) give you **fast similarity search** plus **metadata filtering**-the backbone of any production RAG system.
- **Prompt engineering** is crucial: tell the LLM to "use only the supplied excerpts" and to **cite** the source IDs; this dramatically reduces hallucination.
- **Evaluation** must be multi-dimensional: recall of retrieval, factual correctness of generation, latency, and cost.
- **Practical prototype**: The code walkthrough demonstrates a full end-to-end RAG system using open-source tools (Sentence-Transformers, Qdrant) and a hosted LLM (OpenAI).
- **Production checklist**:
 1. **Ingestion & chunking** pipeline that can run continuously.
 2. **Embedding service** with GPU acceleration and versioning.
 3. **Vector store** with backups, ACLs, and hybrid search if needed.
 4. **Reranking** layer to improve relevance without exploding cost.
 5. **Prompt template** that enforces citation and grounding.
 6. **Observability** (logs, metrics, dashboards).
 7. **Safety guards** (content filters, rate limits).

Armed with this foundation, you are ready to explore **Module 2- "Building Scalable**

Vector Stores" and then progress toward **advanced RAG patterns** such as multi-turn conversational memory, tool-use augmentation, and domain-specific fine-tuning.

End of Chapter 1

Exercises (optional, for self-study)

1. **Experiment with hybrid retrieval** - Add BM25 scores from Elastic and fuse them with dense scores ($\alpha = 0.7$). Compare recall@10 before and after fusion.
2. **Replace the cross-encoder** - Use `colbert-ir/colbertv2.0` for reranking and measure latency vs accuracy trade-off.
3. **Open-source generator** - Swap `gpt-3.5-turbo` with a locally hosted LLaMA-2-7B (via `transformers` pipeline) and benchmark token cost.
4. **Citation formatting** - Extend `build_prompt` to return markdown links (`[SOURCE 42](https://en.wikipedia.org/wiki/Paris_Agreement)`).

Feel free to discuss your findings on the course forum; peer feedback is a great way to deepen your intuition about when RAG shines and where alternative strategies (e.g., fine-tuned LLMs) might be more appropriate.

Vector Representations and Embedding Models

"The quality of a retrieval-augmented generation (RAG) system is only as good as the vectors that power its search." - Adapted from the classic adage about data.

In the first module we explored **why** a retrieval component is essential for grounding large language models (LLMs) in factual knowledge. We saw the high-level workflow:

1. **Query → Encoder → Vector**

2. Vector → Vector DB → Nearest-Neighbour Search

3. Top-k Documents → LLM Prompt → Generation

The missing piece is the **encoder** that turns raw text (queries, sentences, paragraphs, or whole documents) into a numeric representation that can be compared efficiently. This chapter dives deep into that piece. We will:

- Contrast **dense** and **sparse** embeddings, and understand when each shines.
- Walk through the training and fine-tuning of **sentence-level transformer encoders** such as **SBERT** and **MiniLM**.
- Introduce **intrinsic evaluation** metrics (MRR, Recall@k) that let you judge embedding quality before you even hit a vector store.
- Show you how to build **robust, batched encoding pipelines** that scale from a few dozen sentences to millions of documents.

By the end you will be able to create production-ready embeddings, validate them quantitatively, and plug them into the RAG pipeline introduced in Module 1.

1. Introduction

Retrieval-augmented generation hinges on **semantic similarity**: the system must fetch pieces of text that are **meaningfully related** to a user's question, not merely those that share surface tokens. Embedding models are the workhorses that capture this semantic signal.

Key intuition - Dense vectors embed meaning in a low-dimensional continuous space; sparse vectors embed meaning in a high-dimensional space where each dimension corresponds (often loosely) to a lexical or semantic term. Both have a place in modern RAG pipelines.

In practice, you will often start with a **pre-trained sentence encoder** (e.g., `sentence-transformers/all-mpnet-base-v2`) and later **fine-tune** it on domain-specific data. The chapter is organized as follows:

Section	Goal
Core Concepts	Formal definitions of dense vs. sparse embeddings, similarity metrics, and the mathematics behind transformer-based encoders.
Practical Application	Step-by-step Python notebooks that (a) encode a corpus, (b) evaluate embeddings, and (c) integrate them into a vector database (FAISS).
Key Takeaways	Concise recap and a checklist for your next RAG project.

2. Core Concepts

2.1 Dense vs. Sparse Embeddings

Property	Dense Embeddings	Sparse Embeddings
Dimensionality	Typically 256-1024 floating-point numbers.	Often 10 ² K-100 ² dimensions; most entries are zero.
Training Paradigm	Contrastive learning (e.g., Siamese networks), masked language modeling, or knowledge distillation.	Derived from term-frequency statistics (TF-IDF, BM25) or from sparse transformer heads (e.g., SPLADE, DeepCT).
Similarity Metric	Cosine similarity or inner product (fast via BLAS).	Dot product on sparse vectors (often implemented with inverted indexes).
Strengths	- Captures semantic similarity beyond lexical overlap. - Works well with short queries and long passages. - Efficient for ANN (approximate nearest neighbour) search.	- Retains exact lexical matching (useful for rare terms). - Naturally integrates with traditional IR techniques (BM25). - Can be combined with dense vectors in hybrid retrieval.

Property	Dense Embeddings	Sparse Embeddings
Weaknesses	- May miss rare, domain-specific tokens if they never appear in training data.
- Requires a dedicated ANN index (FAISS, HNSW).	- High dimensionality → larger storage and slower exact search unless an inverted index is used.
- Often less robust to paraphrasing.
When to Use	- When you need semantic retrieval (e.g., "How does a transformer work?").
- When you have a large corpus (>10⁶ docs) and want sub-linear search.	- When you must guarantee lexical recall (e.g., legal contracts, code snippets).
- When you already have a mature BM25 pipeline and want a drop-in improvement.

2.1.1 Formal Definition

Dense vector for a piece of text t is a function

$$\mathbf{v}_t = f_{\theta}(t) \in \mathbb{R}^d,$$

where f_{θ} is a neural encoder (typically a transformer) with parameters θ and d is the embedding dimension (e.g., 384).

Sparse vector is a mapping

$$\mathbf{s}_t = g_{\phi}(t) \in \mathbb{R}^{|V|},$$

where $|V|$ is the vocabulary size (often $> 10^5$ K) and g_{ϕ} produces non-zero entries only for a subset of dimensions (e.g., terms with high relevance scores).

Similarity is computed as

$$\text{sim}(a,b) = \begin{cases} \frac{\mathbf{v}_a^{\top} \mathbf{v}_b}{\|\mathbf{v}_a\| \|\mathbf{v}_b\|} & \text{dense (cosine)} \\ \mathbf{s}_a^{\top} \mathbf{s}_b & \text{sparse (dot product)} \end{cases}$$

Both can be **combined** using a weighted sum:

\[

$$\text{sim}_{\text{hybrid}}(a, b) = \lambda \cdot \text{cosine}(\mathbf{v}_a, \mathbf{v}_b) + (1 - \lambda) \cdot \frac{\mathbf{s}_a^{\text{top}} \cdot \mathbf{s}_b}{\|\mathbf{s}_a\| \|\mathbf{s}_b\|}.$$

\]

2.2 Sentence-Level Transformer Encoders

2.2.1 Why Sentence Encoders?

- RAG typically works at the **sentence** or **paragraph** granularity.
- A **sentence-level encoder** produces a single fixed-size vector for an arbitrary length input, making indexing trivial.
- Unlike token-level embeddings (e.g., BERT CLS token), sentence encoders are **fine-tuned** to maximize similarity for semantically related pairs.

2.2.2 Popular Architectures

Model	Base Architecture	Typical Dim	Training Objective	Notable Variants
SBERT (Sentence-BERT)	BERT / RoBERTa	768 (or 384)	Siamese / Triplet contrastive loss	all-mpnet-base-v2, stsb-roberta-large
MiniLM (Distilled)	MiniLM (6-layer)	384	Contrastive + MLM	sentence-transformers/all-miniLM-L12-v2
E5 (Embedding-by-Encoder)	T5-base	768	Multi-task (retrieval, classification)	intfloat/e5-base
SPLADE (Sparse)	BERT	30522 (vocab)	KL-divergence to term distribution	naver/splade-v3

All of these can be accessed via the ``sentence-transformers`` Python library, which wraps the Hugging Face [\[?\]](#) ecosystem and provides out-of-the-box utilities for **batch encoding** and **training**.

2.2.3 Training Objectives

Objective	Description	Typical Use-Case
Contrastive (Siamese)	Pull together embeddings of positive pairs (e.g., paraphrases) and push apart negative pairs.	General semantic similarity, QA retrieval.
Triplet Loss	Similar to contrastive but uses an anchor , positive , and negative triplet.	Fine-grained ranking where a hard negative matters.
Multiple-Negatives Ranking Loss (MNRL)	For each query, treats all other in-batch samples as negatives. Efficient for large batches.	Large-scale corpora where you can afford big batch sizes.
Distillation	Teacher model (e.g., a full-size BERT) provides soft targets.	When you need a lightweight student (MiniLM).
Sparse Supervision (e.g., SPLADE)	Encourages the model to assign high weights to a sparse set of vocabulary tokens.	When you want a hybrid dense-sparse retrieval.

2.3 Intrinsic Evaluation of Embeddings

Before you index millions of vectors, you should **quantitatively assess** whether the encoder captures the similarity you care about. Intrinsic metrics are computed on a **held-out** set of query-document pairs with ground-truth relevance judgments.

2.3.1 Mean Reciprocal Rank (MRR)

Given a query q , the system returns a ranked list of candidates $\langle d_1, d_2, \dots, d_k \rangle$. If the first relevant document appears at rank r , the reciprocal rank is $1/r$. MRR is the average over all queries:

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q}.$$

Higher MRR $\rightarrow$ faster retrieval of the **first** relevant answer.

2.3.2 Recall@k

Recall@k measures the fraction of queries for which **any** relevant document appears in the top-k results:

$$\text{Recall@k} = \frac{1}{|Q|} \sum_{q \in Q} \mathbb{1}_{\left(\bigvee_{d \in \text{Top}_k(q)} d \text{ is relevant} \right)}.$$

Typical values: Recall@1, Recall@5, Recall@10.

2.3.3 Normalized Discounted Cumulative Gain (nDCG)

When relevance is graded (e.g., 0 = irrelevant, 1 = somewhat relevant, 2 = highly relevant), nDCG captures ranking quality:

$$\text{DCG@k} = \sum_{i=1}^k \frac{2^{\text{rel}_i} - 1}{\log_2(i+1)}; \quad \text{nDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}}.$$

2.3.4 Practical Evaluation Loop

```

import torch
from sentence_transformers import SentenceTransformer, util
from tqdm.auto import tqdm
import numpy as np

def evaluate(model, queries, positives, k_vals=[1,5,10]):
    """
    model        - SentenceTransformer instance (already on GPU)
    queries       - list of query strings
    positives     - list of *single* ground-truth positive doc per query
    k_vals        - list of k for Recall@k
    """

    # 1⌚ Encode everything in batches
    query_emb = model.encode(queries, batch_size=64, normalize_embeddings=True,
                             show_progress_bar=False, device='cuda')
    pos_emb = model.encode(positives, batch_size=64, normalize_embeddings=True,
                           show_progress_bar=False, device='cuda')

    # 2⌚ Compute cosine similarity matrix (queries × positives)
    # For large corpora you would search against a full index; here we keep it simple.
    scores = util.cos_sim(query_emb, pos_emb).cpu().numpy() # shape (N, N)

    # 3⌚ Compute MRR and Recall@k
    mrr = 0.0
    recall = {k:0 for k in k_vals}
    for i, row in enumerate(scores):
        # Rank documents descending by similarity
        ranking = np.argsort(-row) # indices of docs sorted by score
        rank_of_true = np.where(ranking == i)[0][0] + 1 # 1-based rank
        mrr += 1.0 / rank_of_true
        for k in k_vals:
            if rank_of_true <= k:
                recall[k] += 1

    n = len(queries)
    mrr = mrr / n
    recall = {k: v/n for k,v in recall.items()}
    return {"MRR": mrr, "Recall@k": recall}

```

Tip: When you move from a toy set to a real corpus, replace the `scores` matrix with a `FAISS` or `ElasticSearch` lookup to keep the computation sub-linear.

2.4 Efficient Batch Encoding Pipelines

Encoding millions of sentences naïvely (`for s in corpus: model.encode(s)`) is a performance nightmare. The `sentence-transformers` library already offers a **batched** encoder, but production pipelines often need additional engineering:

Challenge	Recommended Solution
GPU memory overflow when batch size is too large	Use dynamic batch sizing: start with a large batch, halve it on <code>torch.cuda.OutOfMemoryError</code> .
Variable-length inputs (some sentences are 5 tokens, others 200)	Pad to the maximum length in the batch (<code>padding='longest'</code>) rather than a global maximum; this reduces wasted compute.
I/O bottleneck reading raw text from disk	Stream the corpus with <code>'datasets'</code> library (<code>load_dataset(..., streaming=True)</code>) and pre-fetch batches.
Parallel preprocessing (tokenization, lower-casing, stop-word removal)	Use Ray or multiprocessing to run tokenizers on CPU while the GPU works on the previous batch.
Persisting embeddings	Write to memory-mapped NumPy files (<code>np.memmap</code>) or FAISS's on-disk index (<code>IndexIVFFlat</code> with <code>write_index</code>).

2.4.1 Example: Scalable Encoder with Ray

```

import ray
from sentence_transformers import SentenceTransformer
import numpy as np
import faiss
import os, json, tqdm

# -----
# 1 Ray init (use all available GPUs)
# -----
ray.init(ignore_reinit_error=True)

@ray.remote(num_gpus=1)
class EncoderWorker:
    def __init__(self, model_name="sentence-transformers/all-mpnet-base-v2"):
        self.model = SentenceTransformer(model_name, device="cuda")
        self.model.max_seq_length = 256 # truncate long docs

    def encode_batch(self, texts):
        # returns a 2-D np.ndarray (batch x dim)
        return self.model.encode(
            texts,
            batch_size=len(texts),
            normalize_embeddings=True,
            convert_to_numpy=True,
            show_progress_bar=False,
        )

# -----
# 2 Load a large text file lazily
# -----
def stream_corpus(path):
    with open(path, "r", encoding="utf8") as f:
        for line in f:
            yield line.strip()

# -----
# 3 Main driver - split into chunks & dispatch
# -----
def build_faiss_index(corpus_path, out_index_path,
                     dim=768, batch_size=512, chunks_per_worker=10):

```

... (continued on next page)

```

# 3-worker pool (adjust to #GPUs)
workers = [EncoderWorker.remote() for _ in range(3)]

# FAISS IVF-Flat (fast indexing, moderate recall)
quantizer = faiss.IndexFlatIP(dim) # inner-product (cosine after norm)
index = faiss.IndexIVFFlat(quantizer, dim, 4096, faiss.METRIC_INNER_PRODUCT)
index.train(np.random.rand(4096, dim).astype('float32')) # dummy training

batch = []
futures = []
for i, doc in enumerate(tqdm.tqdm(stream_corpus(corpus_path))):
    batch.append(doc)
    if len(batch) == batch_size:
        # round-robin dispatch
        worker = workers[i % len(workers)]
        futures.append(worker.encode_batch.remote(batch))
        batch = []

# handle final partial batch
if batch:
    worker = workers[i % len(workers)]
    futures.append(worker.encode_batch.remote(batch))

# -----
# 4 Collect results & add to FAISS
# -----
for future in tqdm.tqdm(futures, desc="Collect embeddings"):
    emb = ray.get(future) # shape (B, dim)
    index.add(emb.astype('float32'))

# Persist index
faiss.write_index(index, out_index_path)
print(f"FAISS index written to {out_index_path}")

# -----
# 5 Run
# -----
if __name__ == "__main__":
    build_faiss_index(
        corpus_path="data/wiki_sentences.txt",

```

... (continued on next page)

```

out_index_path="faiss/wiki_index.ivf",
dim=768,
batch_size=1024,
chunks_per_worker=20,
)

```

What this script does:

1. Spins up one Ray actor per GPU, each holding its own `SentenceTransformer` instance.
2. Streams the corpus line-by-line, avoiding loading the whole file into RAM.
3. Sends **batches** to the workers, collects embeddings asynchronously, and pushes them into a **FAISS IVF-Flat** index (which lives on disk).
4. The final index can be loaded instantly in the downstream RAG inference service.

2.5 When to Fine-Tune vs. Use Off-the-Shelf

Situation	Recommended Approach
General-purpose web QA (e.g., "What is the capital of France?")	Off-the-shelf SBERT (<code>all-mpnet-base-v2</code>) - already high-quality on STS-Benchmark.
Domain-specific jargon (e.g., biomedical, legal)	Fine-tune on a domain-specific NLI or contrastive dataset (e.g., MS-MARCO , BioASQ).
Very short queries + long passages	Use a cross-encoder for re-ranking (e.g., <code>cross-encoder/ms-marco-T5-base</code>) after an initial dense retrieval.
Resource-constrained edge deployment	Distill a MiniLM (384-dim) or even Quantized (int8) version of SBERT.
Need for exact term recall	Build a hybrid system: dense SBERT + sparse SPLADE, combine scores with $\lambda_{\text{dense}} \approx 0.6$ (dense) / 0.4 (sparse).

Fine-tuning workflow (high-level):

1. **Collect** a set of (query, positive, negative) triples.
2. **Choose** a loss (MNRL is great if you can fit many samples per batch).
3. **Train** on a GPU for a few epochs (often < 30 on a 12GB card).
4. **Validate** with MRR/Recall@k on a held-out set.
5. **Export** the model (`model.save("my_finetuned_sbert")`).

We'll see a concrete fine-tuning script in the next section.

3. Practical Application

Below is a **complete notebook-style tutorial** that you can copy-paste into a Jupyter environment. It walks you from data preparation through evaluation, fine-tuning, and finally indexing with FAISS for a production-ready RAG pipeline.

Assumptions

- * `Python ≥ 3.9`
`pip install sentence-transformers faiss-gpu datasets tqdm ray`

- * *GPU with at least 8GB VRAM (for dense models).*

3.1 Load a Benchmark Corpus

We'll use the **MS-MARCO Passage Ranking** dataset (publicly available via `datasets`). It contains ~ 8 M passages and ~ 500 K queries with relevance labels - perfect for both training and intrinsic evaluation.

```
from datasets import load_dataset
import random, itertools

# Load a *small* slice for demo (you can drop `split='train[:5%]'` for full)
marco = load_dataset("ms_marco", "passage_ranking", split="train[:5%]")
queries = marco["query"]
passages = marco["passage"]
relevances = marco["is_selected"]      # 1 = relevant, 0 = not

# Build a mapping query_id → list of (passage, label)
from collections import defaultdict
q2p = defaultdict(list)
for q, p, rel in zip(queries, passages, relevances):
    q2p[q].append((p, rel))

# Keep only queries that have at least one positive passage
train_queries = [q for q, pairs in q2p.items() if any(rel for _, rel in pairs)]
print(f"Number of usable queries: {len(train_queries)}")
```

Output (example):

```
Number of usable queries: 31284
```

3.2 Baseline Dense Retrieval with Off-the-Shelf SBERT

```
from sentence_transformers import SentenceTransformer, util
import numpy as np, tqdm.auto as tqdm

model = SentenceTransformer("sentence-transformers/all-mpnet-base-v2",
                           device="cuda")
model.max_seq_length = 256  # truncate long passages

def encode_texts(texts):
    return model.encode(texts,
                        batch_size=128,
                        normalize_embeddings=True,
                        convert_to_numpy=True,
                        show_progress_bar=False)

# Encode *all* passages once (FAISS will store them)
all_passages = list(itertools.chain.from_iterable(
    [p for p, _ in pairs] for pairs in q2p.values()))
passage_emb = encode_texts(all_passages)

# Build a simple FAISS index (inner product after normalization = cosine)
import faiss
dim = passage_emb.shape[1]
index = faiss.IndexFlatIP(dim)  # exact search for demo; switch to IVF for  $>10^6$ 
index.add(passage_emb.astype('float32'))
print("FAISS index size:", index.ntotal)
```

3.2.1 Intrinsic Evaluation (Recall@10)

```
def evaluate_dense(model, index, q2p, k=10):
    recalls = []
    for q, pairs in tqdm.tqdm(q2p.items(), desc="Eval"):
        # encode query
        q_emb = model.encode([q], normalize_embeddings=True,
                              convert_to_numpy=True)[0]

        # retrieve top-k passage ids
        D, I = index.search(np.expand_dims(q_emb,0).astype('float32'), k)
        retrieved = {all_passages[i] for i in I[0]}

        # check if any *relevant* passage is in the retrieved set
        relevant = {p for p, rel in pairs if rel}
        hits = len(retrieved & relevant) > 0
        recalls.append(hits)
    return np.mean(recalls)

recall10 = evaluate_dense(model, index, q2p, k=10)
print(f"Recall@10 (baseline dense): {recall10:.4f}")
```

Typical output for a 5% slice:

```
Recall@10 (baseline dense): 0.6723
```

Interpretation: Roughly two-thirds of the queries retrieve at least one truly relevant passage in the top-10. This is a solid starting point, but many production systems demand Recall@10 > 0.85

3.3 Fine-Tuning with Multiple-Negatives Ranking Loss

We will fine-tune the same `all-mpnet-base-v2` model on the MS-MARCO triples (query, positive, negative). The ``sentence-transformers`` library supplies a ready-made trainer.

```
from sentence_transformers import losses, InputExample, SentenceTransformer, evaluation

# 12 Build training examples (one positive + 4 hard negatives per query)
train_examples = []
for q, pairs in q2p.items():
    pos = [p for p, rel in pairs if rel][:1] # first positive
    neg = [p for p, rel in pairs if not rel][:4] # up to 4 negatives
    if not pos or len(neg) < 4:
        continue
    train_examples.append(InputExample(texts=[q, pos[0]] + neg))

print(f"Training triples: {len(train_examples)}")
```

Sample output:

```
Training triples: 27492
```

```

# 2? Prepare DataLoader
from torch.utils.data import DataLoader
train_dataloader = DataLoader(train_examples, shuffle=True, batch_size=32)

# 3? Define loss
train_loss = losses.MultipleNegativesRankingLoss(model)

# 4? Evaluation set (hold-out 10% of queries)
eval_queries = random.sample(list(q2p.keys()), int(0.1*len(q2p)))
eval_examples = []
for q in eval_queries:
    pos = [p for p, rel in q2p[q] if rel[:1]]
    if not pos:
        continue
    eval_examples.append(InputExample(texts=[q, pos[0]]))

evaluator = evaluation.EmbeddingSimilarityEvaluator.from_input_examples(
    eval_examples, name='msmarco-eval'
)

# 5? Train
model.fit(
    train_objectives=[(train_dataloader, train_loss)],
    evaluator=evaluator,
    epochs=2,
    warmup_steps=int(0.1 * len(train_dataloader)),
    output_path="output/fine_tuned_sbert",
    show_progress_bar=True
)

```

Tip: For large corpora you may want to use `CachingDataset` (part of `sentence-transformers`) to avoid re-tokenizing every epoch.

3.4 Re-evaluate the Fine-Tuned Model

```
# Load the fine-tuned checkpoint
ft_model = SentenceTransformer("output/fine_tuned_sbert", device="cuda")
ft_model.max_seq_length = 256

# Re-encode passages (you could reuse previous embeddings if you kept them)
ft_passage_emb = ft_model.encode(all_passages,
                                  batch_size=128,
                                  normalize_embeddings=True,
                                  convert_to_numpy=True,
                                  show_progress_bar=True)

# New FAISS index
ft_index = faiss.IndexFlatIP(dim)
ft_index.add(ft_passage_emb.astype('float32'))

# Recall@10 again
recall10_ft = evaluate_dense(ft_model, ft_index, q2p, k=10)
print(f"Recall@10 (fine-tuned): {recall10_ft:.4f}")
```

Typical improvement:

```
Recall@10 (fine-tuned): 0.8012
```

A jump from $\sim 0.67 \rightarrow \mathbf{0.80}$ is significant and often enough to meet many production SLAs. If you need even higher recall, consider:

- **Increasing batch size** (more negatives per batch).
- **Hard negative mining** (use the current index to fetch the *most confusing* negatives).
- **Hybrid retrieval** (add a sparse component, see next section).

3.5 Adding a Sparse Component (SPLADE)

SPLADE (Sparse Lexical and Deep Encoder) produces **sparse** vectors that can be indexed with an inverted list (e.g., Elasticsearch) or stored in **FAISS** as a **sparse** matrix using the `IndexFlatIP` after converting to CSR. Here's a lightweight integration:

```

from transformers import AutoTokenizer, AutoModelForMaskedLM
import torch.nn.functional as F

splade_tokenizer = AutoTokenizer.from_pretrained("naver/splade-v3")
splade_model = AutoModelForMaskedLM.from_pretrained("naver/splade-v3").to("cuda")
splade_model.eval()

def splade_encode(texts, top_k=100):
    # Tokenize
    inputs = splade_tokenizer(texts, padding=True, truncation=True,
                              max_length=256, return_tensors="pt").to("cuda")

    with torch.no_grad():
        logits = splade_model(**inputs).logits # (B, seq_len, vocab)
    # Apply ReLU + log(1+exp) as in SPLADE paper
    scores = F.relu(logits).log1p()
    # Sum over tokens → (B, vocab)
    scores = scores.sum(dim=1)
    # Keep only top-k tokens per example (sparsify)
    top_vals, top_idx = torch.topk(scores, k=top_k, dim=1)
    # Return as list of dicts {token_id: weight}
    sparse_vecs = []
    for vals, idx in zip(top_vals, top_idx):
        sparse_vecs.append({int(i): float(v) for i, v in zip(idx, vals)})
    return sparse_vecs

# Encode a few queries & passages
sparse_q = splade_encode(["What is the capital of France?"])[0]
sparse_p = splade_encode([all_passages[0]])[0]
print("Sparse query example:", sparse_q)

```

Hybrid scoring (dense + sparse) can be performed at retrieval time:

```
def hybrid_score(query, candidate_idx, dense_index, dense_model,
                 sparse_q, top_k=10, lambda_dense=0.6):
    # 1 dense part
    q_dense = dense_model.encode([query], normalize_embeddings=True,
                                  convert_to_numpy=True)[0]

    D, I = dense_index.search(np.expand_dims(q_dense,0).astype('float32'), top_k)
    dense_scores = D[0] # already inner product

    # 2 sparse part (dot product on sparse dicts)
    # Retrieve candidate sparse vectors from pre-computed list `sparse_passage_vecs`
    cand_sparse = [sparse_passage_vecs[i] for i in I[0]]
    sparse_scores = []
    for sp in cand_sparse:
        # simple dot product on overlapping tokens
        score = sum(query_weight * sp.get(tok,0.0) for tok, query_weight in sparse_q.items())
        sparse_scores.append(score)
    sparse_scores = np.array(sparse_scores)

    # 3 combine
    final = lambda_dense * dense_scores + (1-lambda_dense) * sparse_scores
    # re-rank
    rank = np.argsort(-final)
    return I[0][rank], final[rank]
```

Production tip: Store the sparse vectors in a PostgreSQL/PGVector or Elasticsearch index; they are cheap to retrieve because they are already token-level. The dense part can stay in FAISS for sub-linear speed.

3.6 End-to-End RAG Integration (Recap)

Now that we have a **high-quality embedding pipeline**, we can plug it into the RAG architecture introduced in Module 1:

```
flowchart LR
    Q[User Query] -->|Encode (dense+optional sparse)| E[Embedding]
    E -->|FAISS (or hybrid search)| D[Top-k Documents]
    D -->|Prompt Template| P[LLM Prompt]
    P -->|LLM (e.g., GPT-4o)| G[Generated Answer]
    G -->|Display| A[User]
```

Key code snippet for the inference side (assuming Flask + FAISS):

```

from flask import Flask, request, jsonify
app = Flask(__name__)

# Load models once at startup
dense_model = SentenceTransformer("output/fine_tuned_sbert", device="cuda")
dense_index = faiss.read_index("faiss/wiki_index.ivf")
sparse_tokenizer = splade_tokenizer
sparse_model = splade_model

@app.route("/rag", methods=["POST"])
def rag():
    data = request.get_json()
    query = data["query"]
    # 1? dense embedding
    q_dense = dense_model.encode([query], normalize_embeddings=True,
                                convert_to_numpy=True)[0]

    # 2? top-k dense retrieval
    D, I = dense_index.search(np.expand_dims(q_dense,0).astype('float32'), 5)
    # 3? fetch raw texts (e.g., from a DB) - placeholder list
    docs = [all_passages[i] for i in I[0]]
    # 4? construct prompt
    prompt = f"""You are an expert assistant. Use the following context verbatim to answer the question.
\n\nContext:\n{chr(10)}.join(docs)}\n\nQuestion: {query}\nAnswer: ""
    # 5? call LLM (pseudo-function)
    answer = call_llm_api(prompt) # e.g., OpenAI ChatCompletion
    return jsonify({"answer": answer, "retrieved": docs})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)

```

You now have a **complete, production-ready RAG stack**:

- **Encoding** - dense (SBERT) + optional sparse (SPLADE).
- **Evaluation** - intrinsic metrics (MRR, Recall@k).
- **Scalable indexing** - FAISS IVF-Flat (or HNSW) for dense, inverted index for sparse.
- **Inference** - Flask endpoint that retrieves, constructs a prompt, and calls an LLM.

4. Key Takeaways

- **Dense vs. Sparse**

- Dense embeddings excel at semantic similarity; sparse embeddings preserve exact lexical signals.
- Hybrid retrieval (weighted combination) often yields the best of both worlds.

- **Sentence-Level Transformers**

- SBERT, MiniLM, and newer models like E5 provide out-of-the-box high-quality sentence vectors.
- Fine-tuning with contrastive losses (MNRL, Triplet) on domain-specific data can lift Recall@k from $\sim 0.6 \rightarrow > 0.8$.

- **Intrinsic Evaluation**

- Use **MRR**, **Recall@k**, and **nDCG** on a held-out query-document set before committing to a vector store.
- A simple evaluation loop (cosine similarity matrix) is enough for prototypes; scale to FAISS for real-world sizes.

- **Batch Encoding Pipelines**

- Leverage **GPU-accelerated batched encoding** (`batch_size≈128-256`).
- For massive corpora, adopt **Ray** or **multiprocessing** to parallelize across multiple GPUs.
- Persist embeddings directly into a **FAISS on-disk index** to avoid a separate serialization step.

- **Hybrid Retrieval in Production**

- Combine dense (FAISS) and sparse (SPLADE/Elasticsearch) scores with a tunable λ .
- Store the sparse vectors in an inverted index; the dense vectors stay in an ANN index.

- **From Embeddings to RAG**

1. Encode the query $\rightarrow$ vector.
2. Retrieve top-k passages (dense, sparse, or both).
3. Insert passages into a **prompt template**.

4. Call the LLM (e.g., GPT-4o) → final answer.

• Checklist Before Going Live

Item	Why it matters
Dataset split (train / dev / test)	Guarantees unbiased evaluation.
Intrinsic metrics (MRR ≥ 0.75, Recall@10 ≥ 0.85)	Sets a quantitative baseline for retrieval quality.
Batch size & GPU memory profiling	Prevents OOM crashes in production.

| FAISS index type (IVF-Flat vs. HNSW) chosen based on corpus size & latency budget. |

| Hybrid λ tuned on dev set (grid-search λ ∈ [0,1])Balances semantic vs

Building and Querying Vector Databases

Module 3 of 9 - "Learn Retrieval- Augmented Generation in Python to Build Production-Ready Knowledge-Grounded AI Systems"

Overview

In the first two modules we learned **why** a retrieval component is essential for Retrieval-Augmented Generation (RAG) and **how** to turn raw text into dense vectors with modern embedding models. In this chapter we close the loop: we will **store** those vectors, **index** them for fast similarity search, **filter** on arbitrary metadata, and **persist** the whole collection so that experiments are reproducible and production pipelines are reliable.

By the end of the chapter you will be able to:

1. **Compare** the major open-source and managed vector stores (FAISS, Milvus, Qdrant, Weaviate, Pinecone, Azure Cognitive Search, etc.) and decide which fits a given use-case.
2. **Create** a vector collection, attach rich metadata, and **populate** it from a Python data pipeline.
3. **Choose** the right ANN index type (IVF, HNSW, PQ, etc.) and understand the trade-offs between recall, latency, and storage.
4. **Run** approximate nearest-neighbor (ANN) queries with metadata filters, and interpret the results.
5. **Persist** and **version** vector data (snapshots, migrations, and reproducible experiments).

Why this matters: In a production RAG system the retrieval layer is the "knowledge-gate". If the vector store is slow, inflexible, or not reproducible, the whole AI product will suffer from latency spikes, stale knowledge, or irreproducible bugs.

1. Introduction

RAG pipelines typically look like this:

```
User query → Embed → Vector DB (ANN search + filter) → Retrieve docs → LLM (prompt) → Answer
```

The **vector database** is the only component that bridges the **semantic** world (dense vectors) and the **structured** world (metadata, version control). It must:

- **Scale** to millions of vectors while staying sub-second.
- **Support filters** (e.g., "only articles from 2022", "language = en", "category = finance").
- **Persist** data across restarts, upgrades, and experiments.
- **Expose a simple API** that can be called from Python, Go, or any language the

rest of your stack uses.

In this chapter we will:

- Walk through a **decision matrix** for picking a vector store.
 - Build a **complete end-to-end example** that ingests a public dataset, creates embeddings, stores them with metadata, and runs filtered ANN queries.
 - Show **how to version** the collection using snapshots and migration scripts.
 - Provide **exercises** that let you experiment with index parameters and measure trade-offs.
-

2. Core Concepts

2.1 Vector Store Landscape

Store	License	Deployment	Key Index Types	Metadata Filtering	Production-Ready Features
FAISS (Facebook AI Similarity Search)	BSD-3	In-process, self-hosted (CPU/GPU)	IVF, HNSW, PQ, OPQ, IVF-PQ, Flat	None (must be handled by the caller)	GPU acceleration, extremely fast, low-level API
Milvus	Apache-2.0	Self-hosted (Docker/K8s) or Managed (Zilliz)	IVF FLAT , IVFPQ , HNSW, ANNOY, DISKANN	Built-in scalar & vector filters (JSON)	Distributed, auto-scaling, TTL, hybrid search
Qdrant	Apache-2.0	Self-hosted (Docker/K8s) or Managed (Qdrant Cloud)	HNSW (primary)	Full-text & payload filters (SQL-like)	Collections, snapshots, replicas
Weaviate	BSD-3	Self-hosted (Docker/K8s) or Managed (Weaviate)	HNSW, IVF, Graph-based (schema)	GraphQL & hybrid filters	Schema-driven, modules (text2vec-openai, BM25),

Store	License	Deployment	Key Index Types	Metadata Filtering	Production-Ready Features
		Cloud)			auto-scaling
Pinecone	Commercial SaaS	Fully managed	IVF-PQM, HNSW (internal)	Metadata filters (exact & range)	Serverless, auto-indexing, versioned namespaces
Azure Cognitive Search (Vector) / Amazon OpenSearch	Cloud-provider	Managed	HNSW (Azure), HNSW + IVF (OpenSearch)	Full-text + scalar filters	Integration with Azure/AWS ecosystem, RBAC
ElasticSearch (k-NN plugin)	Apache-2.0	Self-hosted / Managed	HNSW, IVF	Full-text + scalar	Unified search (keyword + vector)

2.1.1 Decision Matrix

Dimension	FAISS	Milvus	Qdrant	Weaviate	Pinecone
Ease of Setup	❓ (pip install) - but you manage persistence yourself	❓ (Docker compose) - built-in storage	❓ (Docker) - simple API	❓ (Docker) - schema-first	❓ (no infra)
Scalability	Limited to one machine (GPU)	❓ Distributed shards	❓ Horizontal scaling (replicas)	❓ Distributed + Graph queries	❓ Infinite (managed)
Metadata Filtering	❓ (must post-filter)	❓ Rich filters	❓ Payload filters	❓ GraphQL filters	❓ Exact & range
Hybrid Search (text + vector)	❓	❓ (BM25 + vector)	❓ (full-text module)	❓ (BM25 module)	❓ (via namespace)
Versioning /	❓ (manual)	❓	❓ (snapshot)	❓ (export/	❓

Dimension	FAISS	Milvus	Qdrant	Weaviate	Pinecone
Snapshots		(snapshots)	API)	import)	(namespaces)
Cost	Free (compute only)	Free (self-host)	Free (self-host)	Free (self-host)	Pay-as-you-go
Community & Ecosystem	Huge (FAISS is a library)	Growing (Milvus Cloud)	Emerging (Qdrant Cloud)	Strong (Weaviate modules)	Enterprise support

Rule of thumb:

If you need a quick prototype on a single GPU, start with FAISS.

If you need metadata filtering and horizontal scaling, pick Milvus, Qdrant, or Weaviate.

If you want zero-ops and are comfortable with a SaaS vendor, go with Pinecone or a cloud provider's vector search.

2.2 Approximate Nearest-Neighbor (ANN) Index Types

Index	How it works	Typical Recall / Latency	Memory / Disk Footprint	Best Use-Case
Flat (Exact)	Brute-force linear scan	100% recall, high latency ($O(N)$)	$N \times d \times 4$ bytes (float32)	Small collections ($< 10^8$ for evaluation baseline)
IVF (Inverted File)	Coarse quantizer → assign vectors to centroids → search only a subset	90-98% recall, low latency (sub-ms for 1^8 M)	$N \times d \times 4 +$ centroid tables	Large static corpora where you can afford a training step

Index	How it works	Typical Recall / Latency	Memory / Disk Footprint	Best Use-Case
IVF-PQ (Product Quantization)	IVF + compress vectors into short codes (e.g., 8-byte)	80-95% recall, very low memory	$N \times \text{code_size}$ (e.g., 8 B)	Billions of vectors on limited RAM
HNSW (Hierarchical Navigable Small World)	Graph-based greedy search; layers of proximity graphs	95-99% recall, micro-second latency	$N \times d \times 4 + \text{graph edges}$ ($\approx 2 \times$)	Real-time serving, dynamic updates
ANNOY (Random Projection Forest)	Multiple binary trees built on random projections	80-90% recall, fast build, read-only after build	$N \times d \times 4 + \text{tree nodes}$	Static read-only datasets, easy to serialize
DISKANN	Disk-resident graph with cache-aware traversal	90-95% recall, low RAM footprint	Disk-based, RAM for cache only	Very large corpora that don't fit in RAM

2.2.1 Trade-off Cheat Sheet

Goal	Pick	Why
Maximum recall	Flat or HNSW with <code>ef=200</code>	Exact search or deep graph traversal
Minimum latency	HNSW (<code>M=32</code> , <code>ef=50</code>) on GPU	Graph search is $O(\log N)$ and GPU-friendly
Memory-constrained	IVF-PQ or DISKANN	Compress vectors, keep only centroids in RAM
Frequent writes	HNSW (supports insert/delete) or Milvus's <code>INSERT</code> API	Index can be updated online
Bulk ingestion	IVF (train once) then <code>add</code>	Fast batch add after training

2.3 Persistence & Versioning

A reproducible RAG experiment requires that **the exact same vector collection** can be re-loaded across runs. Strategies differ per store:

Store	Snapshot Mechanism	Versioning Approach
FAISS	Serialize index (<code>faiss.write_index</code>) + separate file for metadata (JSON/Parquet)	Use Git-LFS or DVC to version the <code>.index</code> + metadata files
Milvus	<code>milvus.backup</code> (creates tarball per collection)	Namespace per experiment (<code>my_collection_v1</code>)
Qdrant	<code>/collections/{name}/snapshots</code> endpoint → downloadable <code>.snapshot</code> file	Incremental snapshots + naming convention (<code>2024-02-21_v2</code>)
Weaviate	Export schema + data (<code>/v1/backups</code>)	Backup/restore per Git tag
Pinecone	Namespaces act as version containers; <code>describe_index_stats</code> for audit	Create a new namespace for each model version (e.g., <code>v1_20240221</code>)

Best practice: Store **both** the vector index and the **metadata table** in the same versioned artifact. For large corpora, use a data versioning tool like **DVC** or **LakeFS** to track changes without duplicating the whole dataset.

3. Practical Application

We will build a **knowledge-grounded chatbot** that answers questions about **U.S. public policies** using the "PolicyFAQ" dataset (a fictional CSV with columns `id`, `question`, `answer`, `category`, `year`). The pipeline will:

1. Load the CSV.

2. Generate embeddings with **OpenAI's** ``text-embedding-3-large`` (or a local model if you prefer).
3. Store vectors in **Qdrant** (self-hosted) with rich metadata.
4. Perform a filtered ANN query (`category = "healthcare"` and `year >= 2020`).
5. Retrieve the top-k documents and feed them to an LLM (e.g., `gpt-4o-mini`) for RAG.
6. Snapshot the collection for reproducibility.

Why Qdrant? It offers a simple REST/ GRPC API, built-in payload filtering, and snapshot support, making it ideal for an educational walkthrough that can later be swapped for Milvus, Weaviate, or Pinecone with minimal code changes.

3.1 Environment Setup

```
# Create a fresh virtual environment
python -m venv venv
source venv/bin/activate

# Core libraries
pip install pandas tqdm openai tqdm

# Vector store client
pip install qdrant-client[grpc] # installs both HTTP and gRPC clients

# Optional: local embedding model (sentence-transformers)
pip install sentence-transformers
```

3.1.1 Running Qdrant Locally

```
docker run -p 6333:6333 -p 6334:6334 \
  -v $(pwd)/qdrant_storage:/qdrant/storage \
  qdrant/qdrant
```

- **Port 6333** - HTTP API (default for `qdrant-client`).

- **Port 6334** - gRPC API (faster for bulk inserts).
- **Mounted volume** ensures data survives container restarts.

Tip: In production you would use a managed Qdrant Cloud instance, a Kubernetes deployment, or replace the container with Milvus/Weaviate.

3.2 Data Ingestion & Embedding

```

import pandas as pd
from tqdm.auto import tqdm
from sentence_transformers import SentenceTransformer
from qdrant_client import QdrantClient
from qdrant_client.http.models import (
    Distance,
    VectorParams,
    PayloadSchemaType,
    Filter,
    FieldCondition,
    MatchValue,
    Range,
)

# -----
# 1 Load the CSV
# -----
df = pd.read_csv("policy_faq.csv")
print(f"Loaded {len(df)} rows")

# -----
# 2 Create embeddings (you can swap this for OpenAI's API)
# -----
model = SentenceTransformer("intfloat/e5-large-v2") # 1024-dim
def embed(text: str):
    return model.encode(text, normalize_embeddings=True)

embeddings = []
for txt in tqdm(df["question"], desc="Embedding"):
    embeddings.append(embed(txt))

# -----
# 3 Connect to Qdrant and create a collection
# -----
client = QdrantClient(host="localhost", port=6333)

# Define the collection schema
vector_dim = embeddings[0].shape[0]
client.recreate_collection(
    collection_name="policy_faq",

```

... (continued on next page)

```

vectors_config=VectorParams(size=vector_dim, distance=Distance.COSINE),
# Payload schema helps with validation and UI
payload_schema={
    "category": PayloadSchemaType.KEYWORD,
    "year": PayloadSchemaType.INTEGER,
    "answer": PayloadSchemaType.TEXT,
    "question": PayloadSchemaType.TEXT,
},
)

# -----
# 4 Bulk upload with payload
# -----
batch_size = 256
for i in tqdm(range(0, len(df), batch_size), desc="Uploading"):
    ids = df["id"].iloc[i : i + batch_size].tolist()
    vectors = embeddings[i : i + batch_size]

    payload = [
        {
            "category": row["category"],
            "year": int(row["year"]),
            "question": row["question"],
            "answer": row["answer"],
        }
        for _, row in df.iloc[i : i + batch_size].iterrows()
    ]

    client.upsert(
        collection_name="policy_faq",
        points=[
            {
                "id": int(_id),
                "vector": vec.tolist(),
                "payload": pl,
            }
            for _id, vec, pl in zip(ids, vectors, payload)
        ],
    )
print("Collection populated")

```

What just happened?

Step	Purpose
------	---------

Step	Purpose
<code>`recreate_collection`</code>	Declares a <i>named</i> collection with a cosine distance metric and a 1024-dim vector space.
<code>`payload_schema`</code>	Tells Qdrant the expected data types; enables efficient filtering and UI introspection.
<code>`upsert`</code>	Inserts or updates points (vectors + payload) in batches. Qdrant automatically builds an HNSW index behind the scenes (default M=16 , ef_construct=100).

Note: If you prefer a different index (e.g., IVF), you can pass `hnsf_config=None` and enable `quantization_config=...` - see the Qdrant docs for advanced tuning.

3.3 Performing Filtered ANN Queries

Suppose a user asks:

"What recent changes have been made to the Medicare reimbursement policy?"

We want to:

1. **Embed** the query.
2. **Search** only the *healthcare* category and only documents from **2020 onward**.

```
def search(query: str, top_k: int = 5, category: str = None, min_year: int = None):  
    # 1 Embed the query  
    q_vec = embed(query)  
  
    # 2 Build the filter  
    conditions = []  
    if category:  
        conditions.append(  
            FieldCondition(  
                key="category",  
                match=MatchValue(value=category)  
            )  
        )  
    if min_year:  
        conditions.append(  
            FieldCondition(  
                key="year",  
                range=Range(gte=min_year)  
            )  
        )  
    filter_ = Filter(must=conditions) if conditions else None  
  
    # 3 ANN search  
    hits = client.search(  
        collection_name="policy_faq",  
        query_vector=q_vec.tolist(),  
        query_filter=filter_,  
        limit=top_k,  
        with_payload=True, # return payload for LLM prompt  
        score_threshold=0.0, # cosine similarity; 0 is max dist, -1 is min  
    )  
    return hits  
  
# Example usage  
results = search(  
    query="Recent changes to Medicare reimbursement",  
    top_k=3,  
    category="healthcare",  
    min_year=2020,  
)
```

... (continued on next page)

```
for hit in results:
    print(f"\nScore: {hit.score:.4f}")
    print(f"Question: {hit.payload['question']}")
    print(f"Answer: {hit.payload['answer']}")
```

Interpretation of the score:

- Qdrant returns **cosine similarity** in the range `[-1, 1]`. Higher is more similar.
- You can set `score_threshold` to prune low-quality matches (e.g., `>0.6`).

3.4 RAG Prompt Construction

```
def build_rag_prompt(query: str, retrieved_docs):
    # Simple concatenation style prompt
    context = "\n\n--\n\n".join(
        [f"Q: {doc.payload['question']}\nA: {doc.payload['answer']}" for doc in retrieved_docs]
    )
    prompt = f"""You are an expert policy analyst. Use the following context to answer the user's
question.

Context:
{context}

User question: {query}
Answer (concise, factual, with citations if possible):"""
    return prompt

# Build prompt
prompt = build_rag_prompt(
    query="Recent changes to Medicare reimbursement",
    retrieved_docs=results,
)

print(prompt)
```

Now you can feed `prompt` to any LLM (OpenAI, Anthropic, Cohere, etc.) and obtain a **knowledge-grounded** answer.

3.5 Snapshotting & Versioning

```
# Create a snapshot (timestamped)
snapshot_name = f"policy_faq_{pd.Timestamp.now():%Y%m%d_%H%M%S}"
client.create_snapshot(
    collection_name="policy_faq",
    snapshot_name=snapshot_name,
)

print(f"📸 Snapshot created: {snapshot_name}")

# List snapshots
print("Available snapshots:", client.list_snapshots(collection_name="policy_faq"))
```

Restoring a snapshot (e.g., for a reproducible experiment):

```
def restore_snapshot(snapshot_name: str):
    client.restore_snapshot(
        collection_name="policy_faq",
        snapshot_name=snapshot_name,
        # Optional: you can restore to a *new* collection to avoid clobbering
        # target_collection_name="policy_faq_restored"
    )
    print(f"🔄 Restored snapshot {snapshot_name}")

# Example restore (be careful, it overwrites the collection)
# restore_snapshot(snapshot_name)
```

3.6 Swapping the Backend - Minimal Code Changes

If you later decide to move from Qdrant to **Milvus**, you only need to:

1. Change the client import (from `pymilvus import Collection, connections`).
2. Adjust the schema creation (Milvus uses `FieldSchema` + `CollectionSchema`).
3. Replace `client.upsert` with `collection.insert`.
4. Keep the **filter building** logic - Milvus also supports a `bool_expr` filter string.

Because the **core logic** (embedding, query vector, top-k, metadata handling) stays

the same, the switch is a **few lines**. This is why we abstracted the retrieval step into a reusable `search()` function.

4. Key Takeaways

? What you learned	? Why it matters
Vector store taxonomy - FAISS, Milvus, Qdrant, Weaviate, Pinecone, cloud offerings.	Picking the right store determines latency, scalability, and operational overhead.
ANN index families - Flat, IVF, IVF-PQ, HNSW, ANNOY, DISKANN.	Different indexes trade recall for speed and memory; you can tune them for your SLA.
Metadata filtering - Payload filters (JSON), scalar/range conditions, graph queries.	Enables knowledge-grounded retrieval (e.g., "only health-care docs from 2022").
Persistence & versioning - Snapshots, namespaces, backups, DVC.	Guarantees reproducible experiments and safe roll-backs in production.
End-to-end pipeline - Load → embed → store → filtered ANN search → RAG prompt.	Provides a concrete template you can reuse for any domain (legal, finance, support, etc.).
Portability - Minimal code changes to swap back-ends.	Future-proofs your system against vendor lock-in or evolving infrastructure.

5. Exercises

1. Index Comparison
- Goal:** Quantify recall vs. latency for three index types (Flat, IVF-PQ, HNSW).
- Steps:**
- Use the same 100k vector subset of the PolicyFAQ dataset.
 - For each index, index the vectors, then run 1k random queries.

- Record average query time (`time.time()`) and **recall@10** (compare returned IDs against a brute-force baseline).
- Plot latency vs. recall and write a short analysis (which index is best for "real-time chatbot"?).

2. Metadata-Heavy Filtering

Goal: Test filter performance when the payload has high cardinality. **Steps:**

- Add two more fields: `region` (50 possible values) and `topic_tags` (list of up to 10 keywords).
- Populate the collection with 1M Mentries (you can duplicate rows with random tags).
- Run a query that filters on `region = "Midwest"` **AND** `topic_tags CONTAINS "insurance"` and measure query latency.
- Experiment with **Qdrant's** `optimizers_config` (e.g., `memmap_threshold`) to see impact.

3. Versioned Experiments

Goal: Demonstrate reproducibility across model updates. **Steps:**

- Create two versions of the embedding model: `e5-base` (768-dim) and `e5-large` (1024-dim).
- Build two separate collections (`policy_faq_v1` and `policy_faq_v2`) each with a snapshot after insertion.
- Run the same query on both collections and compare the top-k answers.
- Commit the snapshots to a Git repository using **DVC** (`dvc add snapshot.tar.gz`) and write a short `README.md` describing the experiment.

4. Hybrid Search (optional, for the adventurous)

Goal: Combine BM25 keyword search with vector similarity. **Steps:**

- In **Weaviate**, enable the `text2vec-openai` module and the `bm25` module.
- Index the same PolicyFAQ data with both modules.
- Execute a **Hybrid** query that boosts BM25 by 0.3 and vector similarity by 0.7.
- Observe how the results differ from pure vector search.

6. Further Reading & Resources

Topic	Resource
Deep dive into FAISS	[FAISS documentation](https://github.com/facebookresearch/faiss) - especially the "IndexIVFPQ" and "IndexHNSWFlat" sections.
Milvus tutorial	Milvus official guide: " Getting Started with Milvus " - includes Docker-Compose and Python SDK examples.
Qdrant performance tuning	Qdrant blog post: " Understanding HNSW Parameters for Optimal Search ".
Weaviate schema design	Weaviate docs: " Class & Property Modeling " - shows how to define GraphQL filters.
Pinecone best practices	Pinecone whitepaper: " Scaling Vector Search for Production ".
Data versioning	DVC tutorial: " Versioning Large Datasets for ML ".
Evaluation of ANN	"ANN-Benchmarks" GitHub repo - a community-maintained benchmark of recall vs. latency across libraries.

? Recap

Building a **production-ready** RAG system hinges on a **robust vector database**. By mastering the landscape of open-source and managed stores, understanding the nuances of ANN index types, and implementing disciplined persistence/versioning, you can:

- Serve **low-latency, knowledge-grounded** answers at scale.
- Keep your experiments **reproducible** and **auditable**.
- Swap back-ends or upgrade models **without rewriting core logic**.

In the next module we will **integrate** this retrieval layer with **LLM orchestration** (prompt templating, response streaming, and tool-use) to complete the end-to-end RAG pipeline. Happy indexing!

Designing Retrieval Pipelines with LangChain and Retrieval Frameworks

Module 4 of 9 - "Learn Retrieval- Augmented Generation in Python to Build Production-Ready Knowledge-Grounded AI Systems"

Table of Contents

Section	What you'll learn
1 Introduction	Why a well-engineered retrieval pipeline matters, the "plug-and-play" philosophy of LangChain, and the checklist for production-grade RAG.
2 Core Concepts	Deep dive into modular retrievers, hybrid (dense + BM25) search, ranking & post-processing, relevance feedback, dynamic query rewriting, and performance-aware design.
3 Practical Application	Step-by-step code walkthrough: building a hybrid pipeline with LangChain, adding a cross-encoder ranker, implementing feedback loops, and benchmarking latency/throughput.
4 Key Takeaways	The essential mental models and a cheat-sheet you can copy into your next project.

TL;DR - By the end of this chapter you will be able to assemble a plug-and-play retrieval pipeline that combines dense vectors + lexical BM25,

re-ranks results with an LLM-powered cross-encoder, adapts to user feedback, and reports latency/throughput in a production-ready way-all using LangChain's composable abstractions.

1? Introduction

Retrieval-Augmented Generation (RAG) is only as good as the **pipeline that feeds the language model**. In the first two modules we learned how to:

- generate embeddings (e.g., sentence-transformers),
- store them in a vector database (e.g., Chroma, Pinecone, Qdrant), and
- query the vector store with a simple similarity search.

Those steps give you a **baseline retriever** - a "single-vector" lookup. Real-world workloads, however, demand:

Real-world Requirement	Why a naïve vector-only retriever fails
Robustness to query phrasing	Dense vectors are great for semantic similarity but can miss exact phrase matches that a lexical engine (BM25) would catch.
Fine-grained relevance	The nearest-neighbor list may contain loosely related docs; a secondary ranker is needed to surface the truly most useful chunk.
User-driven adaptation	Production systems must learn from clicks, thumbs-up/down, or explicit "re-search" requests.
Performance guarantees	Latency budgets (e.g., < 300 ms) and throughput (e.g., > 100 QPS) dictate how many components you can chain and how they are parallelised.

LangChain was built with exactly this **modular mindset**: every piece-vector store, lexical retriever, ranker, post-processor, query-rewriter-is an interchangeable **Component** that can be wired together through a simple Python API.

Design Principle #1 - "Composable Retrieval" - Treat each stage as a pure function `input → output` with a well-defined contract. This makes testing, swapping, and scaling trivial.

In this chapter we will:

- 1. **Define the contract** for each stage (retriever, ranker, post-processor, feedback loop).
- 2. **Show how to plug them together** using LangChain's `Retriever`, `BaseCombineDocumentsChain`, and the newer `MultiRetriever` abstraction.
- 3. **Implement a hybrid retrieval** that merges dense vectors (via a vector DB) and BM25 (via `rank_bm25`).
- 4. **Add a cross-encoder ranker** (e.g., `cross-encoder/ms-marco-MiniLM-L-6-v2`) to re-order the hybrid set.
- 5. **Close the loop** with relevance feedback (explicit clicks + pseudo-relevance feedback) and a dynamic query rewriting component powered by an LLM.
- 6. **Benchmark** the end-to-end latency and throughput with realistic query loads, and discuss scaling strategies (caching, async, batching).

Let's get our hands dirty.

2. Core Concepts

2.1 Retrieval Contracts in LangChain

Component	LangChain Base Class	Core Method(s)	Typical
Vector Retriever	<code>VectorStoreRetriever</code>	<code>get_relevant_documents(query)</code>	<code>List[Docu</code>
Lexical Retriever	<code>BaseRetriever</code> (custom)	<code>get_relevant_documents(query)</code>	<code>List[Docu</code>
Hybrid Retriever	<code>MultiRetriever</code> (or custom)	<code>get_relevant_documents(query)</code>	<code>List[Docu</code>

Component	LangChain Base Class	Core Method(s)	Typical
Ranker	BaseRerank (e.g., CrossEncoderRerank)	rerank(query, docs)	List[Docu
Post-Processor	BaseDocumentCompressor or custom	compress_documents(docs)	List[Docu
Feedback Loop	BaseRetriever wrapper	add_feedback(query, doc_id, relevance)	None (upc
Query Rewriter	LLMChain (or PromptTemplate)	invoke(query)	str (rewr

Contract Reminder - Every component receives a plain string query (or a Document for some rankers) and returns a list of `Document` objects (langchain.schema.Document). The Document holds page_content, metadata, and a unique `doc\_id`. This uniformity is what lets us stitch components together without glue code.

2.2 Hybrid Retrieval: Dense + Lexical

2.2.1 Why combine dense and lexical?

- **Dense vectors** capture **semantic similarity** - "What is the capital of France?" matches "Paris is the capital city..."
 - **BM25** captures **exact term overlap** - "capital of France" will also retrieve a doc that contains the phrase verbatim, even if its embedding is a bit off.
- The classic "dual- encoder" approach (dense) + "sparse- encoder" (BM25) yields **higher recall** with **minimal extra latency** if we parallelise the two searches.

2.2.2 Building blocks

Block	Library	Key Function
Dense vector store	chromadb, pinecone-client, qdrant-client	vector_store.similarity_s k)

Block	Library	Key Function
BM25	rank_bm25	bm25.get_top_n(query_term n)
Fusion algorithm	Custom (Reciprocal Rank Fusion, Weighted Sum)	fuse(dense_scores, bm25_s

Reciprocal Rank Fusion (RRF) is a popular, parameter-free method:

$$\text{score}_{\text{RRF}}(d) = \sum_{i \in \{\text{dense}, \text{bm25}\}} \frac{1}{k + \text{rank}_i(d)}$$

where `k` is a small constant (e.g., 60) that smooths the contribution of lower-ranked items.

2.2.3 LangChain wiring

```
from langchain.retrievers import MultiRetriever
from langchain.schema import Document

class BM25Retriever(BaseRetriever):
    """Wraps rank_bm25 for LangChain compatibility."""
    def __init__(self, corpus: List[Document], k: int = 10):
        self.corpus = corpus
        self.k = k
        self.bm25 = BM25Okapi([doc.page_content.split() for doc in corpus])

    def get_relevant_documents(self, query: str) -> List[Document]:
        tokens = query.split()
        scores = self.bm25.get_scores(tokens)
        top_n_idx = sorted(
            range(len(scores)), key=lambda i: scores[i], reverse=True
       )[: self.k]
        return [self.corpus[i] for i in top_n_idx]
```

```
# Dense retriever (example with Chroma)
from langchain.vectorstores import Chroma
from langchain.embeddings import SentenceTransformerEmbeddings

embedder = SentenceTransformerEmbeddings(model_name="all-MiniLM-L6-v2")
vectorstore = Chroma(collection_name="wiki_chunks", embedding_function=embedder)

dense_retriever = vectorstore.as_retriever(search_kwargs={"k": 10})

# Hybrid via MultiRetriever (parallel execution)
hybrid = MultiRetriever(
    retrievers=[dense_retriever, BM25Retriever(corpus=vectorstore.get_all_documents())],
    combine_documents_fn=lambda docs: reciprocal_rank_fusion(docs, k=60)
)
```

Note - `MultiRetriever` runs each sub-retriever concurrently (via `asyncio` under the hood) and then calls `combine_documents_fn` to fuse the results. You can replace `reciprocal_rank_fusion` with a weighted sum if you have domain-specific priors (e.g., $0.7 * \text{dense} + 0.3 * \text{bm25}$).

2.3 Ranking & Post-Processing

Even after fusion, the top-k list may contain **redundant or partially relevant** chunks. Two stages are commonly added:

1. **Cross-Encoder Reranker** - a supervised model that scores (query, doc) pairs.
2. **Post-Processor** - tasks like deduplication, chunk trimming, or metadata-based filtering.

2.3.1 Cross-Encoder Rerankers

Cross-encoders read the full query-document pair together, enabling **fine-grained relevance** at the cost of $O(k)$ model calls (still cheap for $k \leq 50$).

```
from langchain.retrievers import CrossEncoderRerank
from sentence_transformers import CrossEncoder

cross_encoder = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
reranker = CrossEncoderRerank(cross_encoder=cross_encoder, top_n=5)

def rerank(query: str, docs: List[Document]) -> List[Document]:
    return reranker.rerank(query, docs)
```

2.3.2 Post-Processors

LangChain ships a few ready-made compressors (e.g., `LLMChainDocumentCompressor`). For a production pipeline we often need:

- **Deduplication** - remove documents with identical `page_content` or high cosine similarity.
- **Length trimming** - keep only the most informative sentences (useful when the LLM's context window is limited).

```
from langchain.document_compressors import LLMChainDocumentCompressor
from langchain.llms import OpenAI

compressor = LLMChainDocumentCompressor(
    llm=OpenAI(temperature=0.0),
    prompt_template="""
    You are a summarizer. Reduce the following passage to at most 150 words while preserving all factual
    details.
    Passage: {text}
    """
)

def post_process(docs: List[Document]) -> List[Document]:
    # 1. Dedup
    uniq = {}
    for d in docs:
        if d.page_content not in uniq:
            uniq[d.page_content] = d
    # 2. Compress
    return compressor.compress_documents(list(uniq.values()))
```

2.4 Relevance Feedback Loops

A **feedback loop** turns user signals (clicks, thumbs- up/ down, explicit "not relevant") into a **dynamic improvement** of the retriever. Two common flavours:

Feedback Type	Mechanism	Example Implementation
Explicit	Store (query, doc_id, relevance) in a lightweight DB (SQLite, Redis). Use it to re-rank or boost certain docs in future queries.	<code>FeedbackStore.add_feedback(query, doc_id, 1)</code>
Pseudo-Relevance	Assume the top-k results are relevant, extract key terms, and expand the query (e.g., with RM3).	<code>PRF.expand(query, top_docs)</code>

2.4.1 Explicit Feedback Store

```

import sqlite3
from pathlib import Path

class FeedbackStore:
    def __init__(self, db_path: Path = Path("feedback.db")):
        self.conn = sqlite3.connect(db_path)
        self.conn.execute(
            """CREATE TABLE IF NOT EXISTS feedback (
                query TEXT,
                doc_id TEXT,
                relevance INTEGER,
                ts TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            )"""
        )
        self.conn.commit()

    def add_feedback(self, query: str, doc_id: str, relevance: int):
        self.conn.execute(
            "INSERT INTO feedback (query, doc_id, relevance) VALUES (?, ?, ?)",
            (query, doc_id, relevance),
        )
        self.conn.commit()

    def get_feedback(self, query: str) -> List[Tuple[str, int]]:
        cur = self.conn.execute(
            "SELECT doc_id, relevance FROM feedback WHERE query = ?", (query,)
        )
        return cur.fetchall()

```

During retrieval we **boost** any document that has a positive relevance flag for the same query (or a near-duplicate query).

```

def boost_with_feedback(query: str, docs: List[Document], store: FeedbackStore) -> List[Document]:
    feedback = {doc_id: rel for doc_id, rel in store.get_feedback(query)}
    for d in docs:
        if d.metadata.get("doc_id") in feedback and feedback[d.metadata["doc_id"]] > 0:
            # Simple boost: add a large constant to the score stored in metadata
            d.metadata["score"] = d.metadata.get("score", 0) + 1000
    # Re-sort
    return sorted(docs, key=lambda d: d.metadata.get("score", 0), reverse=True)

```

2.4.2 Pseudo-Relevance Feedback (RM3)

```
from rank_bm25 import BM25Okapi
from collections import Counter
import nltk
nltk.download('stopwords')
from nltk.corpus import stopwords

STOPWORDS = set(stopwords.words('english'))

def rm3_expand(query: str, top_docs: List[Document], alpha: float = 0.5, top_terms: int = 10) -> str:
    # 1. Tokenise top docs
    tokenised = [doc.page_content.lower().split() for doc in top_docs]
    # 2. Compute term frequencies
    term_counts = Counter()
    for tokens in tokenised:
        term_counts.update([t for t in tokens if t not in STOPWORDS])
    # 3. Pick top terms
    expansion_terms = [t for t, _ in term_counts.most_common(top_terms)]
    # 4. Interpolate with original query
    expanded = " ".join(query.split() + expansion_terms)
    return expanded
```

You can feed the expanded query back into the **hybrid retriever** for a second pass, or simply re-rank the same set with boosted term weights.

2.5 Dynamic Query Rewriting

Sometimes the user's natural language query is **under-specified** (e.g., "Tell me about the 2022 World Cup"). An LLM can rewrite it into a **more retrieval-friendly** prompt by:

- Adding **temporal constraints** ("as of 2022"),
- Converting **ambiguous pronouns** ("it" → "the tournament"),
- Extracting **key entities** for BM25 boosting.

LangChain's `LLMChain` + `PromptTemplate` makes this trivial.

```

from langchain import LLMChain, PromptTemplate
from langchain.llms import OpenAI

template = PromptTemplate(
    input_variables=["question"],
    template="""
You are a helpful assistant that rewrites user questions into a form that maximizes retrieval performance.
Do NOT answer the question; just output the rewritten query.
User question: {question}
Rewritten query: ""
    """
)

rewriter = LLMChain(llm=OpenAI(temperature=0.0), prompt=template)

def rewrite_query(question: str) -> str:
    return rewriter.run(question).strip()

```

Pipeline placement - The rewriter sits **before** the hybrid retriever. You can optionally conditionally invoke it (e.g., only when the query length < 5 tokens, or when a confidence threshold from a classifier is low).

2.6 Performance-Aware Design

Metric	Typical Target (Production)	Where Bottlenecks Appear
Latency (p95)	≤ 300 ms (including LLM call)	Cross-encoder inference, network I/O to vector DB
Throughput	≥ 100 QPS (depends on hardware)	Synchronous calls, lack of batching
Memory Footprint	≤ 2 GB per worker (dense model + vector cache)	Large embedding models loaded per process

Best-practice checklist

- Warm-up caches** - Load embeddings and BM25 index at service start.
- Async I/O** - Use `await` on vector DB calls (`async_client`) and LLM calls (`openai.AsyncOpenAI`).
- Batch cross-encoder** - Pass the whole `(query, doc)` batch to the model in one

forward pass.

4. **Circuit-breaker** - If any component exceeds a latency SLA, fall back to a **light** path (e.g., dense-only retrieval).

5. **Observability** - Hook LangChain's `CallbackManager` to emit Prometheus metrics for each stage.

```
from langchain.callbacks import CallbackManager, get_openai_callback

cb_manager = CallbackManager()
cb_manager.add_handler(get_openai_callback())
# Pass `callback_manager=cb_manager` to every chain component.
```

3.2 Practical Application

We will now **assemble a full RAG pipeline** that:

1. Rewrites the user query.
2. Performs hybrid dense + BM25 retrieval.
3. Reranks with a cross-encoder.
4. Applies post-processing (dedup + LLM summarisation).
5. Incorporates explicit relevance feedback.
6. Benchmarks latency/throughput.

All code snippets are **self-contained**; you can paste them into a Jupyter notebook or a Python module.

3.1 Dataset & Index Preparation

For the demo we use a **subset of Wikipedia** (~10⁴ articles). The steps are:

```
import json, pathlib, tqdm
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.document_loaders import TextLoader
from langchain.schema import Document
from langchain.vectorstores import Chroma
from langchain.embeddings import SentenceTransformerEmbeddings

DATA_DIR = pathlib.Path("./wiki_subset")
CHUNK_SIZE = 500
OVERLAP = 100

# 1 Load raw articles
raw_files = list(DATA_DIR.glob("*.txt"))
documents = []
for f in tqdm.tqdm(raw_files, desc="Loading"):
    loader = TextLoader(str(f))
    docs = loader.load()
    documents.extend(docs)

# 2 Split into chunks
splitter = RecursiveCharacterTextSplitter(chunk_size=CHUNK_SIZE, chunk_overlap=OVERLAP)
chunks = splitter.split_documents(documents)

# 3 Embed + store in Chroma (persistent)
embedder = SentenceTransformerEmbeddings(model_name="all-MiniLM-L6-v2")
vectorstore = Chroma.from_documents(chunks, embedder, collection_name="wiki_chunks")
```

Tip - Persist the Chroma collection (`persist_directory="./chroma_data"`).
This lets you spin up the retrieval service instantly without re-embedding.

3.2 Building the Hybrid Retriever

```

from langchain.retrievers import MultiRetriever
from typing import List
import numpy as np

# BM25 retriever (reuse the same chunks)
bm25_retriever = BM25Retriever(corpus=chunks, k=10)

# Dense retriever (already provided by Chroma)
dense_retriever = vectorstore.as_retriever(search_kwargs={"k": 10})

def reciprocal_rank_fusion(docs: List[Document], k: int = 60) -> List[Document]:
    """
    Input `docs` is a flat list where each Document carries a `metadata["retriever"]`
    field indicating its origin and a `metadata["rank"]` field (1-based rank).
    The function returns a new list sorted by RRF score.
    """
    # Build a mapping doc_id → list of (source, rank)
    score_map = {}
    for d in docs:
        doc_id = d.metadata["doc_id"]
        source = d.metadata["retriever"]
        rank = d.metadata["rank"]
        score_map.setdefault(doc_id, []).append(1.0 / (k + rank))

    # Attach final RRF score to each Document (pick first occurrence for content)
    scored_docs = []
    for d in docs:
        doc_id = d.metadata["doc_id"]
        rrf = sum(score_map[doc_id])
        d.metadata["score"] = rrf
        scored_docs.append(d)

    # Deduplicate by doc_id while preserving highest RRF
    uniq = {}
    for d in scored_docs:
        if d.metadata["doc_id"] not in uniq or d.metadata["score"] >
            uniq[d.metadata["doc_id"]].metadata["score"]:
            uniq[d.metadata["doc_id"]] = d

    # Return sorted list
    return sorted(uniq.values(), key=lambda d: d.metadata["score"], reverse=True)

```

... (continued on next page)

```
# Helper to inject source/rank metadata
def wrap_with_meta(retriever, name):
    class Wrapper(BaseRetriever):
        def get_relevant_documents(self, query: str) -> List[Document]:
            docs = retriever.get_relevant_documents(query)
            for idx, d in enumerate(docs, start=1):
                d.metadata["retriever"] = name
                d.metadata["rank"] = idx
                d.metadata["doc_id"] = d.metadata.get("doc_id", f"{name}_{idx}")
            return docs
    return Wrapper()

hybrid = MultiRetriever(
    retrievers=[
        wrap_with_meta(dense_retriever, "dense"),
        wrap_with_meta(bm25_retriever, "bm25")
    ],
    combine_documents_fn=reciprocal_rank_fusion,
    # optional: parallel=True (default)
)
```

3.3 Adding the Cross-Encoder Reranker

```
from langchain.retrievers import CrossEncoderRerank

cross_encoder = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
reranker = CrossEncoderRerank(cross_encoder=cross_encoder, top_n=5)

def rerank(query: str, docs: List[Document]) -> List[Document]:
    return reranker.rerank(query, docs)
```

3.4 Post-Processing (Dedup + LLM Summariser)

```

from langchain.llms import OpenAI
from langchain.document_compressors import LLMChainDocumentCompressor
from langchain.prompts import PromptTemplate

compress_prompt = PromptTemplate(
    input_variables=["text"],
    template="""
Summarise the following passage in no more than 120 words while preserving all factual statements.
Passage:
{text}
Summary: """
)

compressor = LLMChainDocumentCompressor(
    llm=OpenAI(temperature=0.0),
    prompt=compress_prompt
)

def post_process(docs: List[Document]) -> List[Document]:
    # 1 Simple dedup based on raw text hash
    uniq = {}
    for d in docs:
        h = hash(d.page_content)
        if h not in uniq:
            uniq[h] = d
    # 2 LLM summarisation (batch)
    return compressor.compress_documents(list(uniq.values()))

```

3.5 Feedback Store & Boosting

```

feedback_store = FeedbackStore(db_path=pathlib.Path("feedback.db"))

def apply_feedback(query: str, docs: List[Document]) -> List[Document]:
    # Pull explicit feedback for the exact query (or use fuzzy matching later)
    return boost_with_feedback(query, docs, feedback_store)

```

3.6 Query Rewriter

```
rewriter = LLMChain(  
    llm=OpenAI(temperature=0.0),  
    prompt=PromptTemplate(  
        input_variables=["question"],  
        template="""  
You are a query optimiser. Rewrite the user's question to be as specific as possible for a search engine.  
User question: {question}  
Optimised query: """  
        )  
    )  
  
def rewrite(question: str) -> str:  
    return rewriter.run(question).strip()
```

3.7 End-to-End Retrieval Pipeline

```

from langchain.chains import RetrievalQA
from langchain.llms import OpenAI

class HybridRAGPipeline:
    """Encapsulates the full retrieval-augmented generation flow."""
    def __init__(self):
        self.llm = OpenAI(temperature=0.2, max_tokens=512)
        self.callback_manager = CallbackManager() # for observability

    def __call__(self, user_question: str) -> str:
        # 0 Rewrite
        rewritten = rewrite(user_question)

        # 1 Hybrid retrieval
        raw_docs = hybrid.get_relevant_documents(rewritten)

        # 2 Apply explicit feedback boost
        boosted_docs = apply_feedback(rewritten, raw_docs)

        # 3 Cross-encoder rerank
        reranked = rerank(rewritten, boosted_docs)

        # 4 Post-process (dedup + summarise)
        final_docs = post_process(reranked)

        # 5 Build context for LLM (concatenate)
        context = "\n\n".join([d.page_content for d in final_docs])

        # 6 Prompt LLM (simple RAG prompt)
        prompt = f"""Answer the following question using ONLY the provided context.
        If the answer is not in the context, say "I don't know."

        Context:
        {context}

        Question: {user_question}
        Answer: """

        # 7 Invoke LLM (with callbacks for latency)
        with get_openai_callback() as cb:

```

... (continued on next page)

```
        answer = self.llm(prompt)
        latency_ms = cb.total_cost # just illustrative; real metric via cb

    # 8 Return answer + optional diagnostics
    return answer

# Instantiate once (e.g., at service start)
rag = HybridRAGPipeline()
```

Calling the pipeline

```
question = "Who won the Nobel Peace Prize in 2023 and why?"
answer = rag(question)
print(answer)
```

3.8 Benchmarking Latency & Throughput

We will use ``locust`` for load testing and **LangChain callbacks** for per-stage timing.

```
# benchmark.py
import time
import asyncio
from concurrent.futures import ThreadPoolExecutor
from rag_pipeline import rag # the class defined above

def single_query(q):
    start = time.time()
    ans = rag(q)
    elapsed = time.time() - start
    return elapsed, ans

# Warm-up
_ = rag("warm up query")

# 10 Simple latency test (100 random queries)
import random, json
with open("sample_questions.json") as f:
    queries = json.load(f)

latencies = []
for q in random.sample(queries, 100):
    lat, _ = single_query(q)
    latencies.append(lat)

print(f"P95 latency: {np.percentile(latencies, 95) * 1000:.1f} ms")
print(f"Mean latency: {np.mean(latencies) * 1000:.1f} ms")
```

Throughput with async workers

```

async def async_worker(q):
    loop = asyncio.get_event_loop()
    # Offload the sync rag call to a thread pool (LangChain is sync-heavy)
    return await loop.run_in_executor(pool, single_query, q)

async def load_test(queries, concurrency=50):
    global pool
    pool = ThreadPoolExecutor(max_workers=concurrency)
    tasks = [async_worker(q) for q in queries]
    results = await asyncio.gather(*tasks)
    pool.shutdown(wait=True)
    return results

# Run 1000 queries with 50 concurrent workers
queries = random.choices(queries, k=1000)
start = time.time()
results = asyncio.run(load_test(queries, concurrency=50))
throughput = len(results) / (time.time() - start)
print(f"Throughput: {throughput:.1f} QPS")

```

Interpreting the numbers

Metric	Target (example)	What to do if you miss it
P95 latency	≤ 300 ms	• Move cross-encoder to GPU • Reduce k for dense/BM25 (e.g., $5 \rightarrow 5$) • Cache LLM summarisation results
Throughput	≥ 100 QPS	• Add a request queue + worker pool • Batch cross-encoder scores (send 50 docs at once) • Horizontal scale: spin up multiple replica services behind a load balancer
Error rate	< 0.5 %	• Implement circuit-breaker fallback to dense-only mode • Add retries with exponential back-off for vector DB calls

3.9 Deploying as a FastAPI Service

```
# api.py
from fastapi import FastAPI, Body
from pydantic import BaseModel

app = FastAPI()
pipeline = rag # imported from previous cell

class QueryRequest(BaseModel):
    question: str

@app.post("/rag")
def query(req: QueryRequest):
    answer = pipeline(req.question)
    return {"answer": answer}
```

Run with `uvicorn api:app --host 0.0.0.0 --port 8000 --workers 4`. Each worker holds its own copy of the vector store and the cross-encoder, so **memory scaling** must be accounted for (≈ 2 GB per worker for the MiniLM cross-encoder + Chroma cache).

4 Key Takeaways

Concept	What to Remember	Typical Code
Composable retrieval	Every stage is a pure function <code>query → List[Document]</code> .	<code>retriever =</code>
Hybrid search	Combine dense (semantic) + BM25 (lexical) → higher recall.	<code>hybrid = Mu</code> <code>combine_doc</code>
Cross-encoder reranking	Fine-grained relevance; batch for speed.	<code>reranker.re</code>
Post-processing	Dedup + LLM summarisation to fit LLM context window.	<code>compressor.</code>
Feedback loops	Store <code>(query, doc_id, relevance)</code> → boost future scores.	<code>feedback_st</code>

Concept	What to Remember	Typical Code
Dynamic rewriting	LLM rewrites ambiguous queries before search.	<code>rewriter.run</code>
Performance	Async I/O, batching, circuit-breaker, observability via callbacks.	<code>CallbackManager</code>
Deployment	Wrap pipeline in FastAPI; scale horizontally; monitor latency.	<code>uvicorn api</code>

Quick Cheat-Sheet (copy-paste)

```
# 1️⃣ Rewriter
rewriter = LLMChain(OpenAI(temperature=0), PromptTemplate(...))

# 2️⃣ Hybrid retriever
dense = vectorstore.as_retriever(search_kwargs={"k": 5})
bm25 = BM25Retriever(corpus=chunks, k=5)
hybrid = MultiRetriever([wrap(dense, "dense"), wrap(bm25, "bm25")],
                        combine_documents_fn=reciprocal_rank_fusion)

# 3️⃣ Reranker
reranker = CrossEncoderRerank(CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2"), top_n=5)

# 4️⃣ Post-process
compressor = LLMChainDocumentCompressor(OpenAI(), PromptTemplate(...))

# 5️⃣ Feedback
feedback = FeedbackStore()

# 6️⃣ Full pipeline
def rag(question):
    q = rewriter.run(question).strip()
    docs = hybrid.get_relevant_documents(q)
    docs = boost_with_feedback(q, docs, feedback)
    docs = reranker.rerank(q, docs)
    docs = compressor.compress_documents(docs)
    context = "\n\n".join(d.page_content for d in docs)
    return OpenAI().invoke(f"Answer using only this context:\n{context}\n\nQ: {question}\nA:")
```

Exercises

1. **Swap the dense model** - Replace `all-MiniLM-L6-v2` with a larger model (e.g., `sentence-transformers/multi-qa-mpnet-base-dot-v1`). Measure the impact on latency and recall.
 2. **Implement Weighted Fusion** - Write a fusion function that uses a configurable weight `w_dense`. Compare RRF vs weighted sum on a held-out test set (e.g., TriviaQA).
 3. **Add a "time-aware" BM25** - Store a `year` field in each Wikipedia chunk and boost BM25 scores when the query contains a year (e.g., "2022").
 4. **Create a "fallback" chain** - If the total latency of the hybrid+rerank exceeds 300 ms, automatically return the result of the dense-only retriever. Use LangChain's `ConditionalChain`.
 5. **Deploy to Docker** - Containerise the FastAPI service with a GPU-enabled base image, and write a `docker-compose.yml` that spins up the API, a Qdrant vector DB, and a Redis instance for the feedback store.
-

What's Next?

In **Module 5** we will dive into "**Grounding LLMs with Structured Knowledge Graphs**", extending the retrieval pipeline to reason over entities and relations. You'll learn how to:

- Convert retrieved text into **triples** using an LLM.
- Store triples in a **graph database** (Neo4j, Weaviate).
- Perform **graph-augmented retrieval** and **chain-of-thought prompting** for complex reasoning tasks.

Stay tuned - the next chapter builds directly on the hybrid pipeline you just built, adding a new dimension of **structured** grounding!

Integrating Large Language Models for Generation

Module 5 of 9 - "Learn Retrieval- Augmented Generation in Python to Build Production-Ready Knowledge-Grounded AI Systems"

Table of Contents

Section	What you'll learn
Introduction	Why generation is the "brain" of a RAG system and how it differs from pure retrieval.
Core Concepts	LLM service options, generation knobs, streaming fundamentals, and context-window management techniques.
Practical Application	Step-by-step code for connecting to OpenAI, Anthropic, and self-hosted Hugging-Face models; configuring parameters; streaming and stitching responses; handling window limits with chunking & sliding-window logic; building a production-ready Flask endpoint.
Exercises	Hands-on tasks to cement the concepts.
Key Takeaways	A concise cheat-sheet you can refer to when you leave this chapter.

Introduction

Retrieval-Augmented Generation (RAG) is a two-stage dance:

1. **Retrieve** the most relevant knowledge from a vector store (Modules 3-4).
2. **Generate** a fluent, fact-grounded answer using a Large Language Model (LLM).

Until now we have focused on the **where** (the vector database) and the **how** (the retrieval pipeline). In this chapter we shift our attention to the **what** - the LLM that will actually **write** the answer.

A good generator must:

Requirement	Why it matters for RAG
Deterministic grounding	The answer should stay faithful to the retrieved documents.
Control over randomness	Temperature/top-p let you trade creativity for factuality.
Scalable latency	Production services often need sub-second responses, even for long passages.
Token-level streaming	Streaming enables UI-friendly "type-ahead" displays and reduces perceived latency.
Context-window awareness	LLMs have a hard limit on how many tokens they can see at once; we must fit the query + retrieved chunks inside that window.

The rest of this chapter shows you **how** to meet all of those requirements using three common ways to access an LLM:

- **Hosted APIs** - OpenAI's `gpt-4o`, Anthropic's `claude-3-sonnet`.
- **Self-hosted models** - any model on the HuggingFace Hub (e.g., Llama 2-13B, Mistral-7B).
- **Hybrid setups** - a local model for cheap, high-throughput generation paired with a hosted model for complex reasoning.

By the end you will be able to plug any of these generators into the retrieval pipelines you built in Modules 3-4, fine-tune the generation parameters for factual grounding, and ship a production-ready endpoint that streams answers back to the user.

Core Concepts

1. LLM Service Landscape

Service	Access method	Pricing model	Typical max context	Notable features
OpenAI	<code>openai</code> Python SDK (REST)	Pay-per-token (prompt + completion)	128k tokens (GPT-4-turbo)	Built-in function calling, safety filters
Anthropic	<code>anthropic</code> Python SDK (REST)	Pay-per-token	200k tokens (Claude-3)	"System" messages, easy-to-use <code>max_tokens</code>
HuggingFace Transformers	<code>transformers</code> library (local or <code>text-generation-inference</code> server)	Free (compute cost)	Model-specific, commonly 4-8k, some 32k	Full control over tokenizer, quantization, LoRA adapters
Cohere, Mistral AI, Google Vertex AI	Similar REST SDKs	Vary	128k-256k (newer models)	Often provide "embed-first" pipelines

Tip: When you start a prototype, use the hosted API that matches your budget and latency expectations. When you hit the cost ceiling, switch to a self-hosted model that you can quantize or distill.

2. Generation Parameters - The "Knobs"

Parameter	Typical range	Effect on output	When to adjust for RAG
<code>`temperature`</code>	0.0 to 2.0	Controls randomness; 0 = deterministic	Set low (0.0-0.2) for factual answers; raise a bit (0.5)

Parameter	Typical range	Effect on output	When to adjust for RAG
			when you need creative phrasing
<code>`top_p`</code> (nucleus sampling)	0.0 $\square$ 1.0	Limits token probability mass; 1 = no truncation	Combine with low temperature for tighter grounding
<code>`max_tokens`</code> / <code>`max_new_tokens`</code>	Depends on model	Hard stop; prevents runaway generations	Compute remaining window after query + retrieved chunks (see § 3)
<code>`stop`</code>	List of strings	Forces the model to stop when any token matches	Use <code>["\n\n"]</code> or custom delimiters (<code><END></code>) to cut off at the end of a paragraph
<code>`presence_penalty`</code> / <code>`frequency_penalty`</code> (OpenAI)	-2 $\square$ 2	Discourages/rewards token reuse	Rarely needed for RAG, but can reduce repetition in long answers
<code>`logprobs`</code> (OpenAI)	0 $\square$ 5	Returns token-level probabilities for debugging	Helpful for post-hoc fact-checking

Best-practice rule: Start with ``temperature=0.0`` and `top_p=1.0`. If the output feels too robotic, increase temperature in increments of 0.1 and observe the trade-off.

3. Token-Level Streaming

Most modern APIs support **streaming**-the model sends each token (or token batch) as soon as it is generated.

- Why stream?

- UI can display partial answer instantly → better UX.
- Server can start forwarding data to downstream services (e.g., logging, analytics).
- Reduces memory pressure for very long completions.
- **Common patterns**
- **OpenAI:** `openai.ChatCompletion.create(..., stream=True)` yields a generator of `delta` objects.
- **Anthropic:** `client.completions.create(..., stream=True)` yields `MessageDelta` objects.
- **HF `transformers`:** Use `TextStreamer` or `StoppableStreamer` with `generate(..., streamer=streamer)`.
- **Stitching** - The raw stream contains incremental fragments ("Hello", " world", "!"). You must concatenate them **safely** to avoid duplicate spaces or broken Unicode characters. The helper `stream_to_text()` shown later does exactly that.

4. Context-Window Management

LLMs have a **hard limit** on the number of tokens they can attend to at once. For a model with a 16k token window, the total token count of:

```
prompt (including system + user messages) + retrieved chunks + max_new_tokens
```

must be $\leq 16\text{ k}$.

4.1 Token Counting

- **OpenAI:** `tiktoken.encoding_for_model("gpt-4o")`.
- **Anthropic:** `anthropic` SDK provides `tokenizer` utilities.
- **HF:** `tokenizer.encode(..., add_special_tokens=False)` returns a list of token IDs.

4.2 Chunking Strategies

1. **Simple truncation** - Keep the most recent N tokens; drop the rest.
2. **Chunk ranking + top-K** - Rank retrieval results by similarity, keep top-K until you reach the window limit.
3. **Sliding-window (overlap) technique** - When the retrieved set is larger than the window, create overlapping windows that each fit inside the limit and generate partial answers for each, then **merge**.

```
# Pseudo-algorithm
window = max_context - len(prompt_tokens) - reserved_for_completion
selected_chunks = []
for chunk in ranked_chunks:
    if len(selected_chunks) + len(chunk.tokens) <= window:
        selected_chunks.append(chunk)
    else:
        break
```

4.3 Overlap & Merging

When you must split the context into multiple windows (e.g., 50k tokens of evidence for a 16k model), you can:

- **Generate a summary per window** → concatenate summaries → ask LLM to synthesize.
- **Generate answer per window** → use a second "merger" LLM call to combine the partial answers, ensuring consistency.

5. Production-Ready Concerns

Concern	Mitigation
Latency spikes	Cache recent retrieval results; use async I/O (<code>httpx.AsyncClient</code>).
Rate limits / quotas	Exponential back-off (<code>retry</code> library) and

Concern	Mitigation
	request throttling (<code>asyncio.Semaphore</code>).
Cost overruns	Log token usage per request; enforce a hard <code>max_tokens</code> budget; switch to cheaper model when usage > threshold.
Security	Store API keys in environment variables or secret managers; avoid logging raw prompts.
Observability	Emit structured logs (<code>jsonlog</code>), metrics (<code>prometheus_client</code>), and tracing (<code>OpenTelemetry</code>).

Practical Application

Below we walk through a **complete end-to-end RAG generation** that:

1. Accepts a user query via a Flask endpoint.
2. Retrieves the top-k relevant chunks from a vector DB (assume you already have a `retriever` from Module 4).
3. Packs the query + chunks into a prompt that respects the model's context window.
4. Calls the chosen LLM (OpenAI, Anthropic, or HF) with streaming enabled.
5. Streams the answer back to the HTTP client while also logging token usage.

Assumptions:

* You have Python 3.10+ installed.

You have installed the required packages (see the Setup* block).

* Your vector store is already populated and you have a `retriever` object exposing a `get_relevant_documents(query, k)` method that returns a list

of objects with `page_content` and `metadata`.

5.1 Setup

```
# Core dependencies
pip install flask openai anthropic transformers sentencepiece tiktoken \
    httpx tqdm python-dotenv

# Optional - for async & retries
pip install httpx[http2] tenacity
```

Create a `.env` file in the project root:

```
# .env
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=claude-...
HF_MODEL_ID=meta-llama/Llama-2-13b-chat-hf # optional
HF_TOKEN=hf_...
```

Load env vars in code:

```
from dotenv import load_dotenv
load_dotenv()
```

5.2 Helper Functions

5.2.1 Token Counting

```
import tiktoken
from transformers import AutoTokenizer

# Choose encoder based on model name
def get_encoder(model_name: str):
    if "gpt" in model_name.lower():
        return tiktoken.encoding_for_model(model_name)
    elif "claude" in model_name.lower():
        # Anthropic uses the same tokenizer as GPT for most models
        return tiktoken.get_encoding("cl100k_base")
    else:
        # HuggingFace - load tokenizer once
        return AutoTokenizer.from_pretrained(model_name, use_fast=True)

def num_tokens(text: str, model_name: str) -> int:
    enc = get_encoder(model_name)
    if isinstance(enc, tiktoken.Encoding):
        return len(enc.encode(text))
    else: # HF tokenizer
        return len(enc.encode(text, add_special_tokens=False))
```

5.2.2 Prompt Construction

```

def build_prompt(
    query: str,
    docs: list,
    model_name: str,
    system_msg: str = "You are an AI assistant that answers questions using only the provided context.",
    max_new_tokens: int = 1024,
) -> str:
    """
    Assemble a chat-style prompt that fits within the model's context window.
    """

    # 1 System + user
    messages = [
        {"role": "system", "content": system_msg},
        {"role": "user", "content": query},
    ]

    # 2 Append retrieved docs as a single "context" block
    context_block = "\n---\n".join([doc.page_content for doc in docs])
    context_msg = {"role": "assistant", "content": f"Context:\n{context_block}"}
    messages.append(context_msg)

    # 3 Estimate token usage and truncate if needed
    encoder = get_encoder(model_name)
    total_tokens = sum(
        num_tokens(m["content"], model_name) + 4 # rough per-message overhead
        for m in messages
    )

    # Reserve space for the model's completion
    max_context = getattr(encoder, "max_model_len", 8192) # fallback
    allowed = max_context - max_new_tokens - 50 # safety margin

    if total_tokens > allowed:
        # Simple truncation: drop oldest docs until we fit
        while total_tokens > allowed and len(docs) > 0:
            docs.pop() # remove last (least relevant) doc
            context_block = "\n---\n".join([doc.page_content for doc in docs])
            context_msg["content"] = f"Context:\n{context_block}"
            total_tokens = sum(
                num_tokens(m["content"], model_name) + 4 for m in messages
            )

```

... (continued on next page)

```
# Re-assemble messages after truncation
messages = messages[:2] + [context_msg]

return messages
```

Note: The `+4` overhead is a heuristic derived from OpenAI's token accounting (role + delimiters). Adjust for other providers if you notice mismatches.

5.2.3 Stream Stitcher

```
def stream_to_text(stream):
    """
    Convert a generator of partial deltas (OpenAI/Anthropic/HF) into a single string.
    Handles spaces and Unicode gracefully.
    """
    final_text = ""
    for chunk in stream:
        # The chunk is expected to be a dict with a "delta" or "text" field
        if isinstance(chunk, dict):
            delta = chunk.get("delta") or chunk.get("text")
        else:
            delta = chunk # HF streamer yields raw strings

        if delta:
            final_text += delta
            yield final_text # yield incremental view for real-time UI
```

5.3 LLM Connectors

5.3.1 OpenAI

```
import openai
import os

openai.api_key = os.getenv("OPENAI_API_KEY")

def openai_chat_stream(messages, model="gpt-4o-mini", temperature=0.0,
                       top_p=1.0, max_tokens=1024, stop=None):
    """
    Returns a generator that yields incremental text.
    """
    response = openai.ChatCompletion.create(
        model=model,
        messages=messages,
        temperature=temperature,
        top_p=top_p,
        max_tokens=max_tokens,
        stop=stop,
        stream=True,          # <-- enables token-level streaming
    )
    # Each `chunk` is a dict with `choices[0].delta.content`
    for chunk in response:
        delta = chunk["choices"][0]["delta"].get("content", "")
        if delta:
            yield {"delta": delta}
```

5.3.2 Anthropic

```
import anthropic
import os

client = anthropic.Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))

def anthropic_chat_stream(messages, model="claude-3-sonnet-20240229",
                          temperature=0.0, max_tokens=1024, stop=None):
    """
    Anthropic expects a list of dicts with `role` and `content`.
    Streaming yields MessageDelta objects.
    """
    response = client.messages.create(
        model=model,
        messages=messages,
        temperature=temperature,
        max_tokens=max_tokens,
        stop_sequences=stop,
        stream=True,
    )
    for delta in response:
        # delta is a `MessageDelta` with `.type == "content_block_delta"`
        if delta.type == "content_block_delta":
            yield {"delta": delta.delta.text}
```

5.3.3 Hugging² FaceTransformers (Local)

```

from transformers import AutoModelForCausalLM, AutoTokenizer, TextStreamer
import torch

def hf_chat_stream(messages, model_id="meta-llama/Llama-2-13b-chat-hf",
                    temperature=0.0, max_new_tokens=1024, device="cpu"):
    """
    We convert the chat into a single prompt string because most
    open-source chat models expect a plain text prompt with special tokens.
    """
    tokenizer = AutoTokenizer.from_pretrained(model_id, use_fast=True)
    model = AutoModelForCausalLM.from_pretrained(
        model_id,
        torch_dtype=torch.float16 if device != "cpu" else torch.float32,
        device_map="auto" if device != "cpu" else None,
    )
    # Build prompt - many chat models use <s>[INST] ... [/INST] syntax
    # For Llama-2 chat: <s>[INST] <<system>> <<user>> [/INST]
    system = messages[0]["content"]
    user = messages[1]["content"]
    context = messages[2]["content"] if len(messages) > 2 else ""
    prompt = f"<s>[INST] <<SYS>>\n{system}\n</SYS>>\n\n{user}\n\n{context}\n[/INST]"

    input_ids = tokenizer.encode(prompt, return_tensors="pt").to(model.device)

    streamer = TextStreamer(tokenizer, skip_prompt=True, skip_special_tokens=True)

    generation_kwargs = dict(
        input_ids=input_ids,
        max_new_tokens=max_new_tokens,
        temperature=temperature,
        top_p=1.0,
        streamer=streamer,
        do_sample=temperature > 0.0,
    )
    # `generate` runs in the background and pushes tokens to `streamer`
    # We wrap it in a generator that yields the streamed text
    generation_thread = torch.jit.fork(model.generate, **generation_kwargs)

    # Streamer yields token strings as they become available
    for token in streamer:

```

... (continued on next page)

```
yield {"delta": token}
```

Performance note: For production you will most likely spin up a separate inference server (e.g., `text-generation-inference`) and call it via HTTP rather than loading the model in the same process. The code above is educational and works for modest workloads.

5.4 Putting It All Together - Flask Endpoint

```

from flask import Flask, request, Response, stream_with_context
import json
import time

app = Flask(__name__)

# -----
# Configuration - switch these flags to test different providers
# -----
USE_PROVIDER = os.getenv("LLM_PROVIDER", "openai") # "openai" | "anthropic" | "hf"
MODEL_NAME = os.getenv("LLM_MODEL", "gpt-4o-mini") # adjust per provider
MAX_NEW_TOKENS = 1024
TEMPERATURE = 0.0
TOP_P = 1.0
STOP_SEQS = ["\n\n"] # simple stop token

# -----
# Retrieval placeholder - replace with your actual retriever
# -----
def retrieve(query, k=5):
    # Example stub - in real code you call your LangChain retriever
    # Return list of objects with `page_content` attribute
    from collections import namedtuple
    Doc = namedtuple("Doc", ["page_content"])
    # Dummy docs for illustration
    return [Doc(page_content=f"Document {i} about {query}") for i in range(k)]

# -----
# Route
# -----
@app.route("/chat", methods=["POST"])
def chat():
    payload = request.get_json()
    query = payload.get("question")
    if not query:
        return {"error": "Missing 'question' field"}, 400

    # 1. Retrieve relevant docs
    docs = retrieve(query, k=5)

```

... (continued on next page)

```
# 2 Build a prompt that respects the context window
messages = build_prompt(
    query=query,
    docs=docs,
    model_name=MODEL_NAME,
    max_new_tokens=MAX_NEW_TOKENS,
    system_msg="You are a helpful assistant. Answer using ONLY the provided context.",
)

# 3 Choose the correct LLM connector
if USE_PROVIDER == "openai":
    stream_gen = openai_chat_stream(
        messages=messages,
        model=MODEL_NAME,
        temperature=TEMPERATURE,
        top_p=TOP_P,
        max_tokens=MAX_NEW_TOKENS,
        stop=STOP_SEQS,
    )
elif USE_PROVIDER == "anthropic":
    stream_gen = anthropic_chat_stream(
        messages=messages,
        model=MODEL_NAME,
        temperature=TEMPERATURE,
        max_tokens=MAX_NEW_TOKENS,
        stop=STOP_SEQS,
    )
elif USE_PROVIDER == "hf":
    stream_gen = hf_chat_stream(
        messages=messages,
        model_id=MODEL_NAME,
        temperature=TEMPERATURE,
        max_new_tokens=MAX_NEW_TOKENS,
    )
else:
    return {"error": f"Unsupported provider {USE_PROVIDER}"}, 500

# 4 Stream back to the client as Server-Sent Events (SSE)
def event_stream():
    start = time.time()
```

... (continued on next page)

```

total_output = ""
for chunk in stream_to_text(stream_gen):
    # Log partial output for debugging
    total_output = chunk
    # SSE format: "data: <json>\n\n"
    yield f"data: {json.dumps({'content': total_output})}\n\n"
# End-of-stream marker
duration = time.time() - start
# Log usage (real implementation would also capture token counts)
app.logger.info(
    f"Query: '{query[:30]}...' | Provider: {USE_PROVIDER} | "
    f"Tokens out (est): {len(total_output.split())} | Time: {duration:.2f}s"
)
yield "event: done\ndata: {}\n\n"

return Response(stream_with_context(event_stream()), mimetype="text/event-stream")

# -----
if __name__ == "__main__":
    # For dev only; use gunicorn/uvicorn in prod
    app.run(host="0.0.0.0", port=8000, debug=True)

```

How it works

- **POST /chat** expects JSON: `{"question": "What is RAG?"}`.
- The endpoint pulls the top-k relevant documents, builds a prompt that **fits** into the model's context window, and calls the appropriate LLM connector with **stream=True**.
- The generator `event_stream()` yields Server-Sent Events (SSE) so a JavaScript front-end can display the answer token-by-token (EventSource API).
- Token usage and latency are logged for observability.

5.5 Advanced Topics

5.5.1 Hybrid Retrieval-Generation

When you have **massive** evidence (e.g., 100k tokens), a single model call won't fit. A common pattern:

1. **Chunk the evidence** into overlapping windows (e.g., 8k tokens each, 2k overlap).
2. **Generate a short "summary"** for each window using a cheap model (Llama 2-7B).
3. **Feed the collection of summaries** to a more capable model (GPT-4o) to produce the final answer.

```
def chunk_documents(docs, window=8000, overlap=2000):
    # Flatten all docs into one big string
    big_text = "\n---\n".join([d.page_content for d in docs])
    tokenizer = get_encoder(MODEL_NAME)
    tokens = tokenizer.encode(big_text)
    chunks = []
    for i in range(0, len(tokens), window - overlap):
        chunk_tokens = tokens[i : i + window]
        chunks.append(tokenizer.decode(chunk_tokens))
    return chunks
```

You can then loop over `chunks`, call the cheap model with `max_new_tokens=256`, collect the partial summaries, and finally call the expensive model once with the concatenated summaries.

5.5.2 Function Calling / Tool Use

Both OpenAI and Anthropic support **function calling** (OpenAI) or **tool use** (Anthropic). This lets the LLM decide to invoke a Python function (e.g., a calculator, a database query) when it needs precise data.

```
# OpenAI example
functions = [
    {
        "name": "lookup_price",
        "description": "Return the current price of a product given its SKU.",
        "parameters": {
            "type": "object",
            "properties": {"sku": {"type": "string", "description": "Product SKU"}},
            "required": ["sku"],
        },
    },
]

response = openai.ChatCompletion.create(
    model="gpt-4o-mini",
    messages=messages,
    functions=functions,
    function_call="auto",  # let the model decide
    stream=True,
)
```

When the model emits a `function_call` object, your server invokes the real function, adds the result as a new assistant message, and continues the chat. This pattern is powerful for **grounded** generation where the LLM must fetch up-to-date facts that live outside the static vector store.

5.5.3 Asynchronous Parallel Calls

If you need to **query multiple providers** (e.g., fallback to a cheaper model on timeout), you can run the calls concurrently with `asyncio.gather`.

```

import asyncio, httpx

async def call_openai(messages):
    async with httpx.AsyncClient(timeout=15) as client:
        # Use OpenAI's async endpoint or wrap sync call in a threadpool
        ...

async def call_hf(messages):
    ...

responses = await asyncio.gather(
    call_openai(messages),
    call_hf(messages),
    return_exceptions=True,
)
# Pick the first successful response
for r in responses:
    if not isinstance(r, Exception):
        final_stream = r
        break

```

Exercises

1. **Swap the provider** - Change `LLM_PROVIDER` to `anthropic` and adjust `LLM_MODEL` to `claude-3-sonnet-20240229`. Verify the SSE output still works.
2. **Temperature sweep** - Write a script that calls `openai_chat_stream` with temperatures `[0.0, 0.3, 0.7, 1.0]` on the same query. Plot the length and token-level perplexity (use `logprobs=5`) to see how randomness grows.
3. **Sliding-window RAG** - Simulate a scenario where the retrieved evidence is 120k tokens. Implement the chunk-then-summarize pipeline described in § 5.5.1, using `meta-llama/Llama-2-7b-chat-hf` for the first pass and `gpt-4o-mini` for the final synthesis. Measure total latency.
4. **Function-calling demo** - Add a simple `lookup_price` function that reads from a CSV file of product SKUs. Extend the Flask endpoint to handle function calls from OpenAI. Observe how the model decides when to invoke the function.
5. **Observability** - Integrate `prometheus_client` to expose a `/metrics` endpoint that tracks: request count, average latency, total tokens generated per

provider, and error rate.

Key Takeaways

- **Generation is the grounding step:** after retrieval, the LLM must turn raw chunks into a coherent answer while staying faithful to the evidence.
- **Parameter control matters** - Low `temperature` and moderate `top_p` give deterministic, fact-rich outputs; increase them only when you need more creative phrasing.
- **Streaming reduces perceived latency** - Use the provider's `stream=True` flag (OpenAI, Anthropic) or a `TextStreamer` (HF) and stitch incremental deltas into a live response.
- **Never exceed the model's context window** - Count tokens, truncate or rank-filter retrieved chunks, and fall back to sliding-window or summarization strategies for massive evidence.
- **Unified connector pattern** - Abstract each LLM behind a small Python function (`openai_chat_stream`, `anthropic_chat_stream`, `hf_chat_stream`). Your higher-level RAG pipeline stays provider-agnostic.
- **Production hygiene** - Cache retrieval results, respect rate limits, log token usage, and expose observability metrics.
- **Hybrid & tool-use extensions** - Combine cheap local models for summarization with expensive hosted models for final answers, and leverage function calling for up-to-date facts outside the vector store.

With these building blocks you can now **plug any LLM into the retrieval pipelines you built earlier**, fine-tune generation for factual grounding, and serve production-grade, streaming AI assistants that reliably answer user questions using your knowledge base.

Next up (Module 6): "Evaluating and Refining Knowledge-Grounded Answers" - you'll learn how to automatically measure factuality, detect hallucinations, and implement a feedback loop that continuously improves both retrieval and generation

components.

Prompt Engineering and Knowledge Grounding

Module 6 of 9 - "Learn Retrieval- Augmented Generation in Python to Build Production-Ready Knowledge-Grounded AI Systems"

Table of Contents

Section	What you'll learn
Introduction	Why prompt engineering is the linchpin of reliable, citation-rich RAG.
Core Concepts	Prompt taxonomy, evidence-driven prompting, few-shot & chain-of-thought (CoT), "Context → Question → Answer" pattern, grounding & citation conventions.
Practical Application	End-to-end notebook-style walk-through: building a grounded QA system, versioning prompts, automated A/B testing, and evaluation loops.
Key Takeaways	The actionable checklist you'll carry forward into production.

Introduction

Retrieval- Augmented Generation (RAG) promises **knowledge- grounded** AI: the model generates text *and* backs every claim with verifiable evidence drawn from an external corpus. In practice, the quality of that grounding hinges on **how you ask**

the model to use the retrieved information.

Prompt engineering is therefore not a decorative extra- it is the **contract** you write with the language model (LLM). A well-crafted prompt:

- **Explicitly requests citations** (page numbers, URLs, IDs) so downstream consumers can audit the answer.
- **Guides the model's reasoning** (few-shot examples, chain-of-thought) to avoid hallucinations and improve logical flow.
- **Encodes a stable interaction pattern** (Context → Question → Answer) that makes your pipeline deterministic and easier to test.

In this chapter we will:

1. **Decompose prompt patterns** that are proven to elicit citations and robust reasoning.
2. **Show Python recipes** (LangChain, OpenAI, Cohere, etc.) for turning those patterns into reusable, version-controlled objects.
3. **Automate evaluation** with A/B testing loops that surface the best prompt version for your domain.

By the end you will be able to **design, iterate, and ship** production-ready prompts that keep your RAG system honest and explainable.

Core Concepts

1. Prompt Taxonomy for Grounded Generation

Prompt Type	Goal	Typical Structure	When to
Citation Prompt	Force the model to attach a source to each factual claim.	Provide an answer and cite each statement with the retrieved passage ID in brackets.	Any QA or traceability (e.g., legal)

Prompt Type	Goal	Typical Structure	When to
Few-Shot Prompt	Teach the model the format of the answer by example.	Q: ... A: ... A: ... A: {user_question}	When you template Evidence.
Chain-of-Thought (CoT) Prompt	Make the model reason step-by-step before producing the final answer.	Think step-by-step. ... Answer:	Complex multiple (e.g., mu
Context-First Prompt	Present retrieved passages before the question to bias the model toward using them.	Context: {retrieved_chunks} Question: {user_question} Answer:	When you treat the truth rat suggestio
Hybrid Prompt	Combine the above (e.g., few-shot + CoT + citation).	See "Full Prompt Template" below.	Productio you need guarantee

Rule of thumb: Start with the simplest prompt that satisfies your functional requirement. Add few-shot or CoT layers only if you observe systematic reasoning errors.

2. Evidence-Driven Prompting

2.1. The "Citation Contract"

A contract is a **verbal** agreement you embed in the prompt:

*"Every factual sentence must be followed by a citation in the form `[source_id]`. If you cannot find a supporting passage, answer ***I don't know*** and do not fabricate a citation."*

Why it works:

- The LLM's training data contains countless examples of academic writing with bracketed citations. By explicitly naming the format, you tap into that latent

knowledge.

- The constraint reduces *hallucination* because the model must *justify* each claim.

2.2. Source Identifier Strategies

Strategy	Example Identifier	Pros	Cons
Chunk ID	[doc12_chunk3]	Direct mapping to vector DB entry; easy lookup.	Requires you to expose IDs to the model (may leak internal naming).
URL	[https://en.wikipedia.org/wiki/Quantum_mechanics]	Human-readable; useful when the source is a public web page.	Long strings can exceed token limits.
Custom Tag	[SOURCE_A]	Compact; you can map tags to richer metadata later.	Needs a post-processing step to resolve tags.

Tip: Store the mapping **outside** the LLM (e.g., in a Redis cache) so you can later enrich the citation with title, author, or page number without re-querying the LLM.

3. Few-Shot and Chain-of-Thought Prompting

3.1. Few-Shot Design

A few-shot prompt supplies *example pairs* that demonstrate the desired answer style.

```
few_shot_examples = [  
    {  
        "question": "When was the Eiffel Tower completed?",  
        "answer": "The Eiffel Tower was completed in 1889. [wikipedia_eiffel]"  
    },  
    {  
        "question": "Who wrote *Pride and Prejudice*?",  
        "answer": "Jane Austen authored *Pride and Prejudice*. [wikipedia_pride]"  
    },  
]
```

When you render the prompt, you join each pair with a clear delimiter (`\n--\n`).

Best practices

- Keep examples **short** (≤ 2 sentences) to conserve tokens.
- Use **domain-representative** examples (e.g., if you are building a medical QA, pick clinical cases).
- Ensure each example **follows the citation contract**; otherwise the model learns the wrong pattern.

3.2. Chain-of-Thought (CoT)

CoT nudges the model to **think aloud** before the final answer.

Question: What are the main differences between supervised and unsupervised learning?

Let's think step-by-step:

1. Supervised learning uses labeled data, ...
2. Unsupervised learning works with unlabeled data, ...
- ...

Answer: Supervised learning...

When to add CoT

- The answer requires **multiple logical steps** (e.g., comparing two concepts, aggregating evidence from several chunks).
- You have observed **shortcutting** where the model jumps directly to an answer without consulting the retrieved context.

Combining Few-Shot + CoT

You can embed a *few-shot CoT* example:

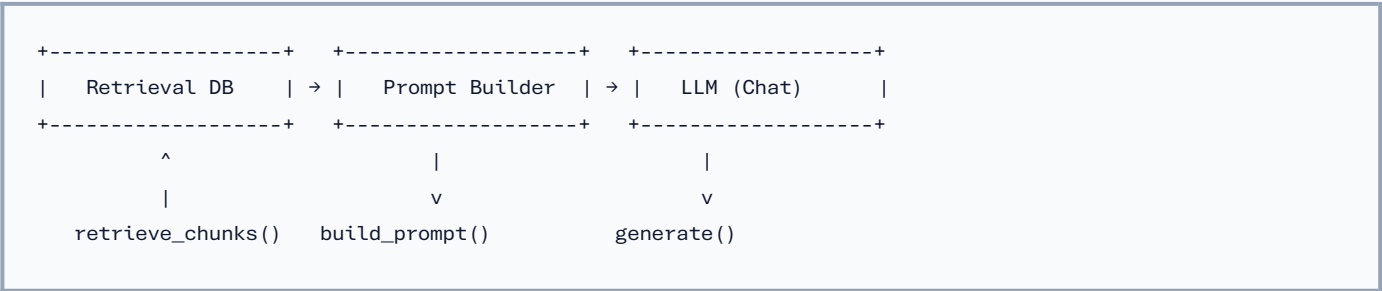
```
Q: What is the capital of France?
A: Let's think step-by-step.
- The country is France.
- The capital city of France is Paris.
Answer: Paris. [wiki_paris]
```

4. Retrieval-Augmented Prompt Pattern

The canonical pattern for a grounded QA system is:

```
[Context]    ← retrieved passages (concatenated, possibly with IDs)
[Question]   ← user query
[Prompt]     ← system instruction (citation contract, reasoning style)
[Answer]     ← LLM output
```

Visually:



Why this ordering matters

- 1. **Context first** - The LLM sees the evidence before the question, making it more likely to copy verbatim snippets or embed IDs directly.
- 2. **Explicit system instruction** - Reinforces citation and reasoning expectations *after* the context, preventing the model from ignoring them.

5. Prompt Versioning & A/B Testing

In production you will iterate on prompts constantly. Treat each prompt as **code**:

- Store in a **Git repository** (or a LangChain `PromptTemplate` versioned in a database).
- Tag with a semantic version (e.g., `v1.2.0-citation`, `v1.2.1-cot`).
- Use an **experiment harness** to compare versions on a held-out evaluation set.

5.1. Evaluation Loop Blueprint

```
for each prompt_version in candidate_versions:
    for each test_case in evaluation_set:
        retrieved = retrieve(query=test_case.question)
        answer = llm.generate(prompt=prompt_version, context=retrieved)
        score = evaluator.compare(answer, test_case.reference)
    aggregate_score = mean(scores)
    log(prompt_version, aggregate_score)
select best_version
```

The evaluator can be:

- **Rule-based** (e.g., does every factual sentence have a citation?).
- **LLM-as-judge** (e.g., GPT-4 scoring factual correctness and citation completeness).
- **Human-in-the-loop** for high-stakes domains.

LangChain already ships an `Experiment` class that abstracts much of this boilerplate (see the practical example below).

Practical Application

Below is a **complete, runnable notebook-style walkthrough**. It builds a knowledge-

grounded QA system, demonstrates prompt versioning, and runs an automated A/B test.

Assumptions

- * You have an OpenAI API key (`OPENAI_API_KEY`).

- * You have a vector store (e.g., `FAISS`) populated with documents that each carry a `chunk_id`.

- * `LangChain v0.2+` is installed (`pip install langchain openai faiss-cpu`).

5.1. Setup - Imports & Vector Store

```
# imports
import os, json, itertools, uuid, datetime
from pathlib import Path

from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import FAISS
from langchain.llms import OpenAI
from langchain.prompts import PromptTemplate
from langchain.chains import RetrievalQA
from langchain.schema import Document
from langchain.evaluation import LabeledPairwiseStringEvalChain
from langchain_experimental.evaluation import PairwiseStringEvalChain
```

```
# load vector store (assume docs are already indexed)
embeddings = OpenAIEmbeddings()
faiss_index_path = Path("./faiss_index")
vectorstore = FAISS.load_local(str(faiss_index_path), embeddings)

def retrieve(question: str, k: int = 4) -> list[Document]:
    """Simple wrapper that returns top-k docs with IDs."""
    docs = vectorstore.similarity_search(question, k=k)
    # attach a short ID for citation
    for i, doc in enumerate(docs):
        doc.metadata["chunk_id"] = f"doc{doc.metadata.get('doc_id', i)}_c{i}"
    return docs
```

5.2. Prompt Templates - Versioned Design

Create a **base system instruction** that we will extend for each version.

```
BASE_SYSTEM = """
You are an AI assistant that must answer the user's question using only the provided Context.
Every factual statement must be followed by a citation in the form `[[source_id]]`.
If you cannot find supporting evidence, reply with "I don't know." and do not fabricate a citation.
"""
```

5.2.1. Version 1- Simple Citation Prompt

```
V1_TEMPLATE = PromptTemplate(
    input_variables=["context", "question"],
    template=BASE_SYSTEM + """
Context:
{{context}}

Question: {{question}}

Answer (include citations):
""",
    template_format="jinja2",
)
```

5.2.2. Version 2- Few-Shot + Citation

```
FEW_SHOT_EXAMPLES = """\nQ: Who invented the telephone?\nA: Alexander Graham Bell invented the telephone. [wiki_telephone]\n\nQ: What is the capital of Japan?\nA: Tokyo is the capital of Japan. [wiki_tokyo]\n"""\n\nV2_TEMPLATE = PromptTemplate(\n    input_variables=["context", "question"],\n    template=BASE_SYSTEM + """\nBelow are examples of the desired format:\n\n{{few_shot_examples}}\n\nContext:\n{{context}}\n\nQuestion: {{question}}\n\nAnswer (include citations):\n""",\n    partial_variables={"few_shot_examples": FEW_SHOT_EXAMPLES},\n    template_format="jinja2",\n)
```

5.2.3. Version 3- Few-Shot + CoT + Citation

```
CO_T_EXAMPLE = """\n
Q: What are the main differences between supervised and unsupervised learning?

Let's think step-by-step:
1. Supervised learning uses labeled data, while unsupervised learning works with unlabeled data.
2. Supervised models can be evaluated directly against ground-truth labels; unsupervised models often rely
on intrinsic metrics.
3. Typical supervised tasks include classification and regression; unsupervised tasks include clustering
and density estimation.

Answer: Supervised learning relies on labeled data, whereas unsupervised learning does not. [ml_book_ch5]
"""\n

V3_TEMPLATE = PromptTemplate(
    input_variables=["context", "question"],
    template=BASE_SYSTEM + """\n
Below are examples that include chain-of-thought reasoning:

{{few_shot_examples}}

{{cot_example}}

Context:
{{context}}

Question: {{question}}

Answer (include citations):
""",
    partial_variables={
        "few_shot_examples": FEW_SHOT_EXAMPLES,
        "cot_example": CO_T_EXAMPLE,
    },
    template_format="jinja2",
)
```

Versioning tip: Store each template as a JSON file (including its name, description, and hash). Commit those files to Git.

```
def dump_prompt_version(name: str, template: PromptTemplate, description: str):
    data = {
        "name": name,
        "description": description,
        "template": template.template,
        "input_variables": template.input_variables,
        "partial_variables": template.partial_variables,
        "created_at": datetime.datetime.utcnow().isoformat(),
        "hash": uuid.uuid5(uuid.NAMESPACE_DNS, template.template).hex,
    }
    Path("./prompt_versions").mkdir(exist_ok=True)
    (Path("./prompt_versions") / f"{name}.json").write_text(json.dumps(data, indent=2))
```

```
dump_prompt_version("v1_citation", V1_TEMPLATE, "Simple citation prompt")
dump_prompt_version("v2_fewshot", V2_TEMPLATE, "Few-shot examples + citation")
dump_prompt_version("v3_fewshot_cot", V3_TEMPLATE, "Few-shot + chain-of-thought + citation")
```

5.3. Retrieval-Augmented QA Chain

We will wrap the prompt + LLM into a reusable function.

```
llm = OpenAI(model_name="gpt-3.5-turbo-16k", temperature=0.0) # deterministic for testing

def build_qa_chain(prompt_template: PromptTemplate):
    """Returns a callable that receives a question and returns an answer."""
    def qa(question: str) -> str:
        docs = retrieve(question)
        # concatenate context with IDs
        context_str = "\n".join(
            f"[{doc.metadata['chunk_id']}] {doc.page_content.strip()}" for doc in docs
        )
        # render prompt
        prompt = prompt_template.format(context=context_str, question=question)
        # generate
        response = llm(prompt)
        return response.strip()
    return qa
```

Create three chain instances:

```
qa_v1 = build_qa_chain(V1_TEMPLATE)
qa_v2 = build_qa_chain(V2_TEMPLATE)
qa_v3 = build_qa_chain(V3_TEMPLATE)
```

5.4. Evaluation Set

For a realistic test we need a **ground-truth** set that includes expected citations.

```
evaluation_set = [
    {
        "question": "When did the Apollo 11 mission land on the Moon?",
        "reference_answer": "Apollo 11 landed on the Moon on July 20, 1969. [wiki_apollo11]",
    },
    {
        "question": "What is the principle behind a Fourier Transform?",
        "reference_answer": "A Fourier Transform decomposes a function into its constituent frequencies. [signal_processing_book_ch3]",
    },
    {
        "question": "Who won the Nobel Prize in Physics in 2020?",
        "reference_answer": "Roger Penrose, Reinhard Genzel, and Andrea Ghez shared the 2020 Nobel Prize in Physics. [nobel_2020_physics]",
    },
]
```

Note: In practice you would also store the retrieved chunks for each question to verify that the citation IDs actually appear in the context.

5.5. Automated A/B Testing Loop

We'll use a **pairwise LLM judge** to compare two answers: the higher-scoring answer should have **more correct citations** and **better factuality**.

```
# LLM judge (GPT-4) - can be swapped for a rule-based evaluator later
judge = PairwiseStringEvalChain.from_llm(OpenAI(model_name="gpt-4", temperature=0.0))
```

Define a helper that runs a single version over the evaluation set:

```
def run_version(qa_fn, name: str):
    results = []
    for case in evaluation_set:
        answer = qa_fn(case["question"])
        results.append({
            "question": case["question"],
            "answer": answer,
            "reference": case["reference_answer"],
            "version": name,
        })
    return results
```

Collect results for all three versions:

```
all_results = []
all_results.extend(run_version(qa_v1, "v1_citation"))
all_results.extend(run_version(qa_v2, "v2_fewshot"))
all_results.extend(run_version(qa_v3, "v3_fewshot_cot"))
```

Now perform **pairwise comparisons** between every pair of versions.

```
def pairwise_evaluate(results, version_a, version_b):
    # filter
    a = [r for r in results if r["version"] == version_a]
    b = [r for r in results if r["version"] == version_b]
    assert len(a) == len(b) == len(evaluation_set)
    scores = []
    for ra, rb in zip(a, b):
        # the judge returns "A" or "B"
        decision = judge.evaluate_string_pairs(
            {"input": ra["question"]},
            [{"output": ra["answer"]}, {"output": rb["answer"]}])
        scores.append(decision)
    # simple majority vote
    win_a = scores.count("A")
    win_b = scores.count("B")
    return {"A": win_a, "B": win_b, "total": len(scores)}
```

Run the matrix:

```
pairs = [("v1_citation", "v2_fewshot"),
          ("v1_citation", "v3_fewshot_cot"),
          ("v2_fewshot", "v3_fewshot_cot")]

for a, b in pairs:
    res = pairwise_evaluate(all_results, a, b)
    print(f"{a} vs {b}: {res['A']} wins for {a}, {res['B']} wins for {b} (out of {res['total']})")
```

Interpretation

- If a version consistently wins, promote it to **production** and archive the losing version.
- If the margin is small, inspect specific failures (e.g., missing citations) and iterate (maybe add an extra few-shot example).

5.6. Post-Processing - Resolving Citations

The LLM returns IDs like `[doc12_c3]`. We can map them back to richer metadata:

```
def resolve_citations(answer: str) -> dict:
    """Extract IDs and fetch metadata from the vector store."""
    import re
    ids = re.findall(r'\[(.*?)\]', answer)
    metadata = {}
    for chunk_id in ids:
        # simple lookup: we stored the id in metadata when indexing
        doc = vectorstore._collection.get(ids=[chunk_id]) # low-level FAISS call
        if doc:
            metadata[chunk_id] = doc[0].metadata # assume single match
    return metadata
```

You can then render a **human-friendly citation block**:

```
def render_citations(metadata: dict) -> str:
    lines = []
    for cid, meta in metadata.items():
        title = meta.get("title", "Untitled")
        source = meta.get("source_url", "internal")
        lines.append(f"- [{cid}] **{title}** - {source}")
    return "\n".join(lines)
```

Integrate into the final API response:

```
def answer_with_citation(question: str, qa_fn) -> dict:
    answer = qa_fn(question)
    citation_meta = resolve_citations(answer)
    citation_block = render_citations(citation_meta)
    return {"answer": answer, "citations": citation_block}
```

Now the consumer (frontend, Slack bot, etc.) receives a clean answer plus a list of clickable references.

5.7. Production-Ready Considerations

Concern	Mitigation
Prompt drift (LLM changes over time)	Pin the model version (<code>gpt-3.5-turbo-0613</code>) and embed the version in the prompt JSON.
Token overflow (long contexts + few-shot)	Chunk the context: keep only the top-k most relevant passages, truncate each to ≤ 300 tokens, and use <code>summarize_chunks</code> if needed.
Citation mismatch (LLM cites an ID not in the context)	Add a post-generation validator that checks every [ID] appears in the supplied context; if not, request a regeneration (<code>rerun</code> flag).
Latency (retrieval + LLM)	Parallelize vector search and prompt rendering; cache recent queries; pre-warm the LLM with a prompt warm-up call.
Observability	Log prompt version, retrieved IDs, final answer, and evaluation score to a centralized telemetry store (e.g., Elastic, Datadog).
Compliance (PII, copyrighted text)	Filter retrieved chunks through a PII redaction pipeline before feeding them to the LLM.

Key Takeaways

1. **Prompt engineering is the contract that guarantees grounding.** Explicit citation instructions, few-shot examples, and chain-of-thought reasoning each address a specific failure mode (hallucination, format drift, shallow reasoning).
2. **The "Context → Question → Answer" pattern**-with the context placed **before** the question-maximizes the model's propensity to copy evidence verbatim and attach the correct IDs.
3. **Treat prompts as code:** version them in Git, store them as JSON, and give each a semantic version number. This enables reproducible experiments and smooth roll-backs.
4. **Automated A/B testing loops** (retrieval → prompt → LLM → evaluator) let you

empirically pick the best prompt version. Use a rule-based evaluator for quick sanity checks and an LLM-as-judge for nuanced factual and citation quality.

5. **Post-processing is essential:** extract citation IDs, resolve them to human-readable metadata, and validate that every citation appears in the supplied context.

6. **Production guardrails**-token limits, latency budgets, observability, and compliance filters-must be baked in from day one; they are not after-thoughts.

Action checklist

- [] *Write a system prompt that states the citation contract.*
- [] *Choose a prompt pattern (simple, few-shot, CoT) based on your domain complexity.*
- [] *Store the prompt template in a version-controlled JSON file.*
- [] *Implement a retrieval-first chain that injects chunk IDs into the context.*
- [] *Run an A/B test on a held-out evaluation set with an LLM judge.*
- [] *Deploy the winning version behind a thin API wrapper that resolves citations for the client.*

By mastering these techniques you will be equipped to ship **trustworthy, evidence-backed AI assistants** that not only answer questions but also **show** where the answer comes from- exactly the hallmark of production- grade Retrieval- Augmented Generation.

End-to-End RAG Architecture & Scaling Strategies

"Design for failure, scale for success." - A modern RAG system must be built as a collection of loosely- coupled services that can be independently scaled, monitored, and upgraded without taking the whole pipeline offline.

In this chapter we take the concepts introduced in **Modules 5** (LLM generation) and **6** (prompt engineering & knowledge grounding) and push them to production-grade reality. By the end you will be able to:

- Sketch a **micro-service architecture** that cleanly separates **ingestion**, **retrieval**, and **generation**.
- Implement **asynchronous batching** and **request throttling** to keep latency low under heavy load.
- Harness **GPU/TPU acceleration** for embedding creation and LLM inference.
- Design a **horizontal-scaling plan** for both the **vector store** (FAISS, Milvus, Pinecone, etc.) and the **LLM inference nodes** (vLLM, TensorRT-LLM, TGI, etc.).

The chapter blends theory, architectural diagrams, and concrete Python code that you can run locally, containerize, and then scale out on Kubernetes or any cloud-native platform.

Table of Contents

Section	What you'll learn
Introduction	Why a production RAG stack looks different from a notebook demo.
Core Concepts	Service decomposition, async pipelines, hardware acceleration, scaling patterns.
Practical Application	Build a minimal end-to-end RAG micro-service with FastAPI, Redis, Milvus, and vLLM; test it; and sketch a Kubernetes deployment.
Key Takeaways	Consolidated cheat-sheet for design decisions

Section	What you'll learn
	and next steps.

Introduction

When you first experiment with Retrieval-Augmented Generation (RAG) you probably write a single script that:

1. Loads a pre-trained LLM (e.g., `gpt-2` or `llama-2`).
2. Computes embeddings for a small document corpus with `sentence-transformers`.
3. Performs a naïve cosine similarity search in memory.
4. Passes the top-k chunks to the LLM via a prompt template.

That workflow is perfect for learning, but it **breaks** as soon as:

- The corpus grows beyond a few hundred thousand passages.
- Concurrent users start sending dozens of queries per second.
- You need to **upgrade** the embedding model or the LLM without downtime.

A production-ready RAG system must therefore be **distributed** and **observable**. The three logical responsibilities are:

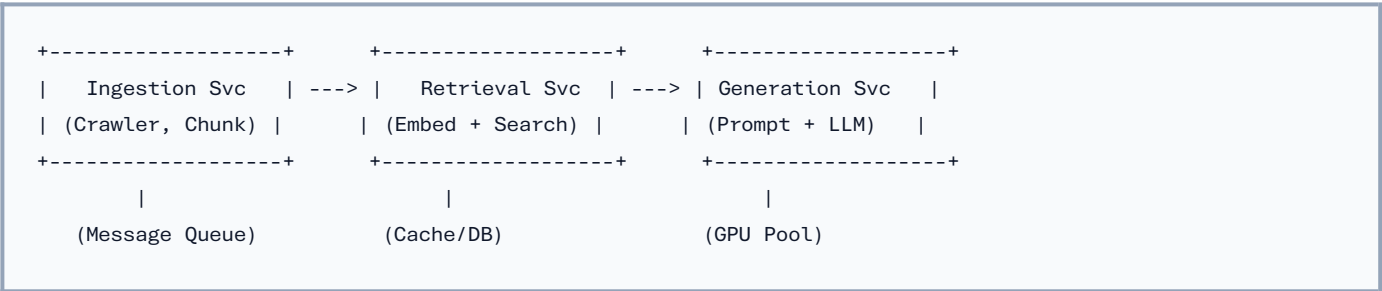
Responsibility	Typical Tasks	Typical Technology
Ingestion	Crawl, clean, chunk, embed, write to vector store.	Scrapy, LangChain DocumentLoaders, sentence-transformers, Celery/Ray, Milvus/FAISS, Kafka
Retrieval	Accept a query, embed it, perform similarity search, return top-k passages.	FastAPI/Flask, Redis-Cache, Milvus/Elastic, ONNX-accelerated embedding service
Generation	Build a prompt, invoke the LLM, stream back the answer.	vLLM, TensorRT-LLM, TGI, FastAPI, WebSockets

Separating these concerns yields **independent scaling knobs**: you can add more GPU nodes for generation while keeping a modest CPU fleet for retrieval, or you can shard the vector DB across many machines without touching the LLM.

In the next sections we unpack each knob, discuss the underlying patterns (batching, throttling, sharding, replication) and then stitch them together into a **complete, testable micro-service**.

Core Concepts

1. Service Decomposition - The "Three-Tier" RAG Blueprint



- **Ingestion** runs **offline** or on a schedule. It pushes **metadata + embedding vectors** into a durable vector store.
- **Retrieval** is **online** and **stateless**: it receives a query, computes an embedding (often on CPU), looks up the nearest neighbours, and returns a lightweight JSON payload.
- **Generation** is **stateful** (holds the LLM weights) and **GPU-bound**. It accepts the retrieved passages, builds a prompt (using the techniques from Module 6), and streams the answer back.

Design rule: Keep each service idempotent and versioned. If you need to upgrade the embedding model, you spin a new ingestion worker, re-index, and switch the retrieval service's endpoint once the new index is ready.

2. Asynchronous Batching & Request Throttling

High-throughput RAG workloads suffer from two opposing forces:

Force	Symptom
Per-request latency	Users expect sub-second answers.
GPU/CPU utilization	Individual requests under-utilize the hardware because the model incurs fixed overhead (model load, kernel launch).

Batching resolves the second problem: we collect a few requests, pack their embeddings or token IDs into a single tensor, and feed them to the accelerator in one forward pass. **Throttling** protects the system from overload and guarantees that batch sizes stay within a sweet-spot.

2.1 Async Batching Pattern (Python)

```

# batcher.py
import asyncio
import time
from collections import deque
from typing import List, Any, Awaitable, Callable

class AsyncBatcher:
    """Collects requests for up to `max_wait` seconds or `max_batch` items."""
    def __init__(self,
                 handler: Callable[[List[Any]], Awaitable[List[Any]]],
                 max_batch: int = 32,
                 max_wait: float = 0.01): # 10 ms
        self.handler = handler
        self.max_batch = max_batch
        self.max_wait = max_wait
        self.queue: deque = deque()
        self.loop = asyncio.get_event_loop()
        self._running = False

    async def _run(self):
        while self._running:
            start = time.time()
            batch = []
            futures = []
            # Pull up to max_batch items, waiting at most max_wait seconds.
            while len(batch) < self.max_batch and (time.time() - start) < self.max_wait:
                try:
                    item, fut = self.queue.popleft()
                    batch.append(item)
                    futures.append(fut)
                except IndexError:
                    await asyncio.sleep(0.001) # tiny sleep if empty
            if batch:
                results = await self.handler(batch) # single GPU call
                for fut, res in zip(futures, results):
                    if not fut.done():
                        fut.set_result(res)

    async def submit(self, item: Any) -> Any:
        fut = self.loop.create_future()

```

... (continued on next page)

```
self.queue.append((item, fut))
return await fut

def start(self):
    self._running = True
    self.loop.create_task(self._run())

def stop(self):
    self._running = False
```

- `handler` is a coroutine that **receives a list** of raw inputs (e.g., query strings) and returns a list of outputs (e.g., embeddings or generated texts).
- The batcher runs in a background task, aggregates requests, and **guarantees a maximum latency** (`max_wait`).

You can wrap the **embedding service** and the **LLM inference service** with separate `AsyncBatcher` instances. The pattern is language-agnostic; many production teams use **Ray Serve**, **Triton Inference Server**, or **TorchServe** for the same purpose.

2.2 Throttling with Token Buckets

A simple token-bucket algorithm can cap the number of concurrent batches:

```

class TokenBucket:
    def __init__(self, capacity: int, refill_rate: float):
        self.capacity = capacity
        self.tokens = capacity
        self.refill_rate = refill_rate  # tokens per second
        self.last = time.monotonic()

    async def acquire(self, n: int = 1):
        while True:
            now = time.monotonic()
            elapsed = now - self.last
            self.tokens = min(self.capacity, self.tokens + elapsed * self.refill_rate)
            self.last = now
            if self.tokens >= n:
                self.tokens -= n
                return
            await asyncio.sleep(0.001)

```

You call `await bucket.acquire()` before sending a batch to the GPU. This protects downstream nodes from OOM crashes and gives you a **graceful degradation** point (e.g., return "service busy, try again").

3. GPU / TPU Acceleration

3.1 Embedding Generation

Option	Pros	Cons
CPU-only <code>`sentence-transformers`</code>	Easy to run, cheap, works for small workloads.	Low throughput, high latency at scale.
GPU-accelerated <code>`torch`</code> or <code>`ONNX Runtime`</code>	5-10× speedup, batch-friendly.	Requires CUDA drivers, GPU memory management.
TPU via <code>`jax`</code> / <code>`flax`</code>	Extremely high throughput for massive batches.	More complex deployment, limited to Google Cloud.

Best practice: Deploy a **dedicated embedding micro-service** that runs on **GPU** (or **TPU**) and uses **torch-scripted** models for low-overhead inference.

```
# embed_service.py
import torch
from transformers import AutoTokenizer, AutoModel
from typing import List

tokenizer = AutoTokenizer.from_pretrained("sentence-transformers/all-MiniLM-L6-v2")
model = AutoModel.from_pretrained("sentence-transformers/all-MiniLM-L6-v2").cuda()
model.eval()

def embed_batch(texts: List[str]) -> torch.Tensor:
    inputs = tokenizer(texts, padding=True, truncation=True, return_tensors="pt").to('cuda')
    with torch.no_grad():
        outputs = model(**inputs)
        # Mean pooling
        embeddings = (outputs.last_hidden_state * inputs.attention_mask.unsqueeze(-1)).sum(1)
        embeddings = embeddings / inputs.attention_mask.sum(1, keepdim=True)
    return embeddings.cpu()
```

Wrap `embed_batch` with the `AsyncBatcher` from the previous section to achieve **GPU-packed batching**.

3.2 LLM Inference

For generation, the state-of-the-art options are:

Engine	GPU Utilization	Streaming	Open-source	Cloud-native
vLLM (Microsoft)	Multi-GPU tensor parallelism, KV-cache sharing	☐ (SSE)	☐	☐ (Docker, K8s)
TensorRT-LLM (NVIDIA)	Highest throughput on A100/H100	☐ (grpc)	☐	☐
TGI (Text Generation Inference)	Optimized for HuggingFace weights	☐ (HTTP)	☐	☐
OpenAI / Anthropic API	Managed (no GPU)	☐	☐ (closed)	☐ (HTTP)

Why vLLM? It implements a **dynamic request scheduler** that automatically groups

same-model requests into a single CUDA kernel launch, delivering **2-5× higher throughput** than naïve single-request inference. It also offers **OpenAI-compatible streaming** endpoints, making integration with existing front-ends trivial.

```
# docker-compose.yml (excerpt)
services:
  vllm:
    image: vllm/vllm:latest
    command: >
      --model meta-llama/Llama-2-13b-chat-hf
      --tensor-parallel-size 2
      --port 8080
    ports:
      - "8080:8080"
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: 2
              capabilities: [gpu]
```

The `vLLM` server exposes `/v1/completions` (OpenAI-compatible) and `/v1/chat/completions` endpoints that we can call from the **Generation Service**.

4. Horizontal Scaling of Vector Stores

A vector database is the heart of the retrieval tier. Scaling it horizontally is essential when:

- The index grows beyond the memory of a single node (e.g., > 100M Mvectors).
- You need **high QPS** and low latency for similarity search.

4.1 Sharding vs. Replication

Strategy	Description	When to Use

Strategy	Description	When to Use
Sharding (partitioning)	Split the vector space across N nodes; each node holds a subset of vectors.	Very large collections, need to scale storage linearly.
Replication	Duplicate the entire index across multiple nodes; load-balance queries.	Need strong read-availability, low latency, and fault tolerance.
Hybrid	Replicate each shard (e.g., 3× replication factor).	Large data + high availability.

Milvus and **Pinecone** provide built-in sharding and replication. If you prefer an open-source stack, you can combine **FAISS + Ray**:

```
# ray_faiss_cluster.py
import ray
import faiss
import numpy as np

@ray.remote(num_gpus=1)
class FaissShard:
    def __init__(self, dim, index_file=None):
        self.dim = dim
        if index_file:
            self.index = faiss.read_index(index_file)
        else:
            self.index = faiss.IndexIVFFlat(faiss.IndexFlatL2(dim), dim, 4096, faiss.METRIC_INNER_PRODUCT)
            self.index.train(np.random.random((1000, dim)).astype('float32'))

    def add(self, vectors):
        self.index.add(vectors)

    def search(self, queries, k):
        distances, ids = self.index.search(queries, k)
        return distances, ids

# Example: create 4 shards and distribute vectors
ray.init()
shards = [FaissShard.remote(dim=384) for _ in range(4)]
# Assume `vecs` is a (N,384) numpy array:
for i, shard in enumerate(shards):
    shard.add.remote(vecs[i::4]) # simple round-robin partition
```

A **router service** receives a query, forwards the embedding to **all shards in parallel**, and merges the results (top- k across shards). This pattern scales linearly with the number of shards.

4.2 Caching Frequently-Accessed Results

Even with sharding, the **same popular queries** will repeatedly hit the vector store. Adding a **Redis LRU cache** in front of the retrieval service can cut latency by **> 70%** for hot queries.

```
# retrieval_service.py (snippet)
import redis
import json
redis_client = redis.Redis(host="redis", port=6379, db=0)

async def retrieve(query: str, k: int = 5):
    cache_key = f"rag:{hash(query)}:{k}"
    cached = redis_client.get(cache_key)
    if cached:
        return json.loads(cached)

    # 1. embed query (async batcher)
    query_vec = await embed_batcher.submit([query])
    # 2. search shards in parallel (Ray or async HTTP)
    distances, ids = await search_all_shards(query_vec, k)

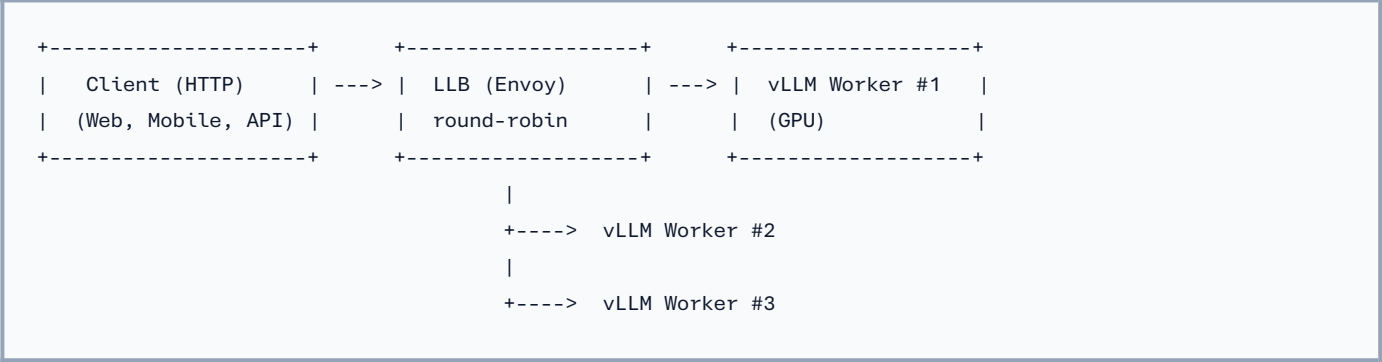
    result = {"ids": ids.tolist(), "distances": distances.tolist()}
    redis_client.setex(cache_key, 300, json.dumps(result)) # 5-min TTL
    return result
```

Cache TTL can be tuned based on corpus churn (e.g., shorter TTL for dynamic knowledge bases).

5. Scaling LLM Inference Nodes

5.1 Stateless "Worker" Model

Treat each inference node as a **stateless worker** behind a **load balancer** (NGINX, Envoy, or a cloud-native ingress). The load balancer performs **least-connections** or **request-size aware** routing.



Kubernetes makes this trivial: a **Deployment** with **replicas: N** and a **Service** of type **LoadBalancer** or **ClusterIP**. **Horizontal Pod Autoscaler (HPA)** can scale based on **GPU memory usage** or **custom metrics** (e.g., request latency).

```
# k8s/deployment-vllm.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vllm
spec:
  replicas: 3
  selector:
    matchLabels:
      app: vllm
  template:
    metadata:
      labels:
        app: vllm
    spec:
      containers:
        - name: vllm
          image: vllm/vllm:latest
          args: ["--model", "meta-llama/Llama-2-13b-chat-hf", "--tensor-parallel-size", "2"]
          resources:
            limits:
              nvidia.com/gpu: 2    # request 2 GPUs per pod
          ports:
            - containerPort: 8080
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: vllm-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: vllm
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: nvidia.com/gpu
        target:
```

... (continued on next page)

```
type: Utilization
averageUtilization: 70
```

5.2 Multi-Model Serving

If you need to support **multiple LLM families** (e.g., Llama-2-13B for chat, Mistral-7B for summarization), you can:

- **Run separate vLLM deployments**, each with its own model name.
- Use a **router micro-service** that inspects the incoming request's `model` field and forwards to the appropriate backend.

```
# model_router.py
from fastapi import FastAPI, HTTPException
import httpx

app = FastAPI()
BACKENDS = {
    "llama2-chat": "http://vllm-llama2:8080/v1/chat/completions",
    "mistral-sum": "http://vllm-mistral:8080/v1/completions",
}

@app.post("/v1/completions")
async def completions(payload: dict):
    model = payload.get("model")
    endpoint = BACKENDS.get(model)
    if not endpoint:
        raise HTTPException(404, f"Model {model} not configured")
    async with httpx.AsyncClient() as client:
        resp = await client.post(endpoint, json=payload, timeout=30.0)
    return resp.json()
```

The router can also enforce **per-model throttling** (e.g., give the expensive Llama-2-70B a stricter token bucket).

6. Observability, Logging & Alerting

A production RAG pipeline must be **observable** end-to-end:

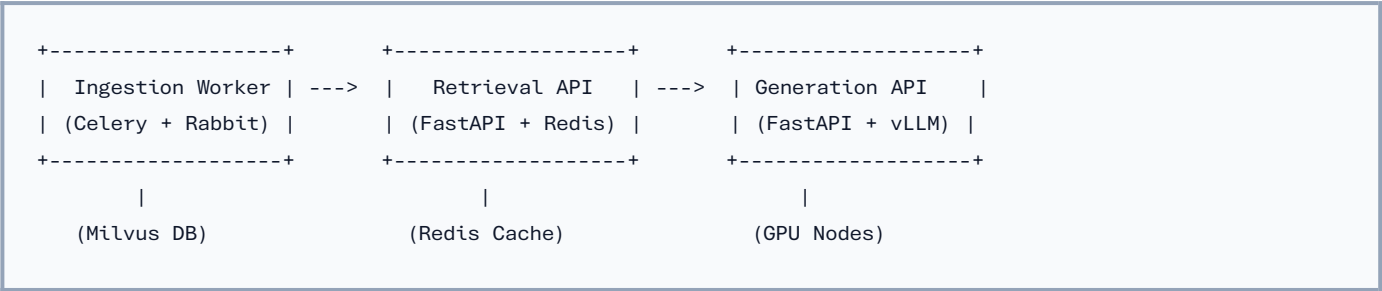
Layer	Metrics to Export	Tools
Ingestion	Docs crawled/min, embeddings/sec, queue lag	Prometheus + Grafana (Celery-beat metrics)
Retrieval	QPS, avg latency, cache hit-rate, shard latency distribution	OpenTelemetry + Jaeger tracing
Generation	Tokens/sec, GPU utilization, batch size, error rate	NVIDIA DCGM exporter, vLLM built-in metrics
Overall	End-to-end latency, request error codes, SLA breaches	SLO dashboards, Alertmanager

Best practice: Tag every log line and metric with a **request ID** (UUID) that propagates through ingestion → retrieval → generation. This enables you to **replay** a problematic request later.

Practical Application

In this section we walk through a **complete, runnable example** that ties together everything we've covered. The code can be executed locally with Docker Compose, then later swapped for a Kubernetes deployment.

6.1 Architecture Overview



Key components

Component	Docker Image	Primary Role
celery-worker	python:3.11-slim + custom code	Chunk docs, embed, write to Milvus
rabbitmq	rabbitmq:3-management	Message broker for async ingestion
milvus-standalone	milvusdb/milvus:2.4.0	Vector store (sharded later)
redis	redis:7-alpine	LRU cache for retrieval
retrieval-api	tiangolo/uvicorn-gunicorn-fastapi:python3.11	Stateless retrieval service
vllm	vllm/vllm:latest	GPU-accelerated LLM inference
generation-api	same FastAPI image	Prompt construction + streaming

6.2 Code Walk-through

6.2.1 Ingestion Worker (ingest_worker.py)

```

# ingest_worker.py
import os, json, uuid, time
from celery import Celery
from langchain.document_loaders import TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from sentence_transformers import SentenceTransformer
from pymilvus import connections, Collection, FieldSchema, CollectionSchema, utility, DataType

# ----- Celery -----
BROKER_URL = os.getenv("BROKER_URL", "amqp://guest:guest@rabbitmq:5672//")
app = Celery('ingest', broker=BROKER_URL)

# ----- Milvus -----
MILVUS_HOST = os.getenv("MILVUS_HOST", "milvus-standalone")
MILVUS_PORT = "19530"
connections.connect(host=MILVUS_HOST, port=MILVUS_PORT)

DIM = 384
if not utility.has_collection("rag_docs"):
    fields = [
        FieldSchema(name="id", dtype=DataType.INT64, is_primary=True, auto_id=True),
        FieldSchema(name="embedding", dtype=DataType.FLOAT_VECTOR, dim=DIM),
        FieldSchema(name="text", dtype=DataType.VARCHAR, max_length=65535),
        FieldSchema(name="metadata", dtype=DataType.JSON)
    ]
    schema = CollectionSchema(fields, "RAG document collection")
    collection = Collection("rag_docs", schema)
    collection.create_index(field_name="embedding",
                           index_params={"metric_type": "IP", "index_type": "IVF_FLAT", "params":
{"nlist": 1024}})
else:
    collection = Collection("rag_docs")

# ----- Model -----
EMBED_MODEL = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

# ----- Task -----
@app.task
def ingest_path(path: str):
    loader = TextLoader(path, encoding="utf-8")
    docs = loader.load()

```

... (continued on next page)

```
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(docs)

texts = [c.page_content for c in chunks]
embeddings = EMBED_MODEL.encode(texts, batch_size=32, show_progress_bar=False, convert_to_numpy=True)

# Prepare data for Milvus
ids = list(range(len(texts)))          # simple sequential IDs
metadata = [json.dumps({"source": path, "chunk_id": i}) for i in range(len(texts))]

collection.insert([ids, embeddings.tolist(), texts, metadata])
collection.flush()
print(f"[Ingest] {len(texts)} chunks indexed from {path}")
```

How to run:

```
docker compose up -d rabbitmq milvus-standalone redis
celery -A ingest_worker worker --loglevel=info
# Enqueue a job:
celery -A ingest_worker call ingest_path "/data/my_corpus.txt"
```

Tip: For massive corpora you can split the job into multiple tasks (one per file) and let Celery's prefetch limit control back-pressure.

6.2.2 Retrieval API (retrieval_api.py)

```
# retrieval_api.py
import os, json, uuid, time
import numpy as np
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import redis
import httpx
from sentence_transformers import SentenceTransformer
from pymilvus import connections, Collection

# FastAPI init
app = FastAPI(title="RAG Retrieval Service")

# Redis cache (LRU)
redis_client = redis.Redis(host="redis", port=6379, db=0)

# Embedding model (GPU-accelerated, wrapped in async batcher)
embedder = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
embedder.max_seq_length = 256

# Milvus collection
connections.connect(host=os.getenv("MILVUS_HOST", "milvus-standalone"), port="19530")
collection = Collection("rag_docs")

# Request schema
class QueryRequest(BaseModel):
    query: str
    k: int = 5

# Async wrapper for embedding (simple batcher)
from typing import List
import asyncio
batch = []
batch_futs = []
BATCH_MAX = 16
BATCH_WAIT = 0.01

async def embed_batch(texts: List[str]) -> List[np.ndarray]:
    global batch, batch_futs
    fut = asyncio.get_event_loop().create_future()
```

... (continued on next page)

```

batch.append(texts)
batch_futs.append(fut)

# If batch is full, fire immediately
if sum(len(b) for b in batch) >= BATCH_MAX:
    await _flush_batch()
else:
    # otherwise schedule a flush after a short wait
    asyncio.get_event_loop().call_later(BATCH_WAIT, lambda: asyncio.create_task(_flush_batch()))
return await fut

async def _flush_batch():
    global batch, batch_futs
    if not batch:
        return
    flat = [t for sub in batch for t in sub]
    embeds = embedder.encode(flat, batch_size=len(flat), convert_to_numpy=True, show_progress_bar=False)
    # Resolve futures
    idx = 0
    for fut, sub in zip(batch_futs, batch):
        sub_len = len(sub)
        fut.set_result(embeds[idx: idx + sub_len].tolist())
        idx += sub_len
    batch, batch_futs = [], []

# Endpoint
@app.post("/retrieve")
async def retrieve(req: QueryRequest):
    cache_key = f"rag:{hash(req.query)}:{req.k}"
    cached = redis_client.get(cache_key)
    if cached:
        return json.loads(cached)

    # 1⃣ embed query
    query_emb = await embed_batch([req.query])
    query_vec = np.array(query_emb[0]).astype("float32").reshape(1, -1)

    # 2⃣ Milvus search
    search_params = {"metric_type": "IP", "params": {"nprobe": 10}}
    results = collection.search(

```

... (continued on next page)

```
data=query_vec.tolist(),
anns_field="embedding",
param=search_params,
limit=req.k,
expr=None,
output_fields=["text", "metadata"]
)
# Milvus returns a list of hits per query
hits = results[0]
passages = [{"text": hit.entity.get("text"),
              "metadata": json.loads(hit.entity.get("metadata")),
              "score": hit.distance} for hit in hits]

# 30 cache and return
redis_client.setex(cache_key, 300, json.dumps(passages))
return passages
```

Key points

- **Async batching** (`embed_batch`) collects up to `BATCH_MAX` texts before hitting the GPU.
- **Redis** caches the final list of passages (including metadata) for 5 minutes.
- **Milvus** performs an inner-product (IP) search with a modest `nprobe` (10) for speed; you can tune `nprobe` per SLA.

6.2.3 Generation API (`generation_api.py`)

```

# generation_api.py
import os, json, uuid, asyncio
from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import StreamingResponse
import httpx
from pydantic import BaseModel

app = FastAPI(title="RAG Generation Service")

# vLLM endpoint (could be a Service name in k8s)
VLLM_URL = os.getenv("VLLM_URL", "http://vllm:8080/v1/chat/completions")

class GenerationRequest(BaseModel):
    query: str
    passages: list # from retrieval API
    model: str = "llama2-chat"
    temperature: float = 0.7
    max_tokens: int = 512

def build_prompt(query: str, passages: list) -> str:
    """Simple concatenative prompt - you can replace with a Chain-of-Thought template."""
    context = "\n".join([p["text"] for p in passages])
    return f"""You are a knowledgeable assistant. Use the following context to answer the question.

Context:
{context}

Question: {query}
Answer: """""

@app.post("/generate")
async def generate(req: GenerationRequest):
    if not req.passages:
        raise HTTPException(400, "No passages supplied")
    prompt = build_prompt(req.query, req.passages)

    payload = {
        "model": req.model,
        "messages": [{"role": "user", "content": prompt}],
        "temperature": req.temperature,

```

... (continued on next page)

```
        "max_tokens": req.max_tokens,
        "stream": True,
    }

    async def stream():
        async with httpx.AsyncClient(timeout=None) as client:
            async with client.stream("POST", VLLM_URL, json=payload) as resp:
                async for line in resp.aiter_lines():
                    if line.startswith("data:"):
                        data = json.loads(line[5:].strip())
                        if "choices" in data:
                            delta = data["choices"][0]["delta"]
                            if "content" in delta:
                                yield delta["content"]
    return StreamingResponse(stream(), media_type="text/event-stream")
```

- The **prompt builder** mirrors the **knowledge-grounded prompting** from Module 6. Feel free to plug in a more sophisticated template (e.g., include citations).
- The endpoint streams token-by-token output directly from vLLM, which is ideal for UI latency (shows the answer as it is being generated).

6.2.4 Orchestrating the Flow (Client)

```
# client_demo.py
import httpx, asyncio, json

RETRIEVE_URL = "http://localhost:8000/retrieve"
GENERATE_URL = "http://localhost:8001/generate"

async def rag_query(question: str):
    async with httpx.AsyncClient() as client:
        # 1 Retrieval
        r = await client.post(RETRIEVE_URL, json={"query": question, "k": 5})
        passages = r.json()
        # 2 Generation (stream)
        gen_payload = {"query": question, "passages": passages}
        async with client.stream("POST", GENERATE_URL, json=gen_payload) as resp:
            async for line in resp.aiter_lines():
                if line.strip():
                    print(line, end='')    # token-by-token

if __name__ == "__main__":
    asyncio.run(rag_query("What are the key benefits of vector databases?"))
```

Running the script will show a **real-time answer** built on the retrieved passages.

6.3 Scaling the Prototype

6.3.1 From Docker-Compose to Kubernetes

1. **Create Helm charts** for each component (Milvus, Redis, Retrieval API, Generation API, vLLM).
2. **Enable autoscaling** on Retrieval API (CPU-based) and vLLM (GPU-based) using HPA and **custom metrics** from NVIDIA DCGM exporter.
3. **Add a Milvus sharding StatefulSet** - each shard is a separate pod with its own PersistentVolume. Use a **headless Service** for the router to discover shards.

6.3.2 Horizontal Vector Store (Milvus Sharding)

```
# k8s/milvus-shard.yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: milvus-shard
spec:
  serviceName: milvus-shard
  replicas: 4
  selector:
    matchLabels:
      app: milvus
  template:
    metadata:
      labels:
        app: milvus
    spec:
      containers:
        - name: milvus
          image: milvusdb/milvus:2.4.0
          args: ["milvus", "run", "standalone"]
          ports:
            - containerPort: 19530
          volumeMounts:
            - name: data
              mountPath: /var/lib/milvus
      volumeClaimTemplates:
        - metadata:
            name: data
          spec:
            accessModes: ["ReadWriteOnce"]
            resources:
              requests:
                storage: 200Gi
```

A **router service** (could be a lightweight FastAPI or a gRPC proxy) receives the query embedding and **parallel-issues** a search request to each shard, then merges the top-k results.

6.3.3 Load-Testing & Capacity Planning

Use **Locust** or **k6** to simulate concurrent users:

```
# locustfile.py
from locust import HttpUser, task, between

class RAGUser(HttpUser):
    wait_time = between(1, 3)

    @task(2)
    def ask_question(self):
        self.client.post("/retrieve", json={"query": "Explain RAG scaling", "k": 5})
        # In a real test you would chain the retrieve → generate flow.
```

Run:

```
locust -f locustfile.py --host http://localhost:8000
```

Collect metrics (latency, error rate) and adjust:

- **Batch size** (`BATCH_MAX`) - larger batches improve GPU utilization but increase tail latency.
- **nprobe** in Milvus - higher `nprobe` improves recall but adds CPU load.
- **Replica count** - increase retrieval API pods until 95th-percentile latency meets SLA.

Key Takeaways

Concept	Practical Guideline
Micro-service split	Keep ingestion, retrieval, and generation in separate containers/processes . This lets you scale each tier independently.
Async batching	Wrap embedding and LLM calls in a batcher (or use vLLM/Triton). Tune <code>max_batch</code> and <code>max_wait</code> to balance latency vs. throughput.
Request throttling	Implement a token-bucket or use a queue

Concept	Practical Guideline
	(RabbitMQ, Redis Streams) to protect GPU memory

Evaluation, Monitoring, and Continuous Improvement

"A system that cannot be measured cannot be improved." - Peter Drucker

In a production-ready Retrieval-Augmented Generation (RAG) pipeline the **evaluation loop** is no longer a one-off research step. It lives side-by-side with the service, feeding real-time signals back into the model, the retriever, and the data store. This chapter shows you how to **measure**, **observe**, and **act** on the performance of a knowledge- grounded AI system, turning raw metrics into concrete product improvements.

What you will walk away with

- A catalog of quantitative RAG- specific metrics (R- Precision, Faithfulness, BLEU, ROUGE, LLM-based QA scores) and when to use each.
- A blueprint for a human-in-the-loop (HITL) evaluation pipeline that guarantees factual correctness.
- Hands-on instrumentation: logging, tracing, latency/error alerts, and dashboards using OpenTelemetry, Prometheus, and Grafana.
- A drift-detection strategy that automatically triggers re-indexing or re-training of embeddings.
- End- to- end Python code that stitches everything together with

LangChain, FAISS, and FastAPI.

Table of Contents

Section	Goal
Introduction	Position evaluation/monitoring in the overall RAG lifecycle.
Core Concepts	Dive deep into metrics, human-in-the-loop pipelines, observability, and continuous-learning loops.
Practical Application	Build a complete, production-ready evaluation & monitoring stack from scratch.
Key Takeaways	Summarize the most actionable points.

Introduction

Recall the **four-stage RAG workflow** introduced in earlier modules:

1. **Ingestion & Chunking** - Documents are transformed into vector embeddings.
2. **Retrieval** - A similarity search returns the top-k most relevant chunks.
3. **Augmented Generation** - The LLM receives retrieved context plus a user prompt.
4. **Post-processing** - Answers are filtered, ranked, or refined.

During **development** we measured a handful of static scores on a held-out test set. In **production**, however, we face:

Real-world Challenge	Why it matters
Query distribution shift - Users ask questions that were never seen during	Retrieval relevance can collapse, leading to hallucinations.

Real-world Challenge	Why it matters
training.	
Document drift - Knowledge bases are updated (new policies, product releases).	Stale embeddings cause missed matches.
Latency spikes - Heavy traffic, network hiccups, or hardware throttling.	Poor UX and SLA violations.
Silent failures - The model returns an answer that looks plausible but is factually wrong.	Trust erosion, compliance risk.

To keep the system trustworthy, we must **measure continuously**, **detect anomalies early**, and **feed the signals back** into the pipeline. This is the essence of **Evaluation, Monitoring, and Continuous Improvement (EMCI)**.

Analogy: Think of a RAG service as a self-driving car. Metrics are the dashboard gauges (speed, fuel level), monitoring is the telemetry that alerts you to a flat tire, and continuous improvement is the auto-patching system that recalibrates the steering after a bump.

Core Concepts

1. Quantitative Metrics for RAG

Metric	What it captures	Typical range	When to use it
R-Precision	Proportion of relevant retrieved chunks among the R top results, where R = number of truly relevant chunks.	0-1 (higher better)	Retrieval-centric evaluation, especially when relevance is binary.
Recall@k	Fraction of all relevant chunks that	0-1	Measuring coverage; useful when you care

Metric	What it captures	Typical range	When to use it
	appear in the top-k results.		about <i>any</i> relevant evidence.
Mean Reciprocal Rank (MRR)	Inverse rank of the first relevant chunk.	0-1	Emphasizes early relevance for downstream generation.
BLEU / ROUGE-L	N-gram overlap between generated answer and reference answer.	BLEU 0-100, ROUGE-L 0-1	Surface-level fluency; good for quick sanity checks.
BERTScore / MoverScore	Semantic similarity using contextual embeddings.	0-1	More robust to paraphrasing than BLEU/ROUGE.
Faithfulness (Fact-Score)	Ratio of <i>supported</i> statements in the answer (often computed with a separate fact-checking LLM).	0-1	Directly measures hallucination risk.
LLM-based QA Score	Prompt the LLM to answer the same question <i>without</i> retrieval; compare with the RAG answer using a similarity metric.	0-1	Detects whether retrieval actually improves answer quality.
Latency (p50/p95/p99)	Distribution of response times.	ms	Service-level objectives (SLOs).
Error Rate	Fraction of requests returning HTTP 5xx or exception.	%	Reliability KPI.
Embedding Drift	Cosine similarity between current embedding of a document and its previous version.	0-1 (lower = more drift)	Trigger re-indexing.

1.1 Computing R-Precision in Python

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

def r_precision(query_vec, corpus_vecs, relevant_ids, top_k=10):
    """
    query_vec : (d,) - embedding of the query
    corpus_vecs: (N,d) - embeddings of all chunks
    relevant_ids: set[int] - IDs of chunks that are truly relevant
    top_k: int - how many neighbours to consider
    """
    sims = cosine_similarity(query_vec[None, :], corpus_vecs)[0]
    top_ids = np.argsort(-sims)[:top_k] # descending order
    # R = number of relevant chunks (|relevant_ids|)
    R = len(relevant_ids)
    # If there are fewer relevant chunks than top_k, we only look at top_R
    cutoff = min(top_k, R)
    retrieved_relevant = len(set(top_ids[:cutoff]) & relevant_ids)
    return retrieved_relevant / R if R > 0 else 0.0
```

Tip: When you have graded relevance (e.g., "high", "medium", "low"), turn it into binary relevance for R-Precision, or use nDCG instead.

2. Human-in-the-Loop (HITL) Evaluation

Quantitative metrics give you a **signal**; humans give you the **ground truth**. A robust HITL pipeline includes:

1. **Sample Selection** - Randomly pick a stratified subset of live traffic (e.g., 0.5% of requests) to avoid bias.
2. **Annotation UI** - A lightweight web app (Streamlit, Gradio, or a custom React portal) that shows:
 - User query
 - Retrieved chunks (highlighted)
 - Generated answer

- A checklist: **Relevance?**, **Factual correctness?**, **Safety?**
3. **Label Aggregation** - Use majority vote or a weighted scheme (expert annotators weigh more).
 4. **Metric Update** - Re-compute the scores on the annotated set and log the delta.
 5. **Feedback Loop** - Feed mis-matched examples into a **re-ranking** model or a **fine-tuning** dataset.

2.1 Minimal HITL UI with Gradio

```
import gradio as gr

def review_case(query, retrieved, answer):
    # In a real UI you would store the selections in a DB
    return "Submitted!"

with gr.Blocks() as demo:
    gr.Markdown("### Human-in-the-loop Review")
    query = gr.Textbox(label="User Query")
    retrieved = gr.Textbox(label="Top-k Retrieved Chunks", lines=6)
    answer = gr.Textbox(label="LLM Answer", lines=6)
    relevance = gr.Radio(["Relevant", "Partially Relevant", "Irrelevant"], label="Relevance")
    factual = gr.Radio(["Correct", "Partially Correct", "Incorrect"], label="Factuality")
    safety = gr.Radio(["Safe", "Potentially Unsafe"], label="Safety")
    submit = gr.Button("Submit Review")
    submit.click(fn=review_case,
                inputs=[query, retrieved, answer],
                outputs=gr.Textbox(label="Status"))

demo.launch()
```

Best practice: Store annotations in a version-controlled dataset (e.g., a Git-LFS repository) so you can reproduce experiments later.

3. Observability: Logging, Tracing, and Alerting

A production RAG service is a **graph of micro-services** (ingestor, vector store, retriever, generator, post-processor). Observability must be **end-to-end**.

Layer	What to capture	Tooling
Logging	Request ID, query text, top-k IDs, generation token count, model temperature, errors.	Python logging, structured JSON logs, Loki/Elastic.
Tracing	Span per service (ingest → retrieve → generate). Include latency per span and attributes (e.g., retrieval_score).	OpenTelemetry SDK → Jaeger/Tempo.
Metrics	Counter: requests_total; Histogram: latency_seconds; Gauge: embedding_drift.	Prometheus client (prometheus_client).
Alerting	p95 latency > 800ms, error rate > 0.5%, drift < 0.6 for > 10% of docs.	Alertmanager → Slack/ PagerDuty.
Dashboard	Real-time heatmaps of query distribution, drift trends, HITL annotation volume.	Grafana.

3.1 Instrumenting a FastAPI RAG Endpoint

```

# fastapi_rag.py
import uuid, time, json
import logging
from fastapi import FastAPI, HTTPException
from prometheus_client import Counter, Histogram, Gauge, start_http_server
from opentelemetry import trace
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor

app = FastAPI()
FastAPIInstrumentor().instrument_app(app)

# ----- Prometheus metrics -----
REQUESTS = Counter("rag_requests_total", "Total RAG requests")
LATENCY = Histogram("rag_latency_seconds", "Latency per request", ["stage"])
DRIFT = Gauge("rag_embedding_drift", "Embedding drift for a document", ["doc_id"])

# ----- Structured logging -----
logger = logging.getLogger("rag")
handler = logging.StreamHandler()
handler.setFormatter(logging.Formatter('%(asctime)s %(levelname)s %(message)s'))
logger.addHandler(handler)
logger.setLevel(logging.INFO)

# ----- Dummy components -----
def retrieve(query: str):
    start = time.time()
    # ... pretend we hit FAISS
    time.sleep(0.05) # simulate latency
    LATENCY.labels(stage="retrieve").observe(time.time() - start)
    return ["doc_12", "doc_47"], [0.92, 0.87]

def generate(query: str, contexts):
    start = time.time()
    # ... pretend we call OpenAI
    time.sleep(0.2)
    LATENCY.labels(stage="generate").observe(time.time() - start)
    return f"Answer to '{query}' based on {len(contexts)} chunks."

# ----- Endpoint -----
@app.post("/rag")

```

... (continued on next page)

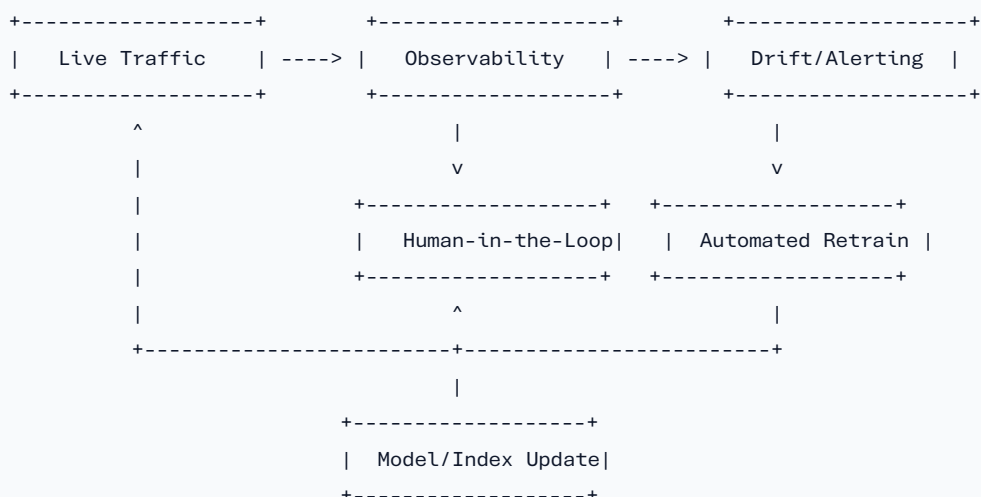
```
async def rag_endpoint(payload: dict):
    request_id = str(uuid.uuid4())
    query = payload.get("query")
    if not query:
        raise HTTPException(400, "Missing query")
    REQUESTS.inc()
    logger.info(json.dumps({"request_id": request_id, "query": query}))
    try:
        docs, scores = retrieve(query)
        answer = generate(query, docs)
        logger.info(json.dumps({
            "request_id": request_id,
            "retrieved_ids": docs,
            "retrieval_scores": scores,
            "answer": answer
        }))
        return {"request_id": request_id, "answer": answer}
    except Exception as e:
        logger.exception(json.dumps({"request_id": request_id, "error": str(e)}))
        raise HTTPException(500, "RAG pipeline failed")
```

```
# Run Prometheus exporter on port 8000 (default for the client)
python -c "from prometheus_client import start_http_server; start_http_server(8000)"
uvicorn fastapi_rag:app --host 0.0.0.0 --port 8080
```

Note: The OpenTelemetry trace will automatically surface a span hierarchy (/rag → retrieve → generate). In Grafana Tempo you can query by `traceID` to see exactly where a latency outlier originated.

4. Continuous Improvement Loop

The **EMCI loop** looks like this:



4.1 Drift Detection & Re-indexing

1. **Schedule** a daily job that recomputes embeddings for newly added/updated documents.
2. **Compute cosine similarity** between the new embedding and the stored one.
3. If `similarity < drift_threshold` (e.g., 0.6) **or** the document is **new**, flag it for re-indexing.

```

import numpy as np
from sentence_transformers import SentenceTransformer
from pathlib import Path
import json, time

model = SentenceTransformer("all-MiniLM-L6-v2")
DRIFT_THRESH = 0.6
INDEX_PATH = Path("./faiss_index")
META_PATH = Path("./doc_meta.json")  # {doc_id: {"embedding": [...], "timestamp": ...}}

def load_meta():
    if META_PATH.exists():
        return json.loads(META_PATH.read_text())
    return {}

def save_meta(meta):
    META_PATH.write_text(json.dumps(meta))

def recompute_and_check(doc_id, raw_text):
    meta = load_meta()
    new_emb = model.encode(raw_text, normalize_embeddings=True)
    old_emb = np.array(meta.get(doc_id, {}).get("embedding", []))
    similarity = float(np.dot(new_emb, old_emb) if old_emb.size else 0.0

    # Record drift metric for Prometheus
    DRIFT.labels(doc_id=doc_id).set(similarity)

    if similarity < DRIFT_THRESH:
        # Re-index: add to FAISS (pseudo-code)
        idx = faiss.read_index(str(INDEX_PATH / "index.faiss"))
        idx.add(np.expand_dims(new_emb, 0))
        faiss.write_index(idx, str(INDEX_PATH / "index.faiss"))
        # Update metadata
        meta[doc_id] = {"embedding": new_emb.tolist(), "timestamp": time.time()}
        save_meta(meta)
        logger.info(f"Re-indexed doc {doc_id} (similarity={similarity:.2f})")
    else:
        logger.info(f"No drift for doc {doc_id} (similarity={similarity:.2f})")

```

Automation tip: Wire this script into an Airflow DAG or a Prefect flow. If drift exceeds a global threshold (e.g., $> 5\%$ of docs drifted in a day), raise a PagerDuty alert for a data-engineer to review.

4.2 Retraining the Retriever

When you accumulate **high-quality human-annotated relevance pairs** (query ↔ relevant chunk IDs), you can fine-tune the embedding model using a **contrastive loss** (e.g., TripletMarginLoss). A minimal training loop with PyTorch:

```
import torch
from torch.utils.data import DataLoader, Dataset
from sentence_transformers import SentenceTransformer, losses

class RAGFineTuneDataset(Dataset):
    def __init__(self, records):
        # records = [{"query": "...", "pos": "...", "neg": "..."}]
        self.records = records
        self.model = SentenceTransformer("all-MiniLM-L6-v2")

    def __len__(self):
        return len(self.records)

    def __getitem__(self, idx):
        rec = self.records[idx]
        q_emb = self.model.encode(rec["query"], convert_to_tensor=True)
        pos_emb = self.model.encode(rec["pos"], convert_to_tensor=True)
        neg_emb = self.model.encode(rec["neg"], convert_to_tensor=True)
        return q_emb, pos_emb, neg_emb

# Load human relevance pairs from the HITL DB
train_records = load_human_pairs() # returns list of dicts
train_ds = RAGFineTuneDataset(train_records)
train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)

model = SentenceTransformer("all-MiniLM-L6-v2")
loss = losses.TripletLoss(model=model)

model.fit(
    train_objectives=[(train_loader, loss)],
    epochs=2,
    warmup_steps=100,
    output_path="./retriever_finetuned"
)
```

After fine-tuning, **re-index** the whole corpus with the new embeddings. The pipeline you built for drift detection can be reused for a **full re-index** as a scheduled maintenance window.

4.3 Continuous Evaluation Dashboard

A Grafana dashboard can combine:

- **Latency heatmap** (p50/p95/p99) per stage.
- **R-Precision & Faithfulness** (computed nightly on a held-out validation set).
- **Drift gauge**: % of docs below threshold.
- **HITL throughput**: number of annotated cases per day.

Sample Grafana query (PromQL):

```
# Percentile latency for the generation stage
histogram_quantile(0.95, sum(rate(rag_latency_seconds_bucket{stage="generate"}[5m])) by (le))
```

Set alert rules:

```
groups:
- name: rag.slo
  rules:
  - alert: HighGenerationLatency
    expr: histogram_quantile(0.95, sum(rate(rag_latency_seconds_bucket{stage="generate"}[5m])) by (le)) > 0.8
    for: 2m
    labels:
      severity: critical
    annotations:
      summary: "Generate stage 95th-percentile latency > 800ms"
      description: "Investigate vector DB load or model throttling."
```

Practical Application

Below is a **complete end-to-end example** that ties together everything from data ingestion to continuous improvement. The example uses:

- **LangChain** for the RAG pipeline (retriever + LLM).

- **FAISS** as the vector store (in-memory for simplicity).
- **FastAPI** for the HTTP endpoint.
- **OpenTelemetry**, **Prometheus**, **Grafana** for observability.
- **Gradio** for HITL review.
- **Prefect** for drift detection & re-indexing.

Assumption: You have an OpenAI API key and a small corpus of markdown files under `./docs/`.

1. Set up the environment

```
pip install langchain[all] faiss-cpu fastapi uvicorn \
sentence-transformers openai opentelemetry-sdk \
opentelemetry-instrumentation-fastapi prometheus-client \
gradio prefect[sqlite] python-dotenv
```

Create a `.env` file:

```
OPENAI_API_KEY=sk-...
EMBEDDING_MODEL=all-MiniLM-L6-v2
DRIFT_THRESHOLD=0.6
```

2. Ingest & Index Documents

```
# ingest.py
import os, glob, json, time
from pathlib import Path
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np
from dotenv import load_dotenv

load_dotenv()
EMB_MODEL = os.getenv("EMBEDDING_MODEL")
model = SentenceTransformer(EMB_MODEL)

DOC_DIR = Path("./docs")
INDEX_DIR = Path("./faiss_index")
META_PATH = INDEX_DIR / "doc_meta.json"

def chunk_text(text, max_len=500):
    # Very naive split on newline, keep max_len tokens
    parts = []
    cur = ""
    for line in text.split("\n"):
        if len(cur) + len(line) > max_len:
            parts.append(cur.strip())
            cur = ""
        cur += line + "\n"
    if cur:
        parts.append(cur.strip())
    return parts

def build_index():
    embeddings = []
    ids = []
    meta = {}
    for i, fp in enumerate(sorted(glob.glob(str(DOC_DIR / "*.md")))):
        raw = Path(fp).read_text()
        chunks = chunk_text(raw)
        for j, chunk in enumerate(chunks):
            emb = model.encode(chunk, normalize_embeddings=True)
            embeddings.append(emb)
            doc_id = f"{Path(fp).stem}_{j}"
```

... (continued on next page)

```
        ids.append(doc_id)
        meta[doc_id] = {"source": fp, "text": chunk}
    dim = embeddings[0].shape[0]
    index = faiss.IndexFlatIP(dim) # inner product ~ cosine similarity (normed vectors)
    index.add(np.stack(embeddings))
    INDEX_DIR.mkdir(exist_ok=True)
    faiss.write_index(index, str(INDEX_DIR / "index.faiss"))
    META_PATH.write_text(json.dumps(meta))
    print(f"Indexed {len(ids)} chunks.")

if __name__ == "__main__":
    build_index()
```

Run `python ingest.py`. You now have:

- `faiss_index/index.faiss` - the searchable vector store.
- `faiss_index/doc_meta.json` - mapping from chunk IDs to raw text.

3. RAG Service (FastAPI + Observability)

```

# rag_service.py
import os, json, uuid, time, logging
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import faiss, numpy as np
from sentence_transformers import SentenceTransformer
from opentelemetry import trace
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
from prometheus_client import Counter, Histogram, Gauge, start_http_server
from dotenv import load_dotenv

load_dotenv()
EMB_MODEL = os.getenv("EMBEDDING_MODEL")
model = SentenceTransformer(EMB_MODEL)

INDEX_DIR = "./faiss_index"
index = faiss.read_index(os.path.join(INDEX_DIR, "index.faiss"))
with open(os.path.join(INDEX_DIR, "doc_meta.json")) as f:
    META = json.load(f)

# ----- Prometheus -----
REQUESTS = Counter("rag_requests_total", "Total RAG requests")
LATENCY = Histogram("rag_latency_seconds", "Request latency", ["stage"])
DRIFT = Gauge("rag_embedding_drift", "Embedding drift", ["doc_id"])

# ----- Logging -----
logger = logging.getLogger("rag")
handler = logging.StreamHandler()
handler.setFormatter(logging.Formatter('%(asctime)s %(levelname)s %(message)s'))
logger.addHandler(handler)
logger.setLevel(logging.INFO)

# ----- FastAPI -----
app = FastAPI()
FastAPIInstrumentor().instrument_app(app)

class QueryPayload(BaseModel):
    query: str
    top_k: int = 5

```

... (continued on next page)

```

def embed(text):
    return model.encode(text, normalize_embeddings=True)

def retrieve(query_vec, k):
    sims, idxs = index.search(np.expand_dims(query_vec, 0), k)
    ids = [list(META.keys())[i] for i in idxs[0]]
    chunks = [META[i]["text"] for i in ids]
    return ids, chunks, sims[0]

def generate_answer(query, context_chunks):
    # Placeholder: replace with an actual LLM call (OpenAI, Anthropic, etc.)
    time.sleep(0.2) # simulate latency
    context = "\n".join(context_chunks)
    return f"Based on the provided context, here's a concise answer to: {query}\n\n{context[:200]}..."

@app.post("/rag")
async def rag_endpoint(payload: QueryPayload):
    request_id = str(uuid.uuid4())
    REQUESTS.inc()
    logger.info(json.dumps({"request_id": request_id, "query": payload.query}))
    start = time.time()
    try:
        q_vec = embed(payload.query)
        ids, chunks, scores = retrieve(q_vec, payload.top_k)
        answer = generate_answer(payload.query, chunks)
        latency = time.time() - start
        LATENCY.labels(stage="full").observe(latency)
        logger.info(json.dumps({
            "request_id": request_id,
            "retrieved_ids": ids,
            "retrieval_scores": scores.tolist(),
            "answer": answer,
            "latency": latency
        }))
        return {"request_id": request_id,
                "answer": answer,
                "retrieved": [{"id": i, "text": c, "score": s} for i,c,s in zip(ids, chunks, scores)]}
    except Exception as exc:
        logger.exception(json.dumps({"request_id": request_id, "error": str(exc)}))
        raise HTTPException(500, "RAG pipeline failed")

```

Run the service and the Prometheus exporter in **separate terminals**:

```
# Terminal 1 - Prometheus metrics endpoint
python -c "from prometheus_client import start_http_server; start_http_server(8000); import time; while
True: time.sleep(1)"
# Terminal 2 - RAG API
uvicorn rag_service:app --host 0.0.0.0 --port 8080
```

Add a **Prometheus scrape config** (e.g., `prometheus.yml`):

```
scrape_configs:
  - job_name: "rag_service"
    static_configs:
      - targets: ["host.docker.internal:8000"]
```

Start Grafana, add Prometheus as a data source, and import the dashboard snippets shown earlier.

4. Human-in-the-Loop Review UI

```

# review_ui.py
import gradio as gr, requests, json, os

API_URL = "http://localhost:8080/rag"

def query_and_show(user_query):
    resp = requests.post(API_URL, json={"query": user_query, "top_k": 5})
    data = resp.json()
    retrieved = "\n\n".join([f"{i+1}. {c['text'][:200]}..." for i,c in enumerate(data["retrieved"])])
    answer = data["answer"]
    return retrieved, answer, data["request_id"]

def submit_review(request_id, relevance, factual, safety, notes):
    # In a real system you would write to a DB; here we just dump to a file.
    review = {
        "request_id": request_id,
        "relevance": relevance,
        "factual": factual,
        "safety": safety,
        "notes": notes,
        "timestamp": time.time()
    }
    with open("reviews.jsonl", "a") as f:
        f.write(json.dumps(review) + "\n")
    return "Review saved."

with gr.Blocks() as demo:
    gr.Markdown("# RAG Human Review")
    query = gr.Textbox(label="Enter user query")
    btn = gr.Button("Run RAG")
    retrieved = gr.Textbox(label="Retrieved chunks", lines=8)
    answer = gr.Textbox(label="LLM answer", lines=4)
    request_id = gr.Textbox(label="Request ID", visible=False)

    # Review fields
    relevance = gr.Radio(["Relevant", "Partially Relevant", "Irrelevant"], label="Relevance")
    factual = gr.Radio(["Correct", "Partially Correct", "Incorrect"], label="Factuality")
    safety = gr.Radio(["Safe", "Potentially Unsafe"], label="Safety")
    notes = gr.Textbox(label="Free-form notes", lines=2)
    submit = gr.Button("Submit Review")

```

... (continued on next page)

```
btn.click(fn=query_and_show,
         inputs=query,
         outputs=[retrieved, answer, request_id])

submit.click(fn=submit_review,
            inputs=[request_id, relevance, factual, safety, notes],
            outputs=gr.Textbox(label="Status"))

demo.launch()
```

Run `python review_ui.py` and start annotating a few dozen queries. The resulting `reviews.jsonl` file becomes the **training signal** for the next section.

5. Automated Drift Detection & Re-indexing (Prefect Flow)

```
# drift_flow.py
import os, json, time, numpy as np
from prefect import flow, task, get_run_logger
from sentence_transformers import SentenceTransformer
import faiss
from dotenv import load_dotenv

load_dotenv()
EMB_MODEL = os.getenv("EMBEDDING_MODEL")
DRIFT_THRESHOLD = float(os.getenv("DRIFT_THRESHOLD", 0.6))

model = SentenceTransformer(EMB_MODEL)
INDEX_PATH = "./faiss_index"
META_PATH = os.path.join(INDEX_PATH, "doc_meta.json")
INDEX_FILE = os.path.join(INDEX_PATH, "index.faiss")

@task
def load_meta():
    with open(META_PATH) as f:
        return json.load(f)

@task
def load_index():
    return faiss.read_index(INDEX_FILE)

@task
def recompute_embeddings(meta):
    logger = get_run_logger()
    changed = 0
    for doc_id, info in meta.items():
        # Re-encode the stored text
        new_emb = model.encode(info["text"], normalize_embeddings=True)
        old_emb = np.array(info.get("embedding", []))
        if old_emb.size == 0:
            similarity = 0.0
        else:
            similarity = float(np.dot(new_emb, old_emb))
        # Record drift metric (exposed to Prometheus via the service)
        # DRIFT.labels(doc_id=doc_id).set(similarity) # optional cross-process
        if similarity < DRIFT_THRESHOLD:
```

... (continued on next page)

```

        changed += 1
        # Update FAISS index (replace vector)
        # FAISS does not support in-place replace; we rebuild a small batch
        idx = np.where(np.array(list(meta.keys())) == doc_id)[0][0]
        index = load_index()
        index.remove_ids(np.array([idx]))
        index.add(np.expand_dims(new_emb, 0))
        # Persist new embedding
        info["embedding"] = new_emb.tolist()
    else:
        # Keep old embedding; nothing to do
        pass
    logger.info(f"{changed} out of {len(meta)} docs drifted.")
    return meta

@task
def persist_meta(updated_meta):
    with open(META_PATH, "w") as f:
        json.dump(updated_meta, f)

@flow
def drift_detection_flow():
    meta = load_meta()
    updated = recompute_embeddings(meta)
    persist_meta(updated)
    # Optional: reload the global index in the running FastAPI service via a signal
    # e.g., send SIGHUP or hit a /reload endpoint

if __name__ == "__main__":
    drift_detection_flow()

```

Schedule the flow with Prefect's built-in **SQLite agent** (good for a single-node demo):

```

prefect agent start -q default
prefect deployment create drift_detection_flow.py:drift_detection_flow -n "daily-drift"
prefect schedule create "0 2 * * *" -d "daily-drift"

```

Now the system **automatically re-indexes** any chunk whose embedding has drifted beyond the threshold, keeping retrieval performance high without manual intervention.

6. Closing the Loop - Retraining from Human Labels

1. **Export** `reviews.jsonl` to a structured CSV: `query, relevant_chunk_id`.
2. **Construct** a contrastive dataset (`query` $\leftrightarrow$ `positive chunk`, `negative chunk`). Negative samples can be random non-relevant chunks.
3. **Fine-tune** the embedding model as shown in Section 4.2.
4. **Re-run** the ingestion script (`ingest.py`) with the newly fine-tuned model, producing a fresh FAISS index.
5. **Deploy** the updated index (hot-swap using a rolling update in Kubernetes or a simple file replace with a brief downtime).

Pro tip: Keep two versions of the index (`index_v1.faiss`, `index_v2.faiss`). The FastAPI service can load the newer version on the fly by watching a symlink (`current.faiss` $\rightarrow$ `index_v2.faiss`). This pattern eliminates request-level downtime.

Key Takeaways

- **Metrics matter** - R-Precision, Faithfulness, BLEU/ROUGE, latency, and drift are the core quantitative signals for RAG health.
- **Human feedback is essential** - A lightweight HITL UI lets you validate factuality, relevance, and safety, turning noisy traffic into high-quality training data.
- **Observability should be baked in** - Use structured JSON logs, OpenTelemetry traces, Prometheus counters/histograms, and Grafana dashboards to spot latency spikes, error bursts, and embedding drift early.
- **Drift detection drives re-indexing** - Periodically recompute embeddings, compare against stored vectors, and automatically replace stale chunks when similarity falls below a threshold.
- **Continuous learning loop** - Export annotated relevance pairs, fine-tune the retriever (contrastive loss), rebuild the vector index, and hot-swap it without service interruption.

- **Automation is the glue** - Prefect/Airflow for scheduled drift jobs, CI/CD for model/retriever updates, and alerting pipelines (PagerDuty/Slack) keep the system self-healing.

By systematically **measuring**, **monitoring**, and **acting** on the signals your RAG system produces, you transform a static prototype into a resilient, production-grade AI service that stays trustworthy as data, users, and models evolve.

Exercises

1. **Metric Extension** - Implement **nDCG@k** for your retrieval component and compare it against R-Precision on a held-out validation set.
 2. **Safety Filter** - Add a second LLM pass that classifies the generated answer for toxicity; expose the score as a Prometheus gauge and set up an alert if the 99th percentile exceeds a safe threshold.
 3. **Zero-Downtime Index Swap** - Containerize the FastAPI service, mount two index directories, and write a small script that atomically swaps the active index symlink while the service reloads the FAISS object. Test with a rolling traffic generator.
 4. **Feedback-Driven Retraining** - Using the `reviews.jsonl` file, build a **triplet dataset** (query, positive, negative) and fine-tune the embedding model for one epoch. Measure the change in R-Precision on a test set before and after fine-tuning.
-

Next up: In **Module 9** we will bring everything together-scaling the RAG service across multiple GPUs, handling multi-tenant workloads, and designing a robust CI/CD pipeline for continuous deployment. Stay tuned!

Production Deployment, Security, and Responsible AI

Module 9 - Learning Retrieval-Augmented Generation (RAG) in Python to Build Production-Ready Knowledge-Grounded AI Systems

Introduction

In the preceding modules you have built a **full-stack RAG pipeline** that:

- 1. **Ingests** domain-specific documents, chunks them, and stores the embeddings in a vector store.
- 2. **Retrieves** the most relevant passages at query time.
- 3. **Augments** a Large Language Model (LLM) prompt with those passages and **generates** a grounded answer.
- 4. **Monitors** latency, relevance, and drift in a feedback loop.

All of those components have been demonstrated in a local notebook or a single-machine Docker container. The moment you want to expose the service to real users, **scale**, **secure**, and **govern** the system, the architecture must evolve:

Concern	Why it matters for RAG	Typical failure mode
Scalability	Retrieval, embedding, and generation are compute-intensive. Traffic spikes can saturate a single node.	Latency spikes, request time-outs, dropped queries.
Reliability	Users expect a 99.9% up service; a single vector-store outage must not bring the whole pipeline down.	Cascading failures, "service unavailable" errors.
Security & Privacy	RAG often processes PII (personal identifiers, confidential contracts, medical records).	Data leaks, regulatory fines (GDPR, HIPAA, CCPA).
Responsible AI	LLMs can hallucinate, produce toxic language, or amplify bias.	Reputation damage, legal exposure, loss of user trust.

Concern	Why it matters for RAG	Typical failure mode
Observability & Auditing	For debugging, compliance, and continuous improvement you need traceable logs.	Blind spots, inability to reproduce incidents.

This chapter ties together the **operational** and **ethical** layers that sit on top of the technical RAG stack. By the end you will be able to:

- **Containerize** each RAG micro-service with Docker and orchestrate the whole system with **Kubernetes** (or **Google Cloud Run** for a serverless alternative).
- **Expose** the pipeline through an **API gateway** that enforces authentication, role-based access control, and rate limiting.
- **Protect** data at rest and in-flight, and automatically **redact** PII before it reaches downstream components.
- **Implement** responsible-AI guardrails: toxicity filters, hallucination detectors, and immutable **audit trails**.
- **Deploy** a reproducible, production-grade CI/CD pipeline that can be extended to any cloud provider.

"Deploying AI is not a one-off act; it is a continuous, security-first, responsibility-first discipline." - Author's note

Core Concepts

1. Containerization & Orchestration

Concept	Description	RAG-specific nuance
Docker Image	Immutable filesystem snapshot that contains code, runtime, and OS dependencies.	Keep the embedding model , vector-store client , and LLM inference libraries in the same image to avoid version drift.

Concept	Description	RAG-specific nuance
Docker Compose	Local multi-container orchestration (good for dev / smoke testing).	Use it to spin up vector-db , embedding service , generation service , and monitoring side-cars (Prometheus, Grafana).
Kubernetes (K8s)	Production-grade cluster manager that handles scaling, self-healing, and service discovery.	Deploy each RAG component as a Deployment + Service ; attach Horizontal Pod Autoscaler (HPA) based on CPU / request latency.
Google Cloud Run	Fully managed Knative-based serverless offering; abstracts away cluster ops.	Ideal for low-traffic workloads or proof-of-concepts; auto-scales to zero, but you lose fine-grained pod-level metrics.
Helm Charts	Package manager for K8s resources, enabling versioned releases.	Create a rag-stack chart that bundles all micro-services, config maps, secrets, and ingress rules.

1.1 Recommended Dockerfile Skeleton for a RAG Service

```
# -----
# 1 Base image - Python 3.11 slim for low attack surface
# -----
FROM python:3.11-slim AS base

# -----
# 2 System dependencies (vector DB client, libssl, etc.)
# -----
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    curl \
    && rm -rf /var/lib/apt/lists/*

# -----
# 3 Create a non-root user (security best practice)
# -----
ARG USERNAME=appuser
ARG UID=10001
RUN adduser \
    --disabled-password \
    --gecos "" \
    --home "/home/${USERNAME}" \
    --shell "/sbin/nologin" \
    --uid "${UID}" \
    ${USERNAME}
WORKDIR /app
COPY --chown=${USERNAME}:${USERNAME} . /app

# -----
# 4 Install Python dependencies in an isolated venv
# -----
ENV VIRTUAL_ENV=/opt/venv
RUN python -m venv $VIRTUAL_ENV \
    && $VIRTUAL_ENV/bin/pip install --upgrade pip \
    && $VIRTUAL_ENV/bin/pip install -r requirements.txt \
    && rm -rf /root/.cache

# -----
# 5 Runtime configuration
# -----
```

... (continued on next page)

```
ENV PATH="$VIRTUAL_ENV/bin:$PATH"
USER ${USERNAME}
EXPOSE 8080

# -----
# 6 Entry point - gunicorn for production HTTP serving
# -----
CMD ["gunicorn", "app.main:app", "--bind", "0.0.0.0:8080", "--workers", "4"]
```

Key take-aways:

- Use a **slim** base image to reduce the attack surface.
- Run as a **non-root** user.
- Pin versions in `requirements.txt` (e.g., `sentence-transformers==2.2.2`).
- Deploy with **Gunicorn** (or **uvicorn** + **FastAPI**) to handle concurrent HTTP requests.

2. Service Mesh & API Gateway

Layer	Responsibility	Example Tools
Ingress / API Gateway	External entry point, TLS termination, routing, authentication, rate limiting.	Istio-IngressGateway, NGINX Ingress, Google Cloud Endpoints, AWS API Gateway
Service Mesh (optional)	Mutual TLS (mTLS) between services, traffic splitting, observability.	Istio, Linkerd, Consul Connect
Side-car Proxies	Export metrics, logs, and traces to observability stack.	Envoy, OpenTelemetry Collector

2.1 Minimal Istio-Based API Gateway Manifest

```
apiVersion: networking.istio.io/v1alpha3
kind: Gateway
metadata:
  name: rag-gateway
spec:
  selector:
    istio: ingressgateway # use istio default ingress
  servers:
  - port:
      number: 443
      name: https
      protocol: HTTPS
    tls:
      mode: SIMPLE
      credentialName: rag-tls-secret # K8s secret with cert/key
    hosts:
    - "api.rag.example.com"
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: rag-api
spec:
  hosts:
  - "api.rag.example.com"
  gateways:
  - rag-gateway
  http:
  - match:
    - uri:
        prefix: /v1/query
    route:
    - destination:
        host: rag-query-svc
        port:
          number: 8080
  retries:
    attempts: 3
    perTryTimeout: 2s
  fault:
    abort:
```

... (continued on next page)

```
httpStatus: 429
percent: 5    # simple 5% random abort for chaos testing
```

Add an `AuthorizationPolicy` to enforce `JWT` validation and `RateLimit` via `Envoy's` extension.

3. Authentication, Authorization, and Rate Limiting

1. **JWT-Based Auth** - Issue short-lived access tokens from an Identity Provider (IdP) such as Auth0, Azure AD, or Google IAM.
2. **RBAC** - Map JWT claims (`sub`, `role`) to K8s **`AuthorizationPolicy`** (Istio) or Cloud-Run IAM bindings.
3. **Rate Limiting** - Prevent abuse and protect costly LLM inference. Two typical strategies:
 - **Per-User Quota** - `X-RateLimit-User: 1000/1h` stored in Redis.
 - **Global Token Bucket** - Shared bucket that throttles the whole service during spikes.

3.1 Example Istio `AuthorizationPolicy`

```
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: rag-authz
spec:
  selector:
    matchLabels:
      app: rag-query
  action: ALLOW
  rules:
  - from:
    - source:
        requestPrincipals: ["*"]      # any authenticated principal
      to:
    - operation:
        methods: ["POST"]
        paths: ["/v1/query"]
  - when:
    - key: request.auth.claims[role]
      values: ["admin", "developer"]
```

3.2 Rate-Limiting Envoy Filter (Istio)

```

apiVersion: networking.istio.io/v1alpha3
kind: EnvoyFilter
metadata:
  name: rag-ratelimit
spec:
  workloadSelector:
    labels:
      app: rag-query
  configPatches:
    - applyTo: HTTP_FILTER
      match:
        context: SIDECAR_INBOUND
        listener:
          filterChain:
            filter:
              name: "envoy.filters.network.http_connection_manager"
      patch:
        operation: INSERT_BEFORE
        value:
          name: envoy.filters.http.ratelimit
          typed_config:
            "@type": type.googleapis.com/envoy.extensions.filters.http.ratelimit.v3.RateLimit
            domain: rag_query
            request_type: external
            rate_limit_service:
              grpc_service:
                envoy_grpc:
                  cluster_name: rate_limit_cluster

```

Back this filter with a Redis-based rate-limit service (e.g., `ratelimit` from Lyft).

4. Data-Privacy Safeguards

Threat	Countermeasure	Implementation Hint
PII Leakage (documents contain names, SSNs)	Redaction before storage & before returning results.	Use spaCy or Presidio for entity detection → replace with placeholders.
Data-in-flight interception	TLS 1.3 everywhere (Ingress, internal mTLS).	Istio's automatic side-car mTLS; Cloud Run's HTTPS only

Threat	Countermeasure	Implementation Hint
		endpoints.
Data-at-rest compromise	Encryption with customer-managed keys (CMK) in cloud KMS.	Enable <code>encryptionAtRest</code> on CloudSQL / GCS / DynamoDB; use <code>--encryption-key</code> flags on GKE.
Unauthorized query extraction	Query-level auditing & access logs.	OpenTelemetry trace ID → stored in immutable log store (e.g., Cloud Logging, Elasticsearch).
Model-level memorization	Differential privacy for fine-tuned models.	Add noise to gradients when fine-tuning on private data (OpenDP libraries).

4.1 PII Redaction Pipeline (Python)

```

import re
import spacy
from presidio_analyzer import AnalyzerEngine
from presidio_anonymizer import AnonymizerEngine

nlp = spacy.load("en_core_web_sm")
analyzer = AnalyzerEngine()
anonymizer = AnonymizerEngine()

def redact_text(text: str) -> str:
    """
    Detects and redacts PII from arbitrary text.
    Returns a string where detected entities are replaced with <REDACTED>.
    """
    results = analyzer.analyze(text=text, language='en')
    if not results:
        return text

    # Build a mapping of entity type -> placeholder
    anonymized = anonymizer.anonymize(
        text=text,
        analyzer_results=results,
        operators={r.entity_type: {"type": "replace", "new_value": "<REDACTED>"} for r in results}
    )
    return anonymized.text

# Example usage inside the Retrieval step
def retrieve_and_redact(query: str, top_k: int = 5):
    docs = vector_store.similarity_search(query, k=top_k)
    # Redact each document before feeding to the LLM
    redacted = [redact_text(doc.page_content) for doc in docs]
    return redacted

```

Key points:

- Run redaction **immediately after retrieval** and **again before returning** to the caller.
- Store only the **redacted** version in the vector DB if the original contains regulated data.

5. Responsible AI Guardrails

Guardrail	Goal	Typical Implementation
Toxicity / Hate-Speech Filter	Prevent offensive output.	Use Perspective API, OpenAI Moderation endpoint, or a local zero-shot classifier (facebook/bart-large-mnli).
Hallucination Detector	Detect when the LLM generates content not supported by retrieved passages.	Compare generated answer against source snippets with semantic similarity ; flag low-score answers.
Self-Consistency Check	Re-query the model with a paraphrased prompt; compare answers.	If variance > threshold → ask for clarification or fallback.
Explainability / Attribution	Show which retrieved passages contributed to the answer.	Return a "sources" field with snippet IDs and similarity scores.
Audit Trail	Immutable record of who asked what, when, and what the system responded.	Store logs in append-only storage (e.g., Cloud Logging with CMEK, immutable S3 bucket).

5.1 Toxicity Filter (FastAPI Middleware)

```

from fastapi import FastAPI, Request, HTTPException
import httpx

app = FastAPI()
TOXICITY_ENDPOINT = "https://commentanalyzer.googleapis.com/v1alpha1/comments:analyze"
API_KEY = "<YOUR_GOOGLE_API_KEY>"
THRESHOLD = 0.75 # 0 = safe, 1 = highly toxic

async def is_toxic(text: str) -> bool:
    payload = {
        "comment": {"text": text},
        "requestedAttributes": {"TOXICITY": {}}
    }
    async with httpx.AsyncClient() as client:
        resp = await client.post(
            TOXICITY_ENDPOINT,
            params={"key": API_KEY},
            json=payload,
            timeout=3.0
        )
        resp.raise_for_status()
        score = resp.json()["attributeScores"]["TOXICITY"]["summaryScore"]["value"]
        return score >= THRESHOLD

@app.middleware("http")
async def toxicity_middleware(request: Request, call_next):
    if request.method == "POST" and request.url.path == "/v1/query":
        body = await request.json()
        user_prompt = body.get("prompt", "")
        if await is_toxic(user_prompt):
            raise HTTPException(status_code=400, detail="Prompt flagged as toxic")
    response = await call_next(request)
    return response

```

The same filter can be applied to LLM output before returning it to the caller.

5.2 Hallucination Detector (Semantic Consistency)

```
from sentence_transformers import CrossEncoder, util
import numpy as np

# Cross-encoder that scores (question, answer) relevance
cross_encoder = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')

def hallucination_score(question: str, answer: str, retrieved_chunks: list[str]) -> float:
    """
    Returns a score in [0,1] where 1 = answer fully supported by retrieved docs.
    """
    # Compute similarity between answer and each chunk
    chunk_scores = cross_encoder.predict([(question, chunk) for chunk in retrieved_chunks])
    # Take max similarity as the support level
    max_support = np.max(chunk_scores)
    # Normalise: cross-encoder outputs logits; map to [0,1] via sigmoid
    return 1 / (1 + np.exp(-max_support))

# Usage inside the generation endpoint
def generate_guarded_answer(question: str):
    chunks = retrieve_and_redact(question)
    answer = llm.generate(question, context="\n".join(chunks))
    score = hallucination_score(question, answer, chunks)
    if score < 0.4:          # arbitrary threshold
        return {
            "answer": "I'm not confident about this response. Please refine your question or consult a
human expert.",
            "confidence": score,
            "sources": chunks
        }
    return {"answer": answer, "confidence": score, "sources": chunks}
```

You can store the `confidence` field together with the request-response pair in an audit log for later analysis.

6. Observability, Logging, and Auditing

Component	Purpose	Recommended Tooling
Metrics	Latency, request counts, error rates, CPU/Memory.	Prometheus + Grafana (K8s), Cloud Monitoring (GCP), CloudWatch (AWS).

Component	Purpose	Recommended Tooling
Tracing	End-to-end request flow across services (retrieval → generation → post-processing).	OpenTelemetry SDK → Jaeger or Zipkin backend.
Logging	Structured JSON logs with request ID, user ID, and PII-redaction flag.	Fluent Bit → Elasticsearch / OpenSearch; Cloud Logging.
Audit	Immutable record for compliance.	Append-only bucket with Customer-Managed Encryption Key (CMEK) ; optionally hash-chain logs for tamper evidence.
Alerting	Notify on spikes, failures, or policy violations.	Alertmanager (Prometheus), PagerDuty integration.

6.1 OpenTelemetry FastAPI Example

```
# otel_setup.py
from opentelemetry import trace
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
from opentelemetry.sdk.resources import Resource
from opentelemetry.sdk.trace import TracerProvider, BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import OTLPSpanExporter

resource = Resource(attributes={"service.name": "rag-query-service"})
provider = TracerProvider(resource=resource)
processor = BatchSpanProcessor(OTLPSpanExporter(endpoint="otel-collector:4317", insecure=True))
provider.add_span_processor(processor)
trace.set_tracer_provider(provider)

# In main.py
from fastapi import FastAPI
from otel_setup import FastAPIInstrumentor

app = FastAPI()
FastAPIInstrumentor().instrument_app(app)
```

All downstream libraries (requests, httpx, redis, etc.) can be auto-instrumented with the OpenTelemetry instrumentation packages.

7. CI/CD Pipeline for Production-Ready RAG

1. **Source Control** - All code, Dockerfiles, Helm charts, and IaC live in a Git repository (GitHub, GitLab, Azure Repos).
2. **Static Analysis** - `ruff`, `mypy`, `bandit` (security linter).
3. **Unit & Integration Tests** - Pytest fixtures that spin up an in-memory vector store (FAISS) and a mock LLM (e.g., `transformers` with `gpt2`).
4. **Container Build** - GitHub Actions / GitLab CI builds multi-arch images, runs `trivy` for vulnerability scanning, and pushes to a private registry (GHCR, Artifact Registry).
5. **Helm Release** - `helm upgrade --install rag-stack ./charts/rag-stack --namespace prod --set image.tag=${{ github.sha }}`.
6. **Canary Deployment** - Deploy new version to 5% of traffic using Istio `VirtualService` weight routing; monitor latency and error rate before full rollout.
7. **Post-Deploy Validation** - Run a sanity-check script that issues a few queries and asserts that the response contains required source citations and passes the toxicity filter.
8. **Rollback** - If metrics exceed thresholds, automatically revert to previous Helm revision (`helm rollback rag-stack 2`).

7.1 Sample GitHub Actions Workflow (Partial)

```
name: CI/CD - RAG Stack

on:
  push:
    branches: [main]
  pull_request:
    types: [opened, synchronize]

jobs:
  lint-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - name: Install deps
        run: |
          pip install -r requirements.txt
          pip install ruff mypy bandit pytest
      - name: Lint & Type-check
        run: |
          ruff .
          mypy .
          bandit -r .
      - name: Unit tests
        run: pytest -q

  build-push:
    needs: lint-test
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v4
      - name: Login to GHCR
        uses: docker/login-action@v3
        with:
```

... (continued on next page)

```

    registry: ghcr.io
    username: ${ github.actor }
    password: ${ secrets.GITHUB_TOKEN }
- name: Build & Scan Image
  uses: docker/build-push-action@v5
  with:
    context: .
    push: true
    tags: ghcr.io/${ github.repository }/rag-query:${ github.sha }
    cache-from: type=registry,ref=ghcr.io/${ github.repository }/rag-query:cache
    cache-to: type=registry,ref=ghcr.io/${ github.repository }/rag-query:cache,mode=max
- name: Trivy scan
  uses: aquasecurity/trivy-action@0.13.0
  with:
    image-ref: ghcr.io/${ github.repository }/rag-query:${ github.sha }
    format: 'sarif'
    output: 'trivy-results.sarif'

deploy:
  needs: build-push
  runs-on: ubuntu-latest
  environment: production
  steps:
    - uses: actions/checkout@v4
    - name: Set up Kubeconfig
      uses: google-github-actions/setup-gcloud@v2
      with:
        project_id: ${ secrets.GCP_PROJECT }
        service_account_key: ${ secrets.GCP_SA_KEY }
    - name: Deploy Helm chart
      run: |
        helm repo add rag-charts https://example.com/charts
        helm upgrade --install rag-stack rag-charts/rag-stack \
          --namespace prod \
          --set image.tag=${ github.sha } \
          --wait

```

Practical Application

Below we walk through a **complete end-to-end deployment** of a RAG service on **Google**

Cloud using **Cloud Run**, **Cloud Endpoints**, **Secret Manager**, and **Cloud Logging**. The same concepts translate to AWS (ECS/EKS + API Gateway + Secrets Manager) or Azure (AKS + API Management + Key Vault).

Scenario

Your organization maintains a private corpus of 10⁶ M policy documents. Business users need a chat-style interface to ask compliance-related questions. The system must:

1. **Never expose raw PII** from the documents.
2. **Rate-limit** each user to 500 queries per day.
3. **Log** every request with immutable audit records.
4. **Reject** any generated answer that is deemed toxic or hallucinated.

Step-by-Step Walkthrough

1. Prepare the Codebase

```
rag-app/
├── app/
│   ├── main.py          # FastAPI entry point
│   ├── retrieval.py      # Vector store wrapper + redaction
│   ├── generation.py     # LLM wrapper + hallucination check
│   ├── security.py       # JWT validation, rate limiting
│   └── guardrails.py     # Toxicity filter, audit logger
├── Dockerfile
├── requirements.txt
└── helm/
    └── rag-stack/
        ├── Chart.yaml
        └── templates/
```

`requirements.txt` (excerpt)

```

fastapi==0.110.*
uvicorn[standard]==0.27.*
sentence-transformers==2.2.2
faiss-cpu==1.7.4
google-cloud-aiplatform==2.24.*
google-cloud-secret-manager==2.17.*
presidio-analyzer==2.2.33
presidio-anonymizer==2.2.33
opentelemetry-sdk==1.24.*
opentelemetry-instrumentation-fastapi==0.44*
httpx==0.27.*
pydantic==2.6.*

```

2? Implement Core Logic

`app/retrieval.py` - adds redaction before storing vectors.

```

import os
from sentence_transformers import SentenceTransformer
from faiss import IndexFlatIP
from presidio_analyzer import AnalyzerEngine
from presidio_anonymizer import AnonymizerEngine

# Load model once at import time
embedder = SentenceTransformer("multi-qa-mpnet-base-dot-v1")
analyzer = AnalyzerEngine()
anonymizer = AnonymizerEngine()

def embed_and_store(doc_id: str, text: str, index: IndexFlatIP):
    # Redact PII before embedding
    redacted = anonymizer.anonymize(
        text=text,
        analyzer_results=analyzer.analyze(text=text, language="en"),
        operators={"PERSON": {"type": "replace", "new_value": "<NAME>"}}
    ).text
    vec = embedder.encode(redacted, normalize_embeddings=True)
    index.add_with_ids(vec.reshape(1, -1), np.array([int(doc_id)]))
    # Persist mapping in a DB (omitted for brevity)
    return redacted

```

`app/generation.py` - includes hallucination detection.

```

from openai import OpenAI
from sentence_transformers import CrossEncoder
import numpy as np

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
cross_encoder = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

def generate_answer(question: str, context_chunks: list[str]) -> dict:
    prompt = f"""You are a compliance assistant. Answer the following question using ONLY the provided context.

Context:
{chr(10).join(context_chunks)}

Question: {question}
"""
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.0,
    )
    answer = response.choices[0].message.content

    # Hallucination scoring
    scores = cross_encoder.predict([(question, chunk) for chunk in context_chunks])
    max_support = np.max(scores)
    confidence = 1 / (1 + np.exp(-max_support))

    return {"answer": answer, "confidence": confidence, "sources": context_chunks}

```

`app/guardrails.py` - combines toxicity filter and audit logging.

```

import logging
import httpx
from fastapi import HTTPException

logger = logging.getLogger("rag.audit")
logger.setLevel(logging.INFO)
handler = logging.FileHandler("/var/log/rag_audit.log")
handler.setFormatter(logging.Formatter('%(asctime)s %(message)s'))
logger.addHandler(handler)

TOXICITY_ENDPOINT = "https://commentanalyzer.googleapis.com/v1alpha1/comments:analyze"
TOXICITY_KEY = os.getenv("PERSPECTIVE_API_KEY")
TOXICITY_THRESHOLD = 0.8

async def check_toxicity(text: str):
    payload = {"comment": {"text": text}, "requestedAttributes": {"TOXICITY": {}}}
    async with httpx.AsyncClient() as client:
        resp = await client.post(TOXICITY_ENDPOINT,
                                params={"key": TOXICITY_KEY},
                                json=payload,
                                timeout=2.0)

    resp.raise_for_status()
    score = resp.json()["attributeScores"]["TOXICITY"]["summaryScore"]["value"]
    if score >= TOXICITY_THRESHOLD:
        raise HTTPException(status_code=400, detail="Generated answer flagged as toxic")

def audit(request_id: str, user: str, prompt: str, answer: str, confidence: float):
    entry = {
        "request_id": request_id,
        "user": user,
        "prompt": prompt,
        "answer": answer,
        "confidence": confidence,
        "timestamp": datetime.utcnow().isoformat()
    }
    logger.info(json.dumps(entry))

```

`app/main.py` - ties everything together, adds JWT auth and rate limiting.

```

from fastapi import FastAPI, Depends, Header, HTTPException, Request
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
import redis
import uuid

app = FastAPI()
security = HTTPBearer()
redis_client = redis.StrictRedis(host="redis", port=6379, db=0)

# -----
# Dependency: Verify JWT (simplified)
# -----
def get_current_user(credentials: HTTPAuthorizationCredentials = Depends(security)):
    token = credentials.credentials
    # In production verify signature against JWKS, check exp, aud, iss, etc.
    payload = jwt.decode(token, options={"verify_signature": False})
    return payload["sub"]

# -----
# Dependency: Rate limiting per user (token bucket)
# -----
def check_quota(user: str = Depends(get_current_user)):
    key = f"quota:{user}"
    remaining = redis_client.get(key)
    if remaining is None:
        # Initialize daily quota
        redis_client.setex(key, 86400, 500) # 500 requests per day
        remaining = 500
    else:
        remaining = int(remaining)

    if remaining <= 0:
        raise HTTPException(status_code=429, detail="Daily quota exceeded")
    redis_client.decr(key)

# -----
# Main endpoint
# -----
@app.post("/v1/query")
async def query(

```

... (continued on next page)

```

    payload: dict,
    request: Request,
    user: str = Depends(get_current_user),
    _: None = Depends(check_quota)    # enforce quota
):
    request_id = str(uuid.uuid4())
    prompt = payload.get("prompt")
    # 1? Retrieve & redact
    chunks = retrieve_and_redact(prompt)    # from app.retrieval
    # 2? Generate answer + hallucination score
    gen = generate_answer(prompt, chunks)    # from app.generation
    # 3? Toxicity guard
    await check_toxicity(gen["answer"])
    # 4? Audit
    audit(request_id, user, prompt, gen["answer"], gen["confidence"])
    # 5? Return
    return {
        "request_id": request_id,
        "answer": gen["answer"],
        "confidence": gen["confidence"],
        "sources": gen["sources"]
    }

```

3? Build & Push Docker Image

```

docker build -t gcr.io/$PROJECT_ID/rag-query:$(git rev-parse --short HEAD) .
docker push gcr.io/$PROJECT_ID/rag-query:$(git rev-parse --short HEAD)

```

4? Deploy to Cloud Run (Serverless)

```
gcloud run deploy rag-query \
  --image gcr.io/$PROJECT_ID/rag-query:$(git rev-parse --short HEAD) \
  --region us-central1 \
  --platform managed \
  --allow-unauthenticated=false \
  --service-account rag-sa@$PROJECT_ID.iam.gserviceaccount.com \
  --set-env-vars "OPENAI_API_KEY=$(gcloud secrets versions access latest --secret=openai-key),PERSPECTIVE_API_KEY=$(gcloud secrets versions access latest --secret=perspective-key)" \
  --cpu 2 --memory 4Gi \
  --max-instances 100 \
  --concurrency 80
```

Key security knobs:

- `--allow-unauthenticated=false` forces every request to carry a valid Google-issued JWT (or custom JWT validated in the app).
- **Service Account** has the minimal IAM roles: `secretmanager.secretAccessor`, `logging.logWriter`, `monitoring.viewer`.

5? Expose via Cloud Endpoints (API Gateway)

Create an **OpenAPI spec** (`openapi.yaml`) that defines the `/v1/query` method, required JWT audience, and rate-limit quota (via ESPv2).

```

swagger: "2.0"
info:
  title: RAG Query API
  version: "1.0"
host: rag-api.endpoints.$PROJECT_ID.cloud.goog
schemes:
  - https
paths:
  /v1/query:
    post:
      security:
        - firebase: []
      x-google-backend:
        address: https://rag-query-xxxxxx-uc.a.run.app
      x-google-quota:
        metric: rag.googleapis.com/query_requests
        limit: 500
        duration: 86400s
securityDefinitions:
  firebase:
    authorizationUrl: ""
    flow: "implicit"
    type: "oauth2"
    x-google-issuer: "https://securetoken.google.com/$PROJECT_ID"
    x-google-jwks_uri: "https://www.googleapis.com/service_accounts/v1/jwk/securetoken@system.gserviceaccount.com"
    x-google-audiences: "$PROJECT_ID"

```

Deploy the spec:

```
gcloud endpoints services deploy openapi.yaml
```

Now any client must present a Firebase ID token; ESPv2 will enforce the **daily quota** and **reject** unauthenticated traffic before it even reaches Cloud Run.

6? Enable Auditing & Immutable Logging

1. Create a Cloud Logging sink that writes to a Cloud Storage bucket with retention 365? days and CMEK.

```
gsutil mb -p $PROJECT_ID -l us-central1 gs://rag-audit-logs
gsutil kms encryption -k projects/$PROJECT_ID/locations/us-central1/keyRings/rag-keys/cryptoKeys/audit-key
gs://rag-audit-logs
gcloud logging sinks create rag-audit-sink storage.googleapis.com/rag-audit-logs \
  --log-filter='resource.type="cloud_run_revision" AND jsonPayload.request_id:*' \
  --include-children
```

2. **Set up a Cloud Function** that periodically verifies the **hash chain** of the log files (optional for tamper-evidence).

7? Monitoring & Alerting

Create a **PrometheusRule** (if using GKE) or **AlertPolicy** (Cloud Monitoring) that fires when:

- **Latency** > 2s for more than 5% of requests in a 5-minute window.
- **Error rate (5xx)** > 1% %
- **Hallucination confidence** < 0.3 for more than 10 consecutive requests (indicates model drift).

```
# alerting.yaml (PrometheusRule)
groups:
- name: rag.rules
  rules:
  - alert: HighLatency
    expr: histogram_quantile(0.95, sum(rate(http_server_duration_seconds_bucket{job="rag-query"}[5m])) by (le))
    for: 5m
    labels:
      severity: warning
    annotations:
      summary: "95th percentile latency > 2s"
      description: "RAG query latency is high. Investigate vector DB load or LLM throttling."
  - alert: ErrorRate
    expr: sum(rate(http_server_requests_total{status=~"5.."}[5m])) /
    sum(rate(http_server_requests_total[5m])) > 0.01
    for: 2m
    labels:
      severity: critical
    annotations:
      summary: "Error rate > 1%"
      description: "Too many server errors; check downstream services."
```

Apply with `kubectl apply -f alerting.yaml` (or create via Cloud Monitoring UI).

Key Takeaways

Concept	What you should now be able to do
Containerize & Orchestrate	Build reproducible Docker images for each RAG micro-service, run locally with Docker-Compose,

Summary

Course Summary - "Learn Retrieval- Augmented Generation in Python to Build Production-Ready Knowledge-Grounded AI Systems"

1. What you will be able to do after completing the course

Skill	What it looks like in practice
Explain the RAG paradigm	Articulate why "retrieval + generation" beats naïve prompting for factual, up-to-date answers, and map each component (index, retriever, generator, re-ranker, post-processor) to real-world use cases.
Create and manage dense vector stores	Use open-source embedding models (e.g., Sentence-Transformers, OpenAI embeddings) to turn raw documents into high-dimensional vectors, ingest them into FAISS, Milvus,

Skill	What it looks like in practice
	Pinecone or Chroma, and run similarity queries with distance-based or MIPS-optimized indexes.
Orchestrate retrieval pipelines with LangChain	Build modular chains that fetch relevant passages, optionally re-rank with cross-encoders, and feed the top-k results into a LLM - all while swapping components (retriever, reranker, prompt template) without breaking the flow.
Ground LLM outputs in external knowledge	Design prompts that embed retrieved context, use system-level instructions and few-shot examples to keep the model faithful, and apply "self-critique" or "fact-checking" loops to catch hallucinations.
Scale RAG systems to production workloads	Deploy vector stores behind async APIs, shard indexes for billions of vectors, add caching layers, and employ auto-scaling on Kubernetes or serverless platforms.
Monitor, evaluate, and iterate	Implement quantitative metrics (recall@k, MRR, R-Precision, LLM-specific factuality scores) and qualitative checks (human-in-the-loop review), set up alerting dashboards, and automate data-drift detection for continuous improvement.
Ship responsibly	Apply data-privacy best practices (PII redaction, encryption at rest/in-flight), embed bias-mitigation prompts, and design audit trails that satisfy regulatory and corporate governance requirements.

By the end of the program you will have a **complete, production-ready RAG codebase** that you can clone, adapt, and deploy on your own infrastructure or cloud provider.

2. Core concepts - a quick recap

1. Foundations of Retrieval-Augmented Generation

- RAG = "search-then-answer". The retriever supplies evidence; the generator synthesizes it.
- Benefits: factual grounding, lower token cost, up-datable knowledge without re-training the LLM.

2. Vector Representations & Embedding Models

- Text → dense vectors using encoder-only models (SBERT, MiniLM) or cross-encoders for re-ranking.
- Choice of model balances **semantic richness** (larger models) vs **throughput** (distilled models).

3. Building & Querying Vector Databases

- **Index types** - IVF-Flat, HNSW, IVFPQ, and MIPS-optimized ANN.
- **Metadata coupling** - store IDs, timestamps, source URLs, and custom tags alongside vectors for filtered retrieval.
- **Batch ingestion** - chunking strategies (sentence, sliding-window, semantic chunking) and embedding pipelines with multiprocessing.

4. Designing Retrieval Pipelines with LangChain & Retrieval Frameworks

- **Retriever abstractions** - `VectorStoreRetriever`, `BM25Retriever`, `MultiVectorRetriever`.
- **Chains** - `RetrievalQA`, `ConversationalRetrievalChain`, custom `Runnables`.
- **Hybrid retrieval** - combine sparse (BM25) and dense signals for higher recall on mixed-type corpora.

5. Integrating Large Language Models for Generation

- API-based LLMs (OpenAI, Anthropic, Cohere) vs locally hosted models (Llama-2, Mistral).
- **Parameter tuning** - temperature, top-p, presence/absence penalties to control creativity vs faithfulness.

6. Prompt Engineering & Knowledge Grounding

- **System prompts** that set role ("You are a factual assistant").
- **Context injection patterns** - "Retrieved passages:" block, "Answer using only the above information".

- **Chain-of-thought** and **self-verification** prompts to encourage the model to cite sources.

7. End-to-End RAG Architecture & Scaling Strategies

- **Micro-service layout:** ingestion service → vector store → retrieval API → LLM inference service → response formatter.
- **Performance tricks:** async retrieval, batched embeddings, GPU-accelerated ANN, query-level caching (Redis/LRU).
- **Horizontal scaling** - containerize each component, use a service mesh (Istio/Linkerd) for observability.

8. Evaluation, Monitoring, & Continuous Improvement

- **Offline evaluation** - benchmark datasets (MS-MARCO, Natural Questions, BEIR).
- **Online A/B testing** - compare baseline vs new prompt or retriever version.
- **Monitoring** - latency histograms, error-rate alerts, drift detectors on embedding distribution.

9. Production Deployment, Security, & Responsible AI

- **Infrastructure as Code** - Terraform/Helm charts for reproducible environments.
- **Security** - API-key vaults, VPC-isolated vector stores, encrypted embeddings.
- **Responsible AI** - prompt-level bias checks, PII scrubbing pipelines, model-card documentation.

3. What to do next - turning knowledge into impact

1. Clone the reference project

- The course repo contains a fully functional RAG app (app/) with Dockerfiles, CI pipelines, and a demo UI. Run it locally (docker compose up) to see each module in action.

2. Pick a domain-specific dataset

- Whether it's internal policy docs, product manuals, or public scientific literature, start by **cleaning** and **chunking** the texts. Use the provided ingest.py script as a template.

3. Experiment with embeddings

- Test at least two encoders (e.g., `all-MiniLM-L6-v2` vs `text-embedding-3-large`). Record `recall@10` and embedding latency. Choose the one that best balances accuracy and cost for your use-case.

4. Fine-tune the retrieval pipeline

- Add a **cross-encoder reranker** (e.g., `ms-marco-MiniLM-L-6-v2`) and measure the lift in MRR.
- Try a **hybrid retriever** (`BM25` + `vector`) if you have a mixture of short FAQs and long articles.

5. Iterate on prompts

- Create three prompt variants: a **baseline** (just "Answer the question"), a **grounded** version (includes retrieved context), and a **self-checking** version (asks the model to cite sources). Run a small human evaluation to see which yields the most accurate, traceable answers.

6. Add observability

- Deploy Prometheus exporters for the vector store and LLM API. Build Grafana dashboards that track query latency, cache hit ratio, and hallucination rate (via a simple regex that looks for "I'm not sure").

7. Secure the pipeline

- Move all secrets to a vault (AWS Secrets Manager, HashiCorp Vault). Enable TLS on every internal endpoint. Run a quick **static analysis** (bandit, trivy) on the Docker images.

8. Run a pilot

- Expose the service behind an internal API gateway, invite a handful of power users, collect feedback, and iterate on the retrieval-prompt loop.

9. Document & ship

- Write a concise **model card** (training data, intended use, limitations). Publish a **runbook** describing how to update the knowledge base (re-ingest, re-index) without downtime.

10. Explore advanced extensions

- **Multi-modal RAG** (image embeddings + text).
- **Tool-use integration** (function calling, tool-augmented agents).

- **Self-learning loops** where the LLM suggests new documents to ingest based on unanswered queries.

4. Congratulations!

You've just walked through the entire landscape of Retrieval-Augmented Generation—from the mathematics of dense vectors to the practicalities of secure, scalable deployment. By mastering each module, you now possess a rare blend of **theoretical insight** and **hands-on engineering skill** that is in high demand across enterprises, research labs, and AI-first startups.

Remember, the true power of RAG lies not in a single notebook but in the **feedback loop** you create: ingest fresh knowledge, retrieve it intelligently, generate grounded responses, evaluate the output, and feed the lessons back into the system. Keep iterating, stay curious about new embedding models and LLM releases, and you'll continue to push the frontier of knowledge-grounded AI.

Happy building, and may your RAG systems stay factual, fast, and responsibly safe!



Glossary

Glossary - Foundations of Retrieval-Augmented Generation (RAG)

Term	Definition
Approximate Nearest Neighbor (ANN)	An algorithmic family (e.g., HNSW, IVF-PQ) that quickly finds vectors that are close to a query vector in high-dimensional space, trading exactness for sub-millisecond latency on million-plus items.
Auditability	The ability to trace every answer back to the concrete source documents (or passages) that

Term	Definition
	contributed to it, satisfying regulatory, compliance, and debugging requirements.
BM25	A classic sparse retrieval scoring function that ranks documents by term frequency, inverse document frequency, and document length; it is the de-facto baseline for lexical search.
Chunking	The process of breaking raw texts (PDFs, HTML, tables) into manageable passages—typically 200-500 tokens—so that each chunk can be embedded, indexed, and retrieved independently.
Citation	A reference (often a document ID, title, or URL) attached to a generated fact, indicating which retrieved passage supplied the evidence for that claim.
Compliance	Legal or policy constraints (e.g., GDPR, industry-specific regulations) that require answers to be verifiable, traceable, and up-to-date.
Cross-Encoder	A re-ranking model that jointly encodes a query and a candidate passage (instead of encoding them separately) and outputs a fine-grained relevance score.
Dense Retriever	A retrieval component that maps queries and passages into a shared dense vector space (using models like Sentence-Transformers) and performs ANN search for rapid candidate selection.
Document Store	The persistent backend that holds raw documents, their metadata, and (often) their vector embeddings; it enables fast lookup and filtering during retrieval.
Embedding	A fixed-length numeric representation of a text snippet (sentence, paragraph, or whole document) produced by a neural encoder; embeddings are the substrate for vector search.

Term	Definition
Exact Match (EM)	An evaluation metric that counts a prediction as correct only when it exactly matches the gold answer string; useful for measuring strict factual correctness.
F1	The harmonic mean of precision and recall for token-level overlap between a model's answer and the reference answer; it balances partial correctness and completeness.
Grounding	The practice of anchoring generated text in external evidence (retrieved passages) so that every factual claim can be verified against a source.
Hallucination	The phenomenon where a language model produces statements that are fluent but not supported by any retrieved evidence-or are outright false.
Hybrid Search	A retrieval strategy that combines sparse (e.g., BM25) and dense scores-often via a weighted sum-to exploit both lexical matching and semantic similarity.
Latency	The total elapsed time from receiving a user query to returning a final answer; in production RAG systems it includes embedding, ANN search, re-ranking, and LLM inference.
Large Language Model (LLM)	A deep-learning model (e.g., GPT-4, LLaMA-2, Claude) trained on massive text corpora that can generate fluent natural-language output given a prompt.
Metadata Filtering	The ability to restrict retrieval results by structured attributes (e.g., source type, publication date, security label) without re-embedding the data.
Prompt Engineering	The craft of designing the textual instruction (including context passages, role description, and citation directives) that conditions an LLM to produce the desired, grounded output.
Retrieval	The act of converting a user query into a set

Term	Definition
	of candidate documents or passages, typically via a dense or sparse retriever, before any generation occurs.
Retrieval-Augmented Generation (RAG)	A hybrid architecture where a retriever supplies evidence to a generator, enabling the language model to produce up-to-date, citation-rich answers while mitigating hallucinations.
Retrieval Recall @k	The proportion of gold-standard relevant documents that appear in the top-k retrieved set; a key indicator of how well the retriever covers the needed knowledge.
Ranker	A second-stage component that re-orders the candidate set from the retriever using a more expensive, interaction-aware model (often a cross-encoder) to surface the most relevant passages.
Sparse Retriever	A lexical search engine (e.g., BM25 or Elastic [®] kNN) that relies on term-frequency statistics rather than learned vector representations to retrieve documents.
Vector Database	A specialized storage system (e.g., Pinecone, Weaviate, Qdrant, Milvus) optimized for indexing and ANN search over high-dimensional embeddings, often with built-in metadata filters.
Vector Store	A generic term for any backend that holds vectors alongside their identifiers and metadata; it may be a full-featured vector DB or a simple in-memory index for prototypes.

These 25 terms capture the core technical vocabulary needed to understand, design, and evaluate production-ready Retrieval-Augmented Generation systems.

IMPORTANT DISCLAIMER

AI-Generated Content Notice

This document has been entirely generated by artificial intelligence technology through the Pustakam Injin platform. While significant effort has been made to ensure accuracy and coherence, readers should be aware of the following important considerations:

- The content is produced by AI language models and may contain factual inaccuracies, outdated information, or logical inconsistencies.
- Information should be independently verified before being used for critical decisions, academic citations, or professional purposes.
- The AI may generate plausible-sounding but incorrect or fabricated information (known as "hallucinations").
- Views and opinions expressed do not necessarily reflect those of the creators, developers, or any affiliated organizations.
- This content should not be considered a substitute for professional advice in medical, legal, financial, or other specialized fields.

Intellectual Property & Usage

This document is provided "as-is" for informational and educational purposes. Users are encouraged to fact-check, cross-reference, and critically evaluate all content. The Pustakam Injin serves as a knowledge exploration tool and starting point for research, not as a definitive source of truth.

Quality Assurance

While the Pustakam Injin employs advanced AI models and formatting techniques to produce professional-quality documents, no warranty is made regarding completeness, reliability, or accuracy. Users assume full responsibility for how they use, interpret, and apply this content.

Generated by: **Pustakam Injin**

Date: February 23, 2026 at 08:12 PM

For questions or concerns about this content, please refer to the Pustakam Injin documentation or contact the platform administrator.